شرح مجلد `lib/core`

تطبيق «بالحكمة» — Flutter / Clean Architecture

توثيق تفصيلي لكل ملف داخل الطبقة الأساسية

عدد الملفات: ٢٢٤ ملفًا — إجمالي الأسطر: ~١٥,١٦٣ سطر

الفرع: `before_ios` — التاريخ: ٢٠٢٦/١٠/٢١

الفهرس

1. نظرة عامة على بنية `lib/core`
2. `app/` — تهيئة التطبيق والتفضيلات العامة (٤ ملفات)
3. `constants/` — الثوابت والمسارات والأصول (٤ ملفات)
4. `di/` — حاقن التبعيات (ملف واحد)
5. `drawer/` — القائمة الجانبية (١٠ ملفات)
6. `enums/` — التعدادات (٣٢ ملفًا)
7. `interceptors/` — معترضات Dio (٢ ملفان)
8. `models/` — النماذج المشتركة (٢١ ملفًا)
9. `network/` — طبقة الشبكة (٦ ملفات)
10. `routing/` — التوجيه والمسارات (٧ ملفات)
11. `services/` — الخدمات (٤٣ ملفًا)
12. `themes/` — الثيمات والتنسيق (٦ ملفات)
13. `utils/` — المساعدات (١٨ ملفًا)
14. `widgets/` — الودجات المشتركة (٧٠ ملفًا)

١ — نظرة عامة على بنية `lib/core`

مجلد `lib/core` هو «العمود الفقري» للتطبيق: كل ما هو مشترك بين أكثر من ميزة (feature) يسكن هنا. القاعدة المعمارية المتبعة صارمة: **الواجهة غبية** (تعرض الحالة وتمرّر الأحداث فقط)، و**الـ Cubit يملك السلوك** (التحميل، الفلترة، مفاتيح الأخطاء)، و**الـ Data Source يملك الاتصال بالشبكة والتخزين**. لا يوجد في المشروع طبقة `domain/` ولا `use_cases/` — الـ Repo رقيق جدًا ويفوّض للـ Data Source فقط.

المجلد	المسؤولية	الملفات
`app/`	إقلاع التطبيق، مراقب الـ Bloc، تفضيلات المستخدم العامة (ثيم/خط/تبويب)	٤
`constants/`	عناوين الـ API، مسارات الأصول، مفاتيح الكاش، قائمة الدول	٤
`di/`	تسجيل كل الـ Cubits والـ Repos والـ Data Sources في GetIt	١
`drawer/`	القائمة الجانبية كميزة مصغّرة كاملة (refactor + widgets)	١٠
`enums/`	كل التعدادات المشتركة، ومعها امتدادات تحويل من/إلى JSON وعناوين مترجمة	٣٢
`interceptors/`	حقن التوكن وتحديثه تلقائيًا، وحقن بصمة الجهاز في كل طلب	٢
`models/`	النماذج التي تستخدمها أكثر من ميزة (الدرس، المادة، الجلسة، التسجيل...)	٢١
`network/`	عميل الـ API، مغلّف الاستجابة، قارئ JSON الآمن، الصفحنة	٦
`routing/`	جدول المسارات، مولّد الصفحات، وسائط الشاشات، وجهة الإشعارات	٧
`services/`	مشغّل الصوت، مشغّل الفيديو، الجلسات، الإشعارات، منع التصوير، الجهاز...	٤٣
`themes/`	الألوان، الثيم الفاتح/الداكن، أنماط النصوص، المسافات، الحركة	٦
`utils/`	الترجمة، السجل، التحقق، الكاش، تنسيق الأحجام والأوقات، الاتجاه	١٨
`widgets/`	مكتبة الودجات المشتركة: الأزرار، الحقول، البطاقات، الهياكل العظمية...	٧٠

تدفّق البيانات النموذجي في هذا المشروع:

```
Widget (tap) → Cubit.method() → Repo → DataSource → ApiClient (Dio + Interceptors)
    ← ApiEnvelope/JsonReader → Model → Cubit.emit(state) → Widget rebuild
```

٢ — مجلد `app/` — إقلاع التطبيق والتفضيلات العامة

هذا المجلد يحتوي على ما يُنفَّذ قبل ظهور أول شاشة، وعلى الـ Cubit الوحيد الذي يعيش مع التطبيق من أوله لآخره (`AppPreferencesCubit`) والذي يتحكم بالثيم وحجم الخط والتبويب المختار.

app_bootstrap.dart ١١٧ سطر

lib/core/app/app_bootstrap.dart

الدور: صندوق أدوات ثابت (`AppBootstrap._()` — لا يُنشأ منه كائن) يحمي مرحلة الإقلاع. الفلسفة المعتمدة فيه: **فشل خطوة واحدة لا يجوز أن يمنع التطبيق من الانطلاق** — يُسجَّل الفشل ويُكمل التطبيق بدونها.

- `stepTimeout = 12s` — المهلة القصوى لأي خطوة إقلاع؛ لو تجاوزتها خطوة (مثل تهيئة Firebase على شبكة ميتة) تُلغى بدل أن تُعلّق الشاشة البيضاء للأبد.
- `_failedSteps` + `failedSteps` (قراءة فقط) + `didStepFail(name)` — سجل بأسماء الخطوات التي فشلت، تستطيع أي شاشة لاحقًا أن تسأل: «هل فشلت خطوة الإشعارات؟» وتتصرف.
- `installErrorHandlers()` — يربط ثلاث شبكات أمان:
  - `FlutterError.onError` لأخطاء إطار Flutter (تُسجَّل، وتُعرض على الشاشة في وضع التطوير فقط).
  - `PlatformDispatcher.instance.onError` لأخطاء غير متزامنة خارج الإطار (تُسجَّل ويُرجَع `true` أي «تمّت المعالجة»).
  - `ErrorWidget.builder` — في وضع الإصدار فقط: بدل الشاشة الحمراء المرعبة يُرجع `SizedBox.shrink()`، أي فراغ غير مرئي، ويُسجَّل الخطأ بصمت.
- `reportFatal(error, stack)` — يُنادى من الـ `runZonedGuarded` في `main.dart` لأي خطأ يهرب من منطقة الإقلاع.
- `runStep(name, step, {timeout})` — يشغّل خطوة غير متزامنة داخل `try/catch` + `timeout`، ويفرّق في السجل بين «انتهت المهلة» و«رمت استثناء».
- `runSyncStep(name, step)` — نفس الشيء للخطوات المتزامنة.
- `tuneImageCache({isLowEndDevice})` — يضبط سعة كاش الصور حسب قوة الجهاز: **٢٤ ميغابايت / ٤٠ صورة** للأجهزة الضعيفة، و**٥٦ ميغابايت / ١٢٠ صورة** للأجهزة القوية. هذا مهم جدًا في تطبيق مليء بأغلفة الدروس والكتب لأن الكاش الافتراضي (١٠٠ ميغا) يقتل الأجهزة الضعيفة.

> الرقم `20 <<` يعني «اضرب في ١٠٢٤×١٠٢٤» أي تحويل الميغابايت إلى بايت بطريقة إزاحة البتات.

bloc/app_bloc_observer.dart ٢٤ سطرًا

lib/core/app/bloc/app_bloc_observer.dart

الدور: مراقب عالمي لكل الـ Cubits في التطبيق، يُسجَّل مرة واحدة في `main.dart` عبر `Bloc.observer = const AppBlocObserver()`.

- `onChange` — يطبع في سجل التطوير كل انتقال حالة بالشكل: `HomeCubit -> HomeLoaded`. يستخدم `dart:developer.log` مع `name: 'bloc'` فتستطيع تصفية السجل في DevTools.
- `onError` — يطبع «`XCubit failed`» مع الخطأ وتتبّع المكدس، ثم ينادي `super` ليستمر السلوك الافتراضي.

> فائدته الحقيقية في التصحيح: لو انبنت شاشة بحالة خاطئة، سجل الـ observer يخبرك بالترتيب الزمني الفعلي للحالات.

cubit/app_preferences_cubit.dart ٧٧ سطرًا

lib/core/app/cubit/app_preferences_cubit.dart

الدور: الـ Cubit الوحيد المسجَّل كـ `lazySingleton` ويعيش طول عمر التطبيق. يملك أربع مسؤوليات: **الثيم**، **مقياس الخط**، **التبويب المختار في الـ Shell**، و**سلوك زر الرجوع**.

الحالة والخصائص

- `isDark` — يُقرأ ابتداءً من الكاش (`CacheKeys.isDark`) وإلا `false`، أي الوضع الفاتح افتراضيًا.
- `fontScale` — يُقرأ من `SettingsDataSource.readFontScale()`، فالقراءة من التخزين تبقى مسؤولية الـ Data Source لا الـ Cubit.
- `selectedTab` — التبويب الحالي، يبدأ من `AppTab.home`.
- `visitedTabs` — مجموعة التبويبات التي زارها المستخدم فعلًا. تُستخدم مع `IndexedStack` لبناء التبويب عند أول زيارة فقط (lazy) بدل بناء الأربعة مع الإقلاع.
- `tabsEpoch` — عدّاد «حقبة». يُزاد عند `resetTabs()` فيستخدم في مفاتيح الودجات (`Key`) لإجبار Flutter على التخلص من الشجرة القديمة بالكامل بعد تسجيل الخروج. هذا هو الحل الجذري لمشكلة «بيانات المستخدم السابق تظهر للمستخدم الجديد».

الدوال

- `changeTheme(bool)` — يحدّث الحالة **ويكتب في الكاش** ثم يبث `AppPreferencesThemeChanged`.
- `applyFontScale(double)` — يتجاهل القيمة إن كانت مطابقة للحالية (حماية من إعادة بناء بلا داعٍ) ولا يكتب في الكاش (الكتابة مسؤولية شاشة الإعدادات).
- `selectTab(AppTab)` — يتجاهل التبويب نفسه، ويضيفه إلى `visitedTabs`.
- `resetTabs()` — يُنادى بعد تسجيل الخروج: يزيد الحقبة، يفرّغ التبويبات المزارة، ويرجع إلى الرئيسية.
- `handleBackPress() → ShellBackAction` — المنطق الشهير «اضغط رجوع مرتين للخروج»:
  • إن لم نكن في التبويب الرئيسي ⇒ نعود إليه ونُرجع `none` (أي لا تخرج).
  • إن كنا في الرئيسية وآخر ضغطة كانت خلال `exitWindow` (ثانيتان) ⇒ `exit`.
  • وإلا ⇒ `confirmExit` فتُظهر الواجهة رسالة «اضغط مرة أخرى للخروج».

> لاحظ الالتزام المعماري: الـ Cubit يُرجع **قرارًا** (`ShellBackAction`) ولا يلمس `BuildContext` ولا يُظهر Toast ولا ينتقل بين الشاشات — الواجهة هي من تترجم القرار.

cubit/app_preferences_state.dart ١٢ سطرًا

lib/core/app/cubit/app_preferences_state.dart

الدور: ملف `part` يعرّف حالات الـ Cubit السابق باستخدام `sealed class` (ميزة Dart 3) مع `@immutable`:

- `AppPreferencesInitial` — الحالة الأولى.
- `AppPreferencesTabChanged` / `AppPreferencesFontScaleChanged` / `AppPreferencesThemeChanged` — حالات «إشارة» بلا بيانات.

> هذا نمط *signal states*: الحالة لا تحمل البيانات، بل البيانات موجودة كخصائص عامة في الـ Cubit نفسه (`selectedTab`, `isDark`, ...). ميزته أن `BlocBuilder` يستطيع التصفية بدقة حسب نوع الحالة (`buildWhen: (p, c) => c is AppPreferencesThemeChanged`) فلا يُعاد بناء شريط التبويبات عند تغيير حجم الخط. كون الكلاس `sealed` يجعل `switch` عليه شاملًا ويُنبّه المحلّل لو أُضيفت حالة ونُسيت معالجتها.

٣ — مجلد `constants/` — الثوابت والمسارات والأصول

api_endpoints.dart ٨٣ سطرًا

`lib/core/constants/api_endpoints.dart`

الدور: المصدر الوحيد للحقيقة لكل عنوان يتحدث معه التطبيق. لا يجوز أن يظهر نص مسار مثل `/user/...` في أي ملف آخر.

- `baseUrl = https://bilhikma.tech/api` — يُمرَّر لـ `Dio` في `DioService`.
- **مسارات عامة (بلا توكن):** `publicEntities` للجهات التعليمية، `appVersion` للتحقق من وجود تحديث إجباري، و `publicEntityCategories(entityId)` كدالة لأنها تحتاج معرّفًا.
- **المصادقة:** `login` , `refreshToken` , `logout`.
- **استعادة كلمة المرور:** `forgotPassword` (طلب الرمز)، ويقابله `sendPasswordOtp` `/verify-otp` (التحقق من الرمز)، ويقابله `setNewPassword` `/reset-password`.
- **المستخدم:** `profile` , `profilePassword` , `myEnrollments` , `activeContext` (الجهة/المسار النشط حاليًا)، `enrollments` (طلب تسجيل جديد).
- **المواد والمحتوى:** `subjects` , `subject(id)` , `content` , `contentDetail(id)` , `contentProgress(id)` لحفظ موضع المشاهدة.
- **رموز الوصول للوسائط:** `contentAccessToken(contentId)` و `attachmentAccessToken(contentId, attachmentId)` و `mediaAlbumItemAccessToken(albumId, itemId)`. هذه هي أساس حماية المحتوى: التطبيق لا يحصل على رابط مباشر للفيديو بل على رمز مؤقت يُستبدل برابط عبر `protectedMedia(mediaToken)`.
- **الكتب:** `textbooks` , `textbookDownload(id)`.
- **الألبومات:** `mediaAlbums` , `mediaAlbum(id)`.
- **البث المباشر:** `liveSessions` , `liveSession(id)` , `liveSessionAttendance(id)` لتسجيل الحضور، و `liveSessionHeartbeat(id)` لنبضة دورية تؤكد أن الطالب ما زال متابعًا.
- **الإشعارات:** `notifications` , `notificationRead(id)` , و `deviceFcmToken` لرفع رمز Firebase.
- `securityEvents` — لرفع الأحداث الأمنية (محاولة تصوير الشاشة مثلًا) إلى الخادم.

> المسارات الثابتة تُعرَّف كـ `static const String` والمسارات ذات المعرّف كدوال `static String f(int)` — تمييز بسيط يمنع أخطاء الدمج النصي اليدوي.

app_assets.dart ١٣٥ سطرًا

lib/core/constants/app_assets.dart

الدور: كتالوج نصي لكل ملف في مجلد `assets/`، فلا يُكتب مسار صورة كنص حر في أي ودجة (الخطأ المطبعي في النص لا يظهر إلا وقت التشغيل، أما هنا فالمحلّل يكتشفه).

المحتوى مقسّم ضمنيًا إلى عائلات:

- **الهوية البصرية:** `assetsAppIcon`، `assetsAppIconSplash`، `assetsLogoText`، `assetsLeftPartLogo`، `assetsRightPartLog`، `assetsBilhikmaTextInTheHomeLogo`.
- **الزخارف والأقنعة:** `assetsFullMask`، `assetsMaskBottomLeft`، `assetsOrnamentRight`، `assetsLogoOrnament`، `assetsBackgroundImageLogo` — وهي التي تُستعمل في `OrnamentedCard` و`CardOrnament` لإعطاء الطابع الزخرفي.
- **أيقونات التبويبات:** `assetsHome`، `assetsSubjects`، `assetsLive`، `assetsProfile`، `assetsNotification` (نسخ SVG ومقابلاتها بصيغة PNG).
- **أنواع المحتوى:** `assetsVideos`، `assetsAudios`، `assetsArticles`، `assetsBookIcon`، `assetsClassIcon`.
- **حالات الحساب:** `assetsAccountAccepted`، `assetsAccountRejected`، `assetsPending`، `assetsYourRequestAccepted`.
- **صور الإشعارات** (واحدة لكل نوع إشعار، تُربط في `NotificationKind`): `assetsLiveSessionNotification`، `assetsInstantReminderNotification`، `assetsApprovalOfRegistrationNotification`، `assetsAccountStatusNotification`، `assetsWarningNotification`، `assetsAdministrativeNotice`، `assetsCustomNotification`، `assetsSpecificNotification`.
- **ملفات الترجمة:** `assetsTranslationsAr`، `assetsTranslationsEn`.

> لاحظ وجود أخطاء مطبعية مثبّتة في أسماء الملفات الأصلية: `buttom_left`، `start_palayer_icon`، `right_mask_backgroung`، `specifc_notification`. تصحيحها يتطلب إعادة تسمية الملف داخل `assets/` أيضًا.

cache_keys.dart ١٧ سطرًا

lib/core/constants/cache_keys.dart

الدور: كل مفاتيح `SharedPreferences` و`FlutterSecureStorage` في مكان واحد، لأن تكرار النص نفسه في عشرة ملفات هو وصفة مؤكدة لخطأ صامت.

- **التفضيلات:** `isDark`، `fontScale`، `sleepTimerMinutes`.
- **«تذكّرني»:** `rememberMeKey`، `rememberedIdentifierKey` (البريد أو الهاتف)، `rememberedMethodKey` (أي طريقة دخول استُخدمت).
- **الكاش:** `cachedUserKey` (بيانات المستخدم كـ JSON لعرضها فورًا قبل وصول الشبكة)، `pdfReadingPositions` (آخر صفحة في كل كتاب)، `pdfPageLayout` (صفحة واحدة أم صفحتان).
- **الأسرار (تذهب للتخزين الآمن لا للتفضيلات):** `tokenKey`، `refreshTokenKey`، `deviceUuidKey`.

app_countries.dart ٥٠٤ أسطر

lib/core/constants/app_countries.dart

الدور: قاعدة بيانات دول مدمجة في التطبيق (بلا شبكة) تخدم حقل رقم الهاتف ومنتقي الدول.

- `defaultIsoCode` قيمته `SY` و`defaultCountry` — سوريا هي الافتراضي، مع احتياط يُرجع أول دولة في القائمة حتى لا تكون النتيجة فارغة أبدًا.
- `byIso(isoCode)` — بحث خطّي يرفع النص لحالة الأحرف الكبيرة أولًا، ويُرجع `Country?`.
- `parse(fullNumber)` — الدالة الذكية في الملف، تُفكّك رقمًا دوليًا كاملًا إلى (الدولة + الرقم المحلي):
  ١. تنظيف كل ما ليس رقمًا أو إشارة زائد عبر تعبير نمطي.
  ٢. ترفض الرقم إن لم يبدأ بإشارة زائد أو بـ `00`.
  ٣. تحذف البادئة ثم تبحث عن **أطول** رمز اتصال مطابق — وهذه النقطة جوهرية: رمز أمريكا `1` ورمز `1809` لا يُلتبسان لأن الأطول يفوز.
  ٤. تُرجع `record` من Dart 3 بالشكل `({Country country, String nationalNumber})`.
- `all` — قائمة `const` بكل الدول، كل سجل يحمل: رمز ISO، رمز الاتصال، الاسم الإنجليزي، **الاسم العربي**، و`minLength` / `maxLength` لعدد أرقام الرقم المحلي.

> وجود `minLength` / `maxLength` لكل دولة هو ما يسمح بالتحقق الدقيق من طول الرقم بحسب الدولة المختارة، بدل قاعدة عامة واحدة فاشلة.

٤ — مجلد `di/` — حاقن التبعيات

service_locator.dart ٢٧٥ سطرًا

`lib/core/di/service_locator.dart`

الدور: نقطة التسجيل الوحيدة في `GetIt`. يُنادى مرة واحدة من `main.dart` عبر `await setupServiceLocator()`.

البنية

`setupServiceLocator()` لا تفعل شيئًا بنفسها، بل تنادي ٢١ دالة خاصة بالترتيب: `_initCore()` ثم ميزة ميزة (`_initAppUpdate`, `_initSplash`, `_initAuth`, `_initForgetPassword`, `_initHome`, `_initLessons`, `_initSidebar`, `_initSettings`, `_initNotifications`, `_initProfile`, `_initSearch`, `_initArchive`, `_initGallery`, `_initEducationalPathways`, `_initSubjects`, `_initSubjectContent`, `_initTextbooks`, `_initEnrollments`, `_initLiveSessions`, `_initSecurity`). التقسيم يجعل الملف قابلًا للقراءة رغم طوله، ويجعل إضافة ميزة = إضافة دالة واحدة ونداءها.

`_initCore()` — الأهم

- ينشئ `GlobalKey<NavigatorState>` ويسجّله كـ `singleton`. **لماذا؟** لأن الخدمات (مثل مُشغّل الإشعارات أو حارس الجلسة) تحتاج أن تنتقل بين الشاشات وهي لا تملك `BuildContext`، فتستخدم `currentState?.pushNamed(...)` من هذا المفتاح.
- يهيّئ `DioService` فقط إن لم يكن مهيّأً — تهيئة حمائية لا تتأثر بالنداء المتكرر.
- يسجّل `AppPreferencesCubit` كـ `lazySingleton` و`Dio` كـ `singleton`.

أنواع التسجيل المستخدمة ودلالتها

النوع	المعنى	يُستخدم هنا لـ
`registerSingleton`	يُنشأ فورًا ويبقى	`Dio`, `navigatorKey`, `SecurityCubit`
`registerLazySingleton`	يُنشأ عند أول طلب ويبقى	كل الـ Repos والـ Data Sources، و`ProfileCubit`, `NotificationsCubit`, `LessonSaveCubit`
`registerFactory`	كائن جديد في كل طلب	Cubits الشاشات: `HomeCubit`, `LoginCubit`, `SearchCubit` ...
`registerFactoryParam`	كائن جديد مع تمرير وسيط	`BookPreviewCubit(args)`, `LessonsCubit(categorySubjectId)`, `SubjectContentCubit(id)`, `SetPasswordCubit(args)`

قراءة معمارية مهمة: اختيار `lazySingleton` للـ `ProfileCubit` و`NotificationsCubit` و`LessonSaveCubit` مقصود: هذه حالات يجب أن تكون **مشتركة** بين عدة شاشات (صورة المستخدم تظهر في القائمة الجانبية وفي شاشة الحساب معًا، وأيقونة «محفوظ» يجب أن تتغير في كل القوائم فورًا). أما `HomeCubit` فهو `factory` لأن لكل زيارة للشاشة دورة حياة مستقلة.

`SubjectContentCubit` يستقبل ثلاثة Repos (`repo`, `lessonsRepo`, `textbooksRepo`) لأن شاشة محتوى المادة تجمع تبويبات من ثلاثة مصادر. و`SavedLessonsDataSource` و`BookDownloadsDataSource` و`EducationalPathwaysDataSource` تستقبل مصادر بيانات أخرى بالتركيب (composition) بدل الوراثة.

٥ — مجلد `drawer/` — القائمة الجانبية

القائمة الجانبية مبنية كـ «ميزة مصغّرة» كاملة داخل `core` لأنها مشتركة بين كل شاشات الـ Shell. تتبع نفس بنية الميزات: `presentation/refactor/` للمنطق التنظيمي و `presentation/widgets/` للمكوّنات الغبية. لا تملك Cubit خاصًا بها، بل تستهلك `ProfileCubit` المشترك.

presentation/refactor/app_drawer.dart ٤١ سطرًا

`lib/core/drawer/presentation/refactor/app_drawer.dart`

الدور: الغلاف الخارجي للقائمة. هو `StatefulWidget` لسبب واحد: الحاجة إلى `initState`.

- `late final ProfileCubit _profile = getIt<ProfileCubit>()` — يجلب الـ Cubit المشترك (لا ينشئ واحدًا جديدًا) ويحتفظ به في الحالة حتى لا يُعاد جلبه في كل إعادة بناء.
- في `initState` ينادي `unawaited(_profile.refresh())` — تحديث بيانات المستخدم في الخلفية كلما فُتحت القائمة. استخدام `unawaited` صريح يوضّح للقارئ ولمحلّل الكود أن عدم انتظار النتيجة **مقصود**.
- يحسب العرض بذكاء: `screenWidth * 0.8` ولكن بسقف `320` — فعلى الهواتف تأخذ القائمة ٨٠٪ من الشاشة، وعلى الأجهزة اللوحية تتوقف عند ٣٢٠ بكسل بدل أن تصبح لوحًا ضخمًا.
- يمرّر الـ Cubit للشجرة عبر `BlocProvider.value` (لا `BlocProvider` العادي، لأن الـ Cubit مشترك ولا يجوز أن تُغلقه القائمة عند إزالتها).
- `shape: RoundedRectangleBorder()` بلا نصف قطر — إلغاء للزوايا الدائرية الافتراضية في Material 3.

presentation/refactor/drawer_navigation.dart ٦٥ سطرًا

lib/core/drawer/presentation/refactor/drawer_navigation.dart

الدور: كل منطق التنقّل وتسجيل الخروج مجموع في كلاس ثابت، فتبقى الودجات غبية تمامًا.

- `open(context, destination)` — يُغلق القائمة أولًا عبر `Scaffold.of(context).closeDrawer()` ثم ينفّذ `switch` شاملًا على `DrawerDestination` ويدفع المسار المناسب (`archive`, `pictures`, `search`, `settings`, `profile`, `educationalPathways`).
- يستخدم `getIt<GlobalKey<NavigatorState>>()` للتنقل لا `Navigator.of(context)` — **وهذا متعمّد**: سياق القائمة الجانبية يُزال من الشجرة لحظة إغلاقها، فالتنقّل عبره قد يفشل. المفتاح العالمي يبقى صالحًا دائمًا.
- `logout(context)` — التسلسل الصحيح لتسجيل الخروج:

١. التقاط `ScaffoldState` **قبل** أي `await`.
٢. إظهار `LogoutConfirmDialog` وانتظار التأكيد؛ الخروج فورًا إن رفض المستخدم.
٣. إغلاق القائمة بشرط `scaffold.mounted`.
٤. `await DeviceSessionService.logout()`.
٥. `getIt<ProfileCubit>().reset()` لتفريغ بيانات المستخدم من الـ Cubit المشترك.
٦. `pushNamedAndRemoveUntil(AppRoutes.login, (route) => false)` — مسح كامل لمكدس الشاشات فلا يستطيع زر الرجوع العودة لمحتوى محمي.

> يوجد في الملف تعليق يشرح **سبب** الانتظار (`await`) في الخطوة الرابعة: لو تُركت المكالمة تعمل وحدها، تبقى الجلسة صالحة بقدر زمن الطلب، وأي استئناف للتطبيق أو إشعار يصل في تلك النافذة يُرفض بـ 401 يُعلنه المعترض كـ «انتهت صلاحية الجلسة» — وهو ظهور خطأ مزعج في توقيت خاطئ. هذا نمط تعليق جيد: يشرح القرار لا الكود.

presentation/widgets/drawer_content.dart ٤٨ سطرًا

lib/core/drawer/presentation/widgets/drawer_content.dart

الدور: التخطيط الرأسي للقائمة في ثلاث مناطق:

- `DrawerHead()` — ثابت في الأعلى.
- `Expanded` يحوي `SingleChildScrollView` فيه `DrawerMenuList` — فقائمة العناصر هي وحدها القابلة للتمرير إن طالت أو صغرت الشاشة.
- `SafeArea(top: false)` يحوي زر «تسجيل الخروج» بعرض كامل — تعطيل الحماية العليا فقط لأن الحماية السفلية (شريط الإيماءات) هي المطلوبة هنا.

كل المسافات بوحدات `flutter_screenutil` (`.h`, `.w`, `.r`) لتتناسب مع كل المقاسات، والنصوص عبر `tr()`.

presentation/widgets/drawer_head.dart ٣٣ سطرًا

lib/core/drawer/presentation/widgets/drawer_head.dart

الدور: يدمج ترويسة العلامة التجارية مع بطاقة المستخدم بتقنية «البطاقة المتدلّية»:

- `Stack` فيه `DrawerBrandHeader` مع `padding bottom: 34.h` — أي يُحفظ فراغ في الأسفل.
- `PositionedDirectional(start, end, bottom: 0)` يضع `DrawerProfileCard` في ذلك الفراغ فتبدو البطاقة نصفها فوق الترويسة الملوّنة ونصفها على الخلفية.

> استخدام `PositionedDirectional` بدل `Positioned` حاسم لدعم العربية: `start` و`end` تنقلبان تلقائيًا بين RTL و LTR، أما `left` / `right` فلا.

presentation/widgets/drawer_brand_header.dart ٦١ سطرًا

lib/core/drawer/presentation/widgets/drawer_brand_header.dart

الدور: الترويسة الملوّنة أعلى القائمة: خلفية بلون `colorScheme.secondary` بزوايا سفلية دائرية `30.r`، مع `clipBehavior: Clip.antiAlias` حتى لا تتجاوز الزخرفة الزوايا.

- `Positioned.fill` يحوي `OrnamentedCard` بلون `primary` — طبقة الزخرفة الخلفية.
- فوقها `Padding(80.h)` يحوي شعار النص (`AppIcon` مع `assetsLogoText`) بلون `tertiary`، ثم شعار المنصة النصي من مفتاح الترجمة `platform_tagline` بنمط `brandTagline`.
- النص محمي بـ `maxLines: 2` و`overflow: ellipsis` فلا يتجاوز الترويسة في أي لغة.

> ملاحظة تحسين: `OrnamentedCard(child: Column(children: []))` يمرّر عمودًا فارغًا فقط لتفعيل الزخرفة — استدعاء أنظف سيكون بإتاحة `child` اختياري في `OrnamentedCard`. كما أن النص يستخدم `'platform_tagline'.tr()` مباشرة بدل ثابت في `LangKeys`.

presentation/widgets/drawer_menu_list.dart ٤٦ سطرًا

lib/core/drawer/presentation/widgets/drawer_menu_list.dart

الدور: يبني عناصر القائمة من `DrawerDestination.values` — أي أن إضافة وجهة جديدة في التعداد تظهر في القائمة تلقائيًا بلا تعديل هنا.

- حلقة `for` داخل قائمة الأبناء (spread operator) تنتج لكل وجهة: فاصلًا اختياريًا ثم `DrawerMenuItem`.
- `_startsNewGroup(destination)` — دالة ثابتة خاصة تُرجع `true` إذا كانت مجموعة الوجهة السابقة مختلفة عن الحالية، فيُرسم `Divider` بين المجموعات. المجموعة نفسها معرّفة داخل التعداد (`destination.group`).
- العنصر الأول لا يسبقه فاصل أبدًا (`if (index == 0) return false`).
- الفاصل بلون `outline` بشفافية ٢٥٪ وبهوامش جانبية `12.w` — فاصل ناعم لا يقطع الشاشة.
- الودجة غبية بالكامل: تستقبل `onDestinationTap` كـ `ValueChanged<DrawerDestination>` ولا تعرف شيئًا عن التنقّل.

presentation/widgets/drawer_menu_item.dart ٤٦ سطرًا

lib/core/drawer/presentation/widgets/drawer_menu_item.dart

الدور: صف واحد في القائمة: أيقونة + عنوان، قابل للنقر عبر `InkWell` (فيظهر تأثير التموّج).

- الأيقونة والعنوان يأتيان من التعداد نفسه: `destination.icon` و`destination.label.tr()` — فالودجة لا تحتوي أي `switch`.
- الأيقونة بلون `colorScheme.secondary` وحجم `22.w`.
- العنوان داخل `Expanded` مع `textAlign: TextAlign.start` (ينقلب مع الاتجاه) و`maxLines: 1` و`ellipsis`، ونمطه `AppTheme.styles(context).rowTitle`.

presentation/widgets/drawer_profile_card.dart ٦٣ سطرًا

lib/core/drawer/presentation/widgets/drawer_profile_card.dart

الدور: البطاقة المتدلّية التي تعرض صورة المستخدم واسمه وتفتح شاشة الحساب عند النقر.

- `Material` شفاف + `InkWell` + `clipBehavior: Clip.antiAlias` — الترتيب الصحيح ليقتصر تأثير التموّج على الزوايا الدائرية.
- المحتوى داخل `OrnamentedCard` بنصف قطر `14.r`.
- **الأهم معماريًا:** يوجد `BlocBuilder` منفصلان:
  • الأول للصورة مع `buildWhen` يقارن `avatarUrl` فقط.
  • الثاني للاسم مع `buildWhen` يقارن `name` فقط.
  النتيجة: تغيير الصورة لا يعيد بناء النص، وتغيير الاسم لا يُعيد تحميل الصورة من الشبكة. هذا هو الاستخدام الصحيح والدقيق لـ `buildWhen`.
- الاسم بنمط `cardTitle` ملوّن بـ `secondaryContainer` مع تسطير متقطّع (`TextDecorationStyle.dashed`) كإشارة بصرية إلى أنه قابل للنقر.

presentation/widgets/drawer_profile_monogram.dart ٣٧ سطرًا

lib/core/drawer/presentation/widgets/drawer_profile_monogram.dart

الدور: صورة المستخدم الدائرية مع بديل أنيق عند غيابها.

- حجم ثابت `52.w` داخل `ClipOval` وخلفية `secondaryContainer`.
- إن كان الرابط `null` أو فارغًا ⇒ يُعرض `DrawerProfileInitials` (الأحرف الأولى) مباشرة بلا أي محاولة شبكة.
- إن وُجد الرابط ⇒ `AppNetworkImage` مع **نفس** `placeholder` ودجة الأحرف الأولى في و`errorWidget` — فلا تظهر فجوة فارغة ولا أيقونة «صورة مكسورة» في أي حالة.
- `memCacheWidth: (size * devicePixelRatio).round()` — تحسين مهم: الصورة تُفكّك في الذاكرة بحجم العرض الفعلي بالبكسلات الفيزيائية فقط، لا بحجمها الأصلي. صورة ١٠٢٤×١٠٢٤ لا تحتل ذاكرة ٤ ميغابايت لأجل دائرة ٥٢ نقطة.

presentation/widgets/drawer_profile_initials.dart ٢٤ سطرًا

lib/core/drawer/presentation/widgets/drawer_profile_initials.dart

الدور: نص الأحرف الأولى من اسم المستخدم كبديل للصورة.

- `BlocBuilder` مع `buildWhen` على `profile.initials` فقط.
- الأحرف الأولى محسوبة في الـ **Cubit/الموديل** (`state.profile.initials`) لا في الودجة — التزام بقاعدة «لا حسابات في الواجهة».
- النمط `cardTitle` بلون `tertiary`، و `maxLines: 1`.

presentation/widgets/logout_confirm_dialog.dart ١٣٧ سطرًا

lib/core/drawer/presentation/widgets/logout_confirm_dialog.dart

الدور: حوار تأكيد تسجيل الخروج، مع نمط ممتاز لإدارة النتيجة.

- `static Future<bool> show(context)` — دالة ثابتة تُظهر الحوار وتُرجع `bool` دائمًا بفضل `?? false`. فإذا أغلق المستخدم الحوار بالنقر خارجه تكون النتيجة «لا» — السلوك الآمن الصحيح. المنادي لا يتعامل مع `showDialog` ولا مع `null` أبدًا.
- الحوار نفسه: `Dialog` بخلفية `surface`، `elevation: 0`، زوايا `20.r`، وهوامش جانبية `28.w`.
- المحتوى: `_LogoutHeader` ثم نص التأكيد (`logout_confirm`) بتباعد أسطر `1.7` للقراءة المريحة بالعربية، ثم `_LogoutActions`.
- `_LogoutHeader` — ودجة خاصة: مربع `38.w` بزوايا `10.r` ولون `secondary` يحوي أيقونة `logout_rounded`، وبجانبه العنوان بنمط `sectionTitle`.
- `_LogoutActions` — صف فيه زرّان متساويان (`Expanded`) لكل منهما): `CustomButton` للتأكيد يُرجع `true`، و `CustomOutlineButton` للإلغاء يُرجع `false`.

> تقسيم الملف إلى ثلاث كلاسات (الحوار + ترويسة خاصة + أزرار خاصة) بدل دوال `_buildHeader()` هو الأسلوب الصحيح في Flutter: كل كلاس يحصل على مكانه في شجرة الودجات ويُعاد بناؤه وحده.

٦ — مجلد `enums/` — التعدادات (٣٢ ملفًا)

التعدادات في هذا المشروع ليست مجرد قوائم أسماء، بل **تحمل سلوكها معها** (enhanced enums في Dart 3): كل تعداد يحمل مفتاحه النصي للـ JSON، ومفتاح ترجمته، وأيقونته، ودوال التحويل والاستعلام. الفائدة المعمارية: الواجهة لا تحتوي أي `switch` على نوع، والـ Data Source لا تحتوي أي `if (type == 'video')`. وكل تعداد يُقرأ من الخادم يملك `fromJson` تُرجع قيمة افتراضية آمنة بدل أن ترمي استثناء لو أضاف الخادم نوعًا جديدًا.

٦٫١ — تعدادات التنقّل والواجهة العامة

app_tab_enum.dart ١٤ سطرًا

lib/core/enums/app_tab_enum.dart

التبويبات الخمسة في شريط التنقّل السفلي، كل واحدة تحمل `label` (مفتاح ترجمة) و`icon` (مسار أصل).

- الترتيب مقصود: `subjects` , `live` , `home` , `saved` , `account` — أي أن **الرئيسية في الوسط**، وهو التصميم الشائع لإبراز التبويب الأهم.
- لأن الأيقونة والعنوان داخل التعداد، فإن `AppBottomNavBar` يبنيها بحلقة على `AppTab.values` بلا أي شرط.

drawer_destination_enum.dart ٢٨ سطرًا

lib/core/enums/drawer_destination_enum.dart

وجهات القائمة الجانبية الستّ. كل وجهة تحمل ثلاثة حقول: `label` و`icon` (من `Icons` مباشرة) و`group` .

- **المجموعة ٠:** `account` (حساب الطالب)، `learningPaths` (مساراتي التعليمية).
- **المجموعة ١:** `saved` , `pictures` , `search` , `settings` .
- حقل `group` هو ما يستخدمه `DrawerMenuList._startsNewGroup` لرسم الفواصل تلقائيًا — فتغيير التجميع البصري يصبح تغيير رقم هنا فقط.

theme_option_enum.dart ٢١ سطرًا

lib/core/enums/theme_option_enum.dart

خياران فقط: `light` و`dark` ، كل منهما يحمل `label` و`icon` و`isDark` .

- `ThemeOption.of(bool isDark)` — جسر بين تمثيل الـ Cubit (قيمة منطقية) وتمثيل الواجهة (تعداد).

> لا يوجد خيار «حسب النظام» (`system`) — قرار تصميمي: التطبيق يفرض اختيارًا صريحًا للمستخدم.

language_option_enum.dart ١٩ سطرًا

lib/core/enums/language_option_enum.dart

اللغتان المدعومتان: `arabic('ar')` و`english('en')` .

- `locale` — خاصية محسوبة تُرجع `Locale(code)` جاهزة لـ `EasyLocalization` .
- `LanguageOption.of(code)` — تحويل آمن من نص، و**الاحتياط هو العربية** لأنها اللغة الأساسية للمشروع.

settings_group_enum.dart ٨ أسطر

lib/core/enums/settings_group_enum.dart

مجموعتان في شاشة الإعدادات: `audio` بعنوان `audio_playback` ، و`general` .

settings_action_enum.dart ٤٠ سطرًا

lib/core/enums/settings_action_enum.dart

أربعة إجراءات في الإعدادات: `fontSize` و`sleepTimer` (مجموعة الصوت)، `clearDownloads` و`shareApp` (عامة). كل إجراء يحمل عنوانه وصورته ومجموعته.

- `SettingsAction.of(group)` — دالة ثابتة تُرجع إجراءات مجموعة واحدة عبر `where().toList(growable: false)`. فشاشة الإعدادات تبني مجموعاتها بحلقة على `SettingsGroup.values` ثم حلقة على `SettingsAction.of(group)` — بلا أي قائمة مكتوبة يدويًا.
- استخدام `growable: false` يوضّح النية (قائمة للقراءة فقط) ويوفّر قليلًا من الذاكرة.

> **خرق معماري:** هذا الملف يستورد `package:bilhikma/core/constants/app_assets.dart` بينما القاعدة في المشروع تمنع الاستيراد بالحزمة داخل `lib/`. الصحيح: `import '../constants/app_assets.dart';` — وهو الملف الوحيد الذي رصدنا فيه هذا الخرق في `core`.

٦٫٢ — تعدادات المحتوى والدروس

content_type_enum.dart ٢٧ سطرًا

lib/core/enums/content_type_enum.dart

النوع الأساسي لأي محتوى: `video`، `audio`، `article`. من أكثر التعدادات استخدامًا في المشروع.

- `isVideo`، `isAudio`، `isArticle` — بدائل مقروءة عن المقارنة المباشرة.
- `hasTimeline` و`isPlayable` — كلاهما يعني «ليس مقالًا». الفصل بين الاسمين مفيد للتعبير: الأول يُستخدم في منطق شريط التقدّم، والثاني في منطق المشغّل.
- `fromJson` — يُحوّل النص لحالة صغيرة ويبحث، وافتراضه `video`.

lesson_status_enum.dart ٢٣ سطرًا

lib/core/enums/lesson_status_enum.dart

حالة الدرس بالنسبة للطالب: `available`، `inProgress`، `locked`، `completed`.

- `resolve({isCompleted, isStarted, isLocked})` — الدالة الأهم: تحوّل ثلاث قيم منطقية قادمة من الخادم إلى حالة واحدة، **بترتيب أولوية صريح**: مكتمل ← مقفل ← قيد التقدّم ← متاح. ترتيب «المكتمل قبل المقفل» مقصود: الدرس الذي أنهاه الطالب يظل يظهر كمكتمل حتى لو أُقفل الوصول لاحقًا.
- `showsProgress`، `isLocked`، `isCompleted` — تقرأها الودجات (`LessonStatusBadge`، `LessonLockScrim`) مباشرة.

> وجود `resolve` هنا يمنع تكرار منطق «أي شارة أعرض؟» في كل قائمة دروس في التطبيق.

lesson_category_enum.dart ١١ سطرًا

lib/core/enums/lesson_category_enum.dart

تبويبات محتوى المادة: `lessons`، `live`، `books`، `pictures`. يحمل كل عنصر `key` نصيًا يُرسل للخادم كمعامل تصفية.

lessons_sort_enum.dart ٨ أسطر

lib/core/enums/lessons_sort_enum.dart

خياران للترتيب: `order` (ترتيب المنهج كما حدّده المعلم) و`publishedAt` (الأحدث أولًا). المفتاح النصي يُمرَّر لمعامل الاستعلام مباشرة.

archive_filter_enum.dart ٢٣ سطرًا

lib/core/enums/archive_filter_enum.dart

مُرشِّحات شاشة المحفوظات الثلاثة، وكل واحد **يحمل مرجعًا إلى** `ContentType` المقابل عبر الحقل `mediaType`.

- `label` و`imagePath` كخاصيتين محسوبتين بنمط `switch` تعبيري (Dart 3).

> نمط جيد: الربط بين «مرشّح واجهة» و«نوع محتوى في البيانات» معرّف مرة واحدة هنا، فلا تحتاج الواجهة لأي خريطة تحويل.

media_source_enum.dart ٢١ سطرًا

lib/core/enums/media_source_enum.dart

مصدر الوسيط: `file` (ملف على سيرفر المنصة) أو `youtube`. يحدّد هذا التعداد أي محرّك تشغيل سيُستخدم.

- `isFile` / `isYoutube`، و`fromJson` بافتراض `file`.

video_source_kind_enum.dart سطر واحد

lib/core/enums/video_source_kind_enum.dart

تعداد بسيط جدًا: `youtube` أو `network`. يُستخدم **داخل خدمة الفيديو** لاختيار التنفيذ المناسب (`YoutubeVideoPlayback` أو `NetworkVideoPlayback`). هو النسخة «الداخلية» من `MediaSource`.

book_page_layout_enum.dart ٢٤ سطرًا

lib/core/enums/book_page_layout_enum.dart

تخطيط قارئ الكتب: `single` (صفحة صفحة) أو `continuous` (تمرير متصل).

- `isSingle` و`toggled` — الأخيرة تُرجع الخيار الآخر، فزر التبديل في الواجهة يصبح سطرًا واحدًا: `emit(layout.toggled)`.
- `fromJson` مع افتراض `single` — يُستخدم لقراءة التفضيل المحفوظ في الكاش.

test_question_type_enum.dart ٢٩ سطرًا

lib/core/enums/test_question_type_enum.dart

ثمانية أنواع أسئلة للاختبارات: `singleChoice`، `multiChoice`، `mostCorrect`، `trueFalse`، `fillBlanks`، `typedBlanks`، `matchPairs`، `orderSentences`.

- `isMultiSelect` — هل يسمح باختيار أكثر من جواب (اختيار متعدّد فقط).
- `usesWordBank` — هل يُعرض «بنك كلمات» للاختيار منه (ملء الفراغات).
- `isTyped` — هل يكتب الطالب الجواب بنفسه.
- `fromJson` ينظّف النص بـ `toLowerCase().trim()` ويفترض `singleChoice`.

> هذه الخصائص الثلاث هي ما يسمح لشاشة السؤال أن تختار الودجة المناسبة بلا سلسلة شروط طويلة.

٦٫٣ — تعدادات التسجيل والحساب

enrollment_status_enum.dart ٢٧ سطرًا

lib/core/enums/enrollment_status_enum.dart

حالة طلب التسجيل في جهة تعليمية: `pending` , `approved` , `archived` , `rejected` .

- `isSelectable` — هل يمكن للطالب اختيار هذا التسجيل كسياق نشط؟ (المعتمد والمؤرشف فقط — لا المعلّق ولا المرفوض).
- `isArchived` .
- `fromJson` هنا **تُرجع** `null` لا قيمة افتراضية، وتُرجع `null` أيضًا للنص الفارغ. الفرق جوهري: لا يصح افتراض حالة حساب — الحالة المجهولة يجب أن تظهر كمجهولة لا كـ «معتمد» مثلًا.
- تستخدم حلقة `for` صريحة بدل `firstWhere` لتفادي الحاجة إلى `orElse` .

login_method_enum.dart ١٥ سطرًا

lib/core/enums/login_method_enum.dart

طريقة الدخول: `phone` أو `email` ، والحقل `fieldName` هو اسم الحقل الذي يُرسل للخادم.

- `isEmail` و `toggled` — زر «الدخول بالبريد بدلًا من الهاتف» يصبح سطرًا واحدًا.
- `fromStorage(value)` — قراءة التفضيل المحفوظ من «تذكّرني»، بافتراض الهاتف.

reset_channel_enum.dart ١٩ سطرًا

lib/core/enums/reset_channel_enum.dart

قناة إرسال رمز استعادة كلمة المرور: `email` , `sms` , `whatsapp` .

- `isEmail` و `wantsEmailAddress` (الثانية تفوّض للأولى، لكن اسمها يعبّر عن سؤال الواجهة: «هل أطلب بريدًا أم رقم هاتف؟»).
- `fromJson` بنمط `switch` على القيمة الخام مع افتراض `sms` .

password_requirement_enum.dart ٣٧ سطرًا

lib/core/enums/password_requirement_enum.dart

متطلبات كلمة المرور الخمسة: الطول الأدنى، حرف كبير، حرف صغير، رقم، رمز. كل متطلب يحمل مفتاح ترجمته.

- `labelArgs` — تُرجع `{'count': '8'}` لمتطلب الطول فقط، فتُستخدم مع الترجمة ذات المعاملات (`tr(args:)`) ولا يُكتب الرقم في نص الترجمة.
- `isSatisfiedBy(password)` — كل متطلب يعرف كيف يتحقق من نفسه عبر `AppRegex` .
- `firstUnsatisfiedIn(password)` — أول متطلب غير محقّق (أو `null`)، لعرض رسالة خطأ واحدة دقيقة بدل خمس رسائل.
- `allSatisfiedBy(password)` — تفوّض للسابقة.

> هذا النموذج يجعل `PasswordRequirementsChecklist` في الواجهة مجرد حلقة على `values` تسأل كل عنصر: «هل أنت محقّق؟» — بلا أي منطق تحقّق في الودجة.

password_strength_enum.dart ٣٨ سطرًا

lib/core/enums/password_strength_enum.dart

قوة كلمة المرور: `none` , `weak` , `fair` , `good` , `strong` ، وكل مستوى يحمل `filledSegments` (عدد المقاطع الملوّنة في المؤشر، من أصل `segments = 4`) ومفتاح عنوانه.

- `PasswordStrength.of(password)` — تحتسب نقاطًا بأربعة معايير: الطول ٨+، (حرف ورقم معًا)، خلط الحالات، (رمز أو طول ١٢+). ثم تحوّل النقاط لمستوى بـ `switch` نمطي يستخدم `>= 4` كنمط علائقي.
- كلمة المرور الفارغة ⟵ `none` مباشرة (فلا يظهر مؤشر «ضعيف» قبل أن يكتب المستخدم حرفًا).

profile_field_enum.dart ١٧ سطرًا

lib/core/enums/profile_field_enum.dart

الحقول المعروضة في بطاقة الحساب: `institution` , `academicPath` , `phone` ، وكل حقل يحمل عنوانه وصورته.

profile_action_enum.dart ١٥ سطرًا

lib/core/enums/profile_action_enum.dart

إجراءات شاشة الحساب الثلاثة: `editInformation` , `changePassword` , `downloadsLog` ، وكل إجراء يحمل عنوانه وأيقونته.

٦,٤ — تعدادات الجلسات والجهاز والأمن

device_session_status_enum.dart سطر واحد

lib/core/enums/device_session_status_enum.dart

نتيجة التحقق من جلسة الجهاز: `noSession` (لا توكن ⟵ شاشة الدخول)، `valid` (كل شيء سليم ⟵ الشاشة الرئيسية)، `deviceMismatch` (التوكن صالح لكن الجهاز مختلف ⟵ رسالة «حسابك مستخدم على جهاز آخر»).

> تعداد صغير جدًا لكن مركزي: هو ما تبني عليه شاشة البداية (`Splash`) قرار التوجيه الأول في التطبيق.

security_event_type_enum.dart ٩ أسطر

lib/core/enums/security_event_type_enum.dart

أنواع الأحداث الأمنية التي ترفعها السياسة إلى الخادم: `screenshot` (التقاط صورة)، `screenRecord` (تسجيل الشاشة)، `externalDisplay` (توصيل شاشة خارجية). المفتاح النصي يُرسل كما هو.

app_version_status_enum.dart ١٧ سطرًا

lib/core/enums/app_version_status_enum.dart

نتيجة فحص إصدار التطبيق من الخادم: `ok` , `optionalUpdate` , `forceUpdate` , `maintenance` .

- `fromApi(value)` — تنظيف ثلاثي (`toString().trim().toLowerCase()`) ثم `switch` ، والافتراضي `ok` . أي قيمة غريبة من الخادم لا تقفل التطبيق — خيار صحيح.
- `isMaintenance` .
- `blocksApp` — القيمة التي تهم الواجهة فعلًا: هل يجب حجب التطبيق؟ (نعم للتحديث الإجباري والصيانة، لا للتحديث الاختياري).

> تسمية الدالة `fromApi` لا `fromJson` فرق مقصود: هذه تقرأ حقل حالة لا كائنًا كاملًا.

٦,٥ — تعدادات التشغيل والتحميل

playback_speed_enum.dart ٥١ سطرًا

lib/core/enums/playback_speed_enum.dart

أغنى تعداد في المجلد منطقًا. سبع سرعات من `0.5x` إلى `2x` ، كل واحدة تحمل `rate` (رقم) و `label` (نص للعرض).

- `maxOnFirstWatch = 1.5` — **قاعدة تربوية مزروعة في الكود:** السرعات فوق ١٫٥× لا تُتاح إلا بعد إكمال الدرس مرة، كي لا يتخطى الطالب المحتوى في أول مشاهدة.
- `maxOnLowEndDevice = 1.25` — **قاعدة تقنية:** الأجهزة الضعيفة لا تتحمّل تشغيلًا أسرع من ذلك بسلاسة.
- `needsCompletedLesson` و `exceedsDevice({isLowEndDevice})` — اختباران منفصلان لكل قاعدة.
- `cappedFor({isLessonCompleted, isLowEndDevice})` — تطبّق القاعدتين بالترتيب وتُرجع أعلى سرعة مسموحة فعلًا. فالواجهة تنادي هذه الدالة ولا تعرف شيئًا عن القواعد.
- `fromRate(double?)` — تُرجع أقرب سرعة معرّفة لرقم خام (قادم من محرّك التشغيل أو من الكاش) بحساب أصغر فرق مطلق، وتفترض `normal` للقيمة الفارغة. الشرط `if (gap >= smallestGap) continue;` يجعل الأولوية للأول عند التعادل.

sleep_timer_option_enum.dart ٢٤ سطرًا

lib/core/enums/sleep_timer_option_enum.dart

مؤقّت النوم: `off(0)` , ١٥, ٣٠, ٤٥, ٦٠, ٩٠ دقيقة.

- `isOff` — تُقاس بالدقائق لا بالهوية (`minutes == 0`).
- `selectable` — القائمة المعروضة في الواجهة بعد استبعاد «مغلق» (الإغلاق له زر منفصل).
- `fromMinutes(minutes)` — تستخدم `lastWhere(option.minutes <= minutes)` أي «أكبر خيار لا يتجاوز القيمة المحفوظة». فلو كانت القيمة في الكاش ٥٠ دقيقة (من إصدار قديم) تُقرأ كـ ٤٥ بدل أن تفشل.

download_phase_enum.dart ١٣ سطرًا

lib/core/enums/download_phase_enum.dart

مرحلة التحميل: `downloading` , `completed` , `failed` .

- `isDownloading` , `isCompleted` , `isFailed` .
- `isFinished` — تعني «ليس جاريًا» (مكتمل أو فاشل). تُستخدم لإخفاء شريط التقدّم في كلتا النهايتين بشرط واحد.

live_session_status_enum.dart ٢٠ سطرًا

lib/core/enums/live_session_status_enum.dart

حالة جلسة البث: `scheduled` , `live` , `ended` , `cancelled` ، مع `key` نصي و `fromJson` بافتراض `scheduled` .

> افتراض «مجدولة» آمن: لو وصلت حالة غريبة تُعرض الجلسة كقادمة لا كمنتهية — فلا يفقد الطالب جلسة قائمة.

٦,٦ — تعدادات الإشعارات

notification_tone_enum.dart سطر واحد

lib/core/enums/notification_tone_enum.dart

«نبرة» الإشعار البصرية: `administrative` , `alert` , `lesson` . تُترجم إلى ألوان في `themes/notification_tone_palette.dart` .

> فصل «النبرة» عن «النوع» تصميم ذكي: ثمانية أنواع إشعارات تتقاسم ثلاث لوحات ألوان فقط، فإضافة نوع جديد لا تتطلب لونًا جديدًا.

notification_kind_enum.dart ٦١ سطرًا

lib/core/enums/notification_kind_enum.dart

أنواع الإشعارات الثمانية، وهو التعداد الذي يربط كل شيء ببعضه.

- **الأنواع ونبراتها:** `newContent` , `liveSession` , `liveReminder` , `enrollmentApproved` ⟵ نبرة `lesson` ؛ `accountStatus` , `inactivityWarning` ⟵ نبرة `alert` ؛ `adminAnnouncement` , `custom` ⟵ نبرة `administrative` .
- `labelKey` — مفتاح الترجمة لكل نوع.
- `imagePath` — الصورة المخصّصة لكل نوع من `AppAssets` .
- `hasOwnTile` — `true` للإعلان الإداري والمخصّص فقط: هذان النوعان لهما بطاقة بشكل خاص في القائمة.
- `hasMonochromeIcon` — نقيض السابقة: بقية الأنواع تُعرض أيقوناتها بلون واحد متناسق مع النبرة.
- `fromJson` بافتراض `custom` — فأي نوع إشعار جديد يضيفه الخادم يظهر كإشعار مخصّص بدل أن يختفي أو يُسبّب استثناء. هذا سلوك مقاوم للمستقبل (forward-compatible) بامتياز.

> النتيجة: بطاقة الإشعار في الواجهة تكتفي بـ `kind.imagePath` و `kind.labelKey.tr()` و `kind.tone` ولا تحتوي أي `switch` من ثمانية فروع.

> **ملاحظة عامة على المجلد:** كل مفاتيح الترجمة هنا مكتوبة كنصوص حرّة داخل التعدادات (`'home'` , `'settings'` , ...) بينما تنصّ قواعد المشروع على تعريفها كثوابت في `lib/core/localization/lang_keys.dart` . هذا الملف غير موجود في المشروع

حاليًا، والترجمة تُستدعى بـ `'key'.tr()` مباشرة. توحيدها في `LangKeys` سيمنع أخطاء المفاتيح الصامتة (المفتاح الخاطئ يُعرض كما هو للمستخدم بلا أي تحذير).

٧ — مجلد `interceptors/` — معترضات Dio

المعترضات هي الطبقة التي تجعل بقية التطبيق لا تفكر في التوكن ولا في بصمة الجهاز ولا في انتهاء الجلسة. تُسجَّل مرة واحدة في `DioService.init()` بالترتيب: `DeviceInterceptor` ثم `AuthInterceptor`.

device_interceptor.dart ٤٧ سطرًا

`lib/core/interceptors/device_interceptor.dart`

الدور: حقن هوية الجهاز واللغة في كل طلب، والتعامل مع تعارض الأجهزة.

`onRequest`

- إن كانت `DeviceService` مهيّأة ← يضيف `DeviceService.headers` (معرّف الجهاز، نوعه، إصدار النظام...). الشرط `isInitialized` يحمي من إرسال ترويسات فارغة لو سبق طلبٌ ما تهيئة الخدمة.
- يضيف دائمًا `Accept-Language: AppLanguage.code` — فالخادم يُرجع نصوصه (أسماء المواد، رسائل الأخطاء) باللغة الصحيحة تلقائيًا بلا معامل في كل نداء.

`onError`

- إن لم يكن الخطأ تعارض جهاز (`DeviceSessionService.isDeviceConflict`) ← يمرّره للتالي بلا تدخّل.
- إن كان الطلب هو **طلب الدخول نفسه** ← يمرّره أيضًا، **وهذا شرط حرج**: عند الدخول من جهاز جديد يرد الخادم بتعارض جهاز، ويجب أن تراه شاشة الدخول لتعرضه كرسالة («حسابك مرتبط بجهاز آخر») لا أن يتحوّل إلى تسجيل خروج قسري لمستخدم غير مسجّل أصلًا.
- غير ذلك ← `SessionGuard.forceSignOut(noticeKey: 'session_bound_to_another_device')` ثم `handler.reject(err)` فلا يرى أي كود أعلى هذا الخطأ.

`_isLoginRequest` تطبّع المسار قبل المقارنة (تحذف الشرطة المائلة البادئة ثم تعيدها) حتى لا يفشل التطابق بسبب `//login` أو `login`.

auth_interceptor.dart ٢٢٣ سطرًا

lib/core/interceptors/auth_interceptor.dart

الدور: أعقد ملف في طبقة الشبكة، ومسؤوليته واحدة: **تجديد التوكن تلقائيًا وإعادة الطلب الفاشل**، وإنهاء الجلسة عند استحالة ذلك. كل هذا يحدث بلا علم الـ Cubit ولا الواجهة.

الحقل الحاسم: `_refreshFuture`

مرجع واحد لعملية التجديد الجارية. وجوده يحل المشكلة الكلاسيكية: لو انتهى التوكن وكانت خمسة طلبات متزامنة في الطريق، فكل منها سيحصل على 401 وسيحاول التجديد ← خمس عمليات تجديد متوازية، أربعة منها ستفشل (لأن `refresh_token` يُستهلك مرة واحدة عادة) والنتيجة تسجيل خروج خاطئ.

- `_getOrCreateRefresh()` — إن وُجدت عملية جارية تُرجعها **كما هي** فينتظرها الجميع ويحصلون على نفس النتيجة. وإلا تُنشئ واحدة وتخزّنها.
- التنظيف في `whenComplete` محمي بـ `identical(_refreshFuture, refreshFuture)` — فلا تمسح عملية منتهية مرجعًا لعملية أحدث بدأت بعدها.

منطق `onError` بالترتيب

1. **تطبيع المسار** عبر `replaceFirst(RegExp(r'^/+'), '')` ثم إضافة شرطة واحدة — مقارنة موثوقة مع `ApiEndpoints`.
2. **تحديد نوع الطلب:** هل هو `refreshToken`؟ هل هو `login`؟ (معًا: `isAuthEndpoint`).
3. **رموز تقتل الجلسة فورًا:** إن لم يكن طلب مصادقة، ولم يكن الرمز 401، وقال `ErrorMapper.endsSession(statusCode)` نعم ← تسجيل خروج ورفض. (مثل 403 حساب موقوف).
4. **ليس 401؟** ← `handler.next(err)`، أي «ليس شأني»، فيعالجه `ErrorMapper` لاحقًا كخطأ عادي.
5. **401 على مسار مصادقة:** إن كان على مسار التجديد نفسه ← تسجيل خروج (لأن `refresh_token` نفسه منتهٍ، ولا مجال لمحاولة أخرى). وفي كل حال ← رفض. و401 على `login` يصل لشاشة الدخول كـ «بيانات خاطئة» بشكل صحيح.
6. **الحماية من الحلقة اللانهائية:** يفحص `requestOptions.extra['auth_retry']`. إن كان الطلب قد أُعيد مرة ← تسجيل خروج ورفض، فلا تدور الحلقة (401 ← تجديد ← إعادة ← 401 ← ...).
7. **المسار السعيد:** يضع العلم `auth_retry = true`، ينادي `_getOrCreateRefresh()`، وإن عاد بتوكن غير فارغ يُعيد الطلب ثم `handler.resolve(response)` — **فالكود المنادي يستلم الاستجابة الناجحة ولا يعلم أن شيئًا حدث أبدًا.**
8. أي استثناء في هذا المسار ← تسجيل خروج ورفض.

`_refreshAccessToken()`

- يقرأ `refresh_token` من الكاش، ويعود بـ `null` فورًا إن غاب.
- **ينشئ نسخة `Dio` جديدة ونظيفة** للتجديد — نقطة حاسمة: استخدام الـ `Dio` الأصلي كان سيُدخل الطلب في نفس المعترض ويُنتج تعاشقًا (recursion).
- يُرسل `refresh_token` في الجسم، ويرفق التوكن القديم في ترويسة `Authorization` إن وُجد (لأن الخادم يستخدمه للتحقق من الهوية).
- يتحقق من بنية الاستجابة طبقة طبقة (`body is! Map`، ثم `data is! Map`) قبل التحليل — لا ثقة بشكل الاستجابة.
- يحلّل عبر `RefreshTokenResponseModel.fromData`، ويتحقق من `hasSession`.

- **عند النجاح أربع عمليات:** كتابة التوكن الجديد، كتابة `refresh_token` الجديد (أو الاحتفاظ بالقديم إن لم يُرسل جديد)، ربط الجلسة بالجهاز عبر `DeviceSessionService.bind`، وتحديث ترويسة `Dio` الحالية.
- كل الأخطاء تُلتقط وتُرجع `null` — فالفشل يُعبّر عنه بقيمة لا باستثناء، وهذا يبسّط المنادي.

`_retryRequest()`

يبني الطلب من جديد مع **نسخ** كل خصائصه (`method`، `data`، `queryParameters`، `cancelToken`، دوال التقدّم، المهلات، `validateStatus`، `followRedirects`، ...) ويستبدل ترويسة `Authorization` فقط، ويثبّت `auth_retry: true` في `extra`. الحرص على نسخ `onSendProgress` / `onReceiveProgress` مهم حتى لا يتعطّل شريط تقدّم تحميل كتاب لو صادف انتهاء التوكن في منتصفه.

> **نقاط للتحسين:** يستخدم النصوص الخام `'token'` و `'refresh_token'` بدل `CacheKeys.tokenKey` / `CacheKeys.refreshTokenKey`، كما يستخدم `debugPrint` بدل `AppLogger` في فرعي الفشل (بينما يستخدم `AppLogger` في الأعلى).

٨ — مجلد `network/` — طبقة الشبكة

api_client.dart ٨٨ سطرًا

`lib/core/network/api_client.dart`

الدور: الواجهة التي تستخدمها كل مصادر البيانات. تغلّف `DioService` وتحوّل الاستجابة الخام إلى `ApiEnvelope` مُتحقَّق منه، أو تُرمي `AppException` بمفتاح خطأ مترجم.

- `get`، `post`، `put`، `patch`، `delete` — كلها تفوّض لـ `_send()` بتمرير دالة الطلب كـ closure.
- `getObject<T>(path, parse)` — يجلب ويحلّل كائنًا واحدًا: يمرّر `envelope.dataMap` لدالة التحليل. فمصدر البيانات يكتب سطرًا واحدًا بدل خمسة.
- `getPage<T>(path, parse)` — يجلب صفحة ويُرجع `PaginatedResult<T>` عبر `fromDynamic`.

`_send()` — القلب

1. ينفّذ الطلب.
2. إن لم يكن الجسم `Map` ⟵ يرمي `AppException('something_went_wrong')`. (استجابة HTML من بروكسي أو صفحة خطأ مثلًا).
3. يبني `ApiEnvelope`، وإن لم يكن ناجحًا (`status_code != 1`) يرمي `ErrorMapper.fromEnvelope(...)`.
4. في `catch` الشامل: يحوّل **أي** خطأ إلى `AppException` عبر `ErrorMapper.toException`، ثم ينادي `await SessionGuard.endIfRefused(exception)` قبل إعادة الرمي.

> **التعليق الموجود في الملف يشرح أهم قرار هنا:** هذا الخادم قد يرفض الطلب وهو يُرجع `200 OK` مع رمز الرفض في **جسم** الاستجابة. مثل هذه الحالات لا تمرّ عبر `onError` في معترض Dio إطلاقًا (لأن النقل نجح). لذلك يُقرأ «الحكم» هنا أيضًا، فتصبح نهاية الجلسة معتمدة على **الرمز** لا على **طريق وصوله**. هذه حماية مزدوجة مقصودة لا تكرار.

api_envelope.dart ٥٣ سطرًا

lib/core/network/api_envelope.dart

الدور: تمثيل الشكل الموحّد لاستجابة هذا الـ API: `status_code` , `data` , `message` , `errors` .

- `successCode = 1` — موثّق بتعليق: هذا الـ API لا يستخدم رموز HTTP للنجاح بل رمزًا خاصًا في الجسم. كل رمز آخر رفض.
- `isSuccess` .
- `dataMap` — يُرجع `data` كخريطة آمنة عبر `Json.asMap` (خريطة فارغة لا `null`).
- `firstError` — أول رسالة خطأ من أي حقل، للعرض السريع في Toast.
- `errorsFor(field)` — أخطاء حقل محدّد (لتوزيعها على حقول النموذج)، وتُرجع قائمة فارغة لا `null` .
- `_parseErrors` — يحلّل بنية أخطاء Laravel المعتادة، ويتعامل مع الحالتين: القيمة قائمة رسائل، أو رسالة واحدة (فيلفّها في قائمة).

json_reader.dart ١٢٨ سطرًا

lib/core/network/json_reader.dart

الدور: أكثر ملف يُستدعى في المشروع. كلاس `Json` الثابت الذي يجعل تحليل أي حقل **مستحيل الفشل**: لا يرمي استثناء أبدًا، بل يُرجع قيمة افتراضية آمنة. هذا هو ما يحمي التطبيق من الانهيار عندما يرسل الخادم `"5"` بدل `5` أو `null` بدل نص.

الدالة	السلوك
`asString`	نص دائمًا؛ `null` تصبح نصًا فارغًا.
`asOptionalString`	يقلّم المسافات ويُرجع `null` للنص الفارغ — مهم للتمييز بين «لا قيمة» و«قيمة فارغة».
`asInt` / `asOptionalInt`	يقبل `int`، ويُدوّر `num`، ويحاول تحليل النص؛ الافتراضي صفر أو `null`.
`asDouble`	مثلها، الافتراضي صفر.
`asBool`	يقبل `bool`، ويعدّ أي رقم غير صفري صحيحًا، ويقبل النصين `true` و`1`.
`asOptionalBool`	تمييز ثلاثي: صحيح / خطأ / مجهول — ضروري لحقل مثل «هل أكمل الدرس؟» حيث «مجهول» ليس «لا».
`asStringList`	قائمة نصوص مع **استبعاد الفراغات**.
`asIntList`	قائمة أعداد مع `whereType<int>()` لإسقاط ما تعذّر تحليله.
`asMapList`	قائمة خرائط، تُسقط العناصر غير الخرائط بصمت.
`asDateTime`	يحلّل ويحوّل **للتوقيت المحلي** (`toLocal()`) — للأوقات (بدء جلسة بث).
`asDate`	يحلّل **بلا** تحويل للتوقيت المحلي — للتواريخ المجرّدة (تاريخ ميلاد)، فالتحويل كان سيُزحزح اليوم.
`asDuration`	يحوّل عددًا من الثواني إلى `Duration`، أو `null`.
`asMap` / `asOptionalMap`	خريطة آمنة / خريطة أو `null`.
`asList<T>(value, parse)`	الدالة العامة لتحليل مصفوفة كائنات: تُسقط ما ليس خريطة ثم تنادي `parse` على كل عنصر.

الدالتان الأذكى: `asLocalizedString` و`asOptionalLocalizedString`

الخادم يُرجع الحقول القابلة للترجمة ككائن مثل `{"ar": "...", "en": "..."}`. هذه الدالة:

1. إن لم تكن القيمة خريطة ← تعاملها كنص عادي (توافق خلفي مع الحقول غير المترجمة).
2. تبني ترتيب أولويات: **لغة المستخدم الحالية** ← **لغة الاحتياط** ← **أي لغة متوفرة** (عبر `...translations.keys`).
3. تُرجع أول نص غير فارغ.

> النتيجة العملية: اسم مادة أُدخل بالعربية فقط يظهر للمستخدم الإنجليزي بالعربية بدل أن يظهر فراغًا. وهذا بالضبط السلوك المطلوب في تطبيق محتواه عربي أساسًا. وملاحظة `<br>`: كل القوائم تُرجع `growable: false` — قوائم للقراءة فقط، أخف وأكثر أمانًا.

pagination_meta.dart ٣٥ سطرًا

lib/core/network/pagination_meta.dart

الدور: بيانات الصفحة: `count` , `perPage` , `currentPage` , `total` مع افتراضات منطقية (صفحة ١، ١٥ عنصرًا).

- `lastPage` — محسوبة: `(total / perPage).ceil()` مع `clamp(1, 1 << 30)` وحماية من القسمة على صفر. لا تعتمد على الخادم في إرسالها.
- `hasMore` — `currentPage < lastPage` : الجواب الوحيد الذي تحتاجه قائمة «التمرير اللانهائي».
- `nextPage` .
- `fromData(data)` — تتعامل مع التعشيش الحقيقي في هذا الـ API: `data.meta.pagination` ، وكل طبقة تُقرأ بـ `Json.asMap` فلا تنهار لو غابت إحداها.

paginated_result.dart ٣٨ سطرًا

lib/core/network/paginated_result.dart

الدور: حاوية عامة `PaginatedResult<T>` تضم `items` و `meta` .

- `isEmpty` و `hasMore` (تفوّض لـ `meta`) — وهما ما تقرأه الـ Cubits لتحديد حالة «فارغ» وحالة «حمّل المزيد».
- `const PaginatedResult.empty()` — نتيجة فارغة ثابتة، مفيدة كقيمة ابتدائية للحالة بلا تخصيص ذاكرة جديد.
- `fromData(map, parse)` — للشكل المصفَّح: `data.items` + `data.meta` .
- `fromDynamic(data, parse)` — **مرونة مهمة:** إن كانت `data` مصفوفة مباشرة (نقاط نهاية لا تصفّح) تُلفّ كقائمة بلا `meta` ؛ وإلا تُعامل كخريطة مصفَّحة. فنقطة النهاية الواحدة قد تتغير شكلًا بلا كسر التطبيق.

media_access_model.dart ٤٢ سطرًا

lib/core/network/media_access_model.dart

الدور: نموذج «تصريح الوصول» للوسائط المحمية — حجر الأساس في حماية المحتوى.

- الحقول: `token` (الرمز المؤقت)، `expiresAt` ، `source` (ملف أم يوتيوب)، `youtubeUrl` .
- `isExpired` — تُقارن بالوقت الحالي، و**تعدّ الرمز بلا تاريخ انتهاء صالحًا** (لا منتهيًا) — الافتراض المتسامح الصحيح.
- `isValid` — رمز غير فارغ وغير منتهٍ.
- `isYoutube` — ليست مجرد فحص للمصدر: تتحقق أيضًا أن `youtubeUrl` موجود وغير فارغ بعد التقليم. فلا يُحوَّل التشغيل لمحرّك يوتيوب بلا رابط.
- `url({required deviceUuid})` — تبني الرابط النهائي:
  • يوتيوب ← الرابط كما هو (مقلّمًا).
  • ملف ← `baseUrl + /media/{token}?device_uuid=...` مع `Uri.encodeQueryComponent` للترميز الصحيح.

> **كيف يحمي هذا المحتوى؟** الرابط النهائي يحتوي رمزًا قصير الأجل **و** معرّف الجهاز. فلو نسخه أحدهم وأرسله لجهاز آخر لا يعمل (معرّف مختلف)، ولو حُفظ لوقت لاحق لا يعمل (الرمز منتهٍ). كون `deviceUuid` معاملًا **مطلوبًا** في الدالة يجعل من المستحيل بناء رابط بلا بصمة جهاز — تصميم صحيح يفرضه نظام الأنواع.

٩ — مجلد `models/` — النماذج المشتركة (٢١ ملفًا)

> هنا تسكن النماذج التي تستخدمها **أكثر من ميزة** (النماذج الخاصة بميزة واحدة تسكن في `features/<feature>/data/models/`). كل النماذج هنا تتبع نمطًا واحدًا: كلاس `const` بحقول `final`، مع `factory fromJson` تقرأ كل حقل عبر `Json.*` (فلا تنهار أبدًا)، و`toJson` فقط للنماذج التي تُكتب أو تُخزّن، وخصائص محسوبة (`get`) تضع **المنطق في النموذج** لا في الواجهة.

٩٫١ — نماذج المستخدم والتسجيل

auth_user_model.dart ٧٦ سطرًا

`lib/core/models/auth_user_model.dart`

الدور: المستخدم المسجَّل كما يراه التطبيق. يُحفظ أيضًا في الكاش (`cachedUserKey`) لذلك يملك `toJson`.

- الحقول: `id`, `fullName`, `email`, `phone`, `role`, `studentNo`, `activeEnrollmentId`, `canCaptureScreen`, `enrollments`.
- `canCaptureScreen` — صلاحية قادمة من الخادم تحدّد هل يُسمح لهذا المستخدم بتصوير الشاشة. تقرأها `ScreenCapturePolicy`. افتراضها `false` أي **المنع** — الافتراض الآمن.
- `initials` — تحسب الأحرف الأولى: تقلّم، تقسّم على المسافات بـ `RegExp(r'\s+')` (فتتحمّل المسافات المتعددة)، تستبعد الفراغات، تأخذ أول كلمتين، تأخذ أول حرف من كل واحدة. فـ «محمد عبد الله الأحمد» ⟵ «مع».
- `activeEnrollment` — **بثلاث طبقات احتياط**: التسجيل المطابق لـ `activeEnrollmentId` ⟵ وإلا أول تسجيل عليه علم `isCurrent` ⟵ وإلا أول تسجيل في القائمة ⟵ وإلا `null`. هذا التدرّج يمنع ظهور شاشة فارغة بسبب حقل واحد ناقص من الخادم.

enrollment_model.dart ٤٩ سطرًا

`lib/core/models/enrollment_model.dart`

الدور: تسجيل الطالب في (جهة + صف). يحمل `id`, `entityId`/`entityName`, `categoryId`/`categoryName`, `status`, `isCurrent`, `rejectionReason`.

- في `fromJson`: `EnrollmentStatus.fromJson(...) ?? EnrollmentStatus.pending` — لأن `fromJson` في التعداد تُرجع `null`، يُفرض هنا «معلّق» كافتراض. وهو الافتراض الصحيح: **لا تمنح وصولًا** لحالة مجهولة.
- `rejectionReason` — سبب الرفض لعرضه للطالب، `null` في الحالات الأخرى.
- يملك `toJson` لأنه جزء من `AuthUserModel` المحفوظ في الكاش، ويكتب `status.key` لا اسم التعداد.

enrollment_node_model.dart ٦٣ سطرًا

lib/core/models/enrollment_node_model.dart

الدور: عقدة في **شجرة** الصفوف داخل جهة تعليمية (مرحلة ← صف ← شعبة). نموذج تعاوُدي: كل عقدة تحمل `children` من نوعها.

- `isGroup` — عقدة بلا `enrollmentId`، أي مجرد عنوان تجميعي لا يُسجَّل فيه.
- `isSelectable` — لها `enrollmentId` **وحالتها** تسمح بالاختيار (`status?.isSelectable ?? false`).
- `hasChildren`.
- `flattened()` — `Iterable` كسول بـ `sync*`: يُنتج العقدة ثم يغوص في أبنائها بـ `yield*`. فتسطيح شجرة بأي عمق يصبح سطرين، وبلا بناء قائمة وسيطة في الذاكرة.
- `pathTo(enrollmentId)` — بحث تعاوُدي يُرجع **المسار الكامل** من الجذر إلى العقدة المطلوبة (أو قائمة فارغة). هذا ما يبني مسار التنقّل «المعهد ← المرحلة الثانية ← الصف الأول».

enrollment_entity_model.dart ٤٠ سطرًا

lib/core/models/enrollment_entity_model.dart

الدور: جهة تعليمية مع شجرة صفوفها، كما تظهر في «تسجيلاتي».

- `entityId`, `entityName`, `logoUrl`, `isActiveEntity`, `tree`.
- `hasClasses`.
- `selectableClasses` — تُسطّح كل الجذور ثم تُرشّح القابل للاختيار فقط، باستخدام `collection for` مع `spread` — صيغة مكثّفة وقارئة.

my_enrollments_model.dart ٦٤ سطرًا

lib/core/models/my_enrollments_model.dart

الدور: الاستجابة الكاملة لـ `/user/my-enrollments`: `activeEnrollmentId` + قائمة الجهات. وهو النموذج الذي يجيب على سؤال «أين أنا الآن؟».

- `isEmpty` — لعرض حالة «لا تسجيلات».
- `activeEntity` — الجهة التي عليها علم `isActiveEntity`.
- `activeClass` — الصف النشط: حلقة ثلاثية (جهات ← جذور ← عقد مسطّحة) حتى تجد العقدة ذات `enrollmentId` المطابق.
- `activePath` — المسار الكامل للصف النشط عبر `root.pathTo(...)`.
- `activePathNames` — نفس المسار كأسماء نصية نظيفة (مع استبعاد الأسماء الفارغة) — جاهزة مباشرة لودجة `EducationalPathText`.

> هذا النموذج مثال نموذجي على القاعدة: **الواجهة لا تحسب شيئًا**. الشاشة تطلب `activePathNames` وتعرضها، وكل الغوص في الشجرة محصور هنا.

active_context_model.dart ٤٧ سطرًا

lib/core/models/active_context_model.dart

الدور: «السياق النشط» المبسّط: أي جهة وأي صف يتصفّح الطالب الآن. يضم كلاسين:

- `ActiveContextModel` — `enrollmentId` , `entityId` / `entityName` , `logoUrl` , `categoryId` / `categoryName` , `status` , `isArchived` . نموذج قراءة فقط (بلا `toJson`).
- `SwitchContextRequestModel` — نموذج **كتابة** فقط (بلا `fromJson`) يحمل `enrollmentId` ويُرجع `{'enrollment_id': ...}` .

> وجود الكلاسين في ملف واحد مقصود: هما طرفا نفس العملية (اقرأ السياق / بدّل السياق). والفصل بين نموذج القراءة ونموذج الكتابة يمنع إرسال حقول لا يتوقعها الخادم.

enrollment_request_model.dart ٤٥ سطرًا

lib/core/models/enrollment_request_model.dart

الدور: طلب التسجيل في صف جديد — كلاسان أيضًا:

- `ApplyEnrollmentRequestModel` — نموذج الإرسال: `categoryId` + `entityId` ⟵ `toJson` .
- `EnrollmentRequestModel` — نموذج الاستجابة بعد تقديم الطلب: `id` , `entityName` , `categoryName` , `status` , `isCurrent` . يُبنى بـ `fromData` (تسمية تعني «من داخل مفتاح `data` في المغلّف» لا من الاستجابة كاملة).

public_entity_model.dart ٣٩ سطرًا

lib/core/models/public_entity_model.dart

الدور: الجهة التعليمية كما تُعرض **قبل** تسجيل الدخول (شاشة اختيار الجهة).

- `id` , `name` , `slug` , `type` , `description` , `logoUrl` , `orderNo` .
- `isActive` و `isOpenForRegistration` — افتراضهما `true` هنا (بخلاف معظم الأعلام في المشروع): فالجهة تُعرض وتُتاح للتسجيل إلا إذا قال الخادم صراحة لا. وهذا هو السلوك المرغوب في شاشة تسجيل جديد — عدم إرسال الحقل لا يجب أن يُغلق التسجيل على الجميع.

public_category_model.dart ٣٨ سطرًا

lib/core/models/public_category_model.dart

الدور: شجرة الصفوف العامة لجهة ما (قبل الدخول). نموذج تعاوُدي مبسّط: `id` , `name` , `orderNo` , `isSelectable` , `children` .

- `hasChildren` و `flattened()` بنفس نمط `EnrollmentNodeModel` .
- الفرق عن النسخة الخاصة: لا يحمل `enrollmentId` ولا `status` — لأن المستخدم غير مسجّل بعد، فالقابلية للاختيار تأتي كعلم مباشر من الخادم.

٩٫٢ — نماذج المحتوى والدروس

lesson_model.dart ٦٦ سطرًا

`lib/core/models/lesson_model.dart`

الدور: الدرس — أكثر نموذج استخدامًا في التطبيق (القوائم، البحث، المحفوظات، شاشة التفصيل).

- `id` , `categorySubjectId` , `title` , `type` (`ContentType`), `description` , `durationSeconds` , `order` , `thumbnailUrl` , `publishedAt` .
- `isLocked` — هل الدرس مقفل للطالب.
- `checkpoints` — `List<int>` بثوانٍ: نقاط تفتيش داخل الدرس (يُتوقّع عندها سؤال أو منع التخطي).
- `isProtectedMedia` — هل يحتاج الوسيط رمز وصول (`MediaAccessModel`) أم أن رابطه مباشر.
- `progress` — كائن `LessonProgressModel` متداخل، وافتراضه كائن فارغ `const` لا `null` — فلا حاجة لفحص `null` في أي مكان.
- `duration` — تحويل الثواني إلى `Duration` .
- `status` — **الخاصية الأذكى**: تفوّض لـ `LessonStatus.resolve` بتمرير `progress.isCompleted` و `progress.isStarted` و `isLocked` . فسؤال «ما حالة هذا الدرس؟» له جواب واحد في التطبيق كله.
- في `fromJson` : العنوان والوصف يُقرآن بـ `asLocalizedString` / `asOptionalLocalizedString` — فيختار التطبيق لغة العرض تلقائيًا.

lesson_progress_model.dart ٦٨ سطرًا

`lib/core/models/lesson_progress_model.dart`

الدور: تقدّم الطالب في درس + نموذج نبضة التقدّم. كلاسان:

`LessonProgressModel`

- `maxPositionSeconds` — أقصى موضع وصل إليه الطالب (لا الموضع الحالي).
- `watchedSeconds` — مجموع الثواني المشاهدة فعلًا (يختلف عن السابق عند التقديم والإرجاع).
- `progressPercent` , `isCompleted` .
- `allowedPositionSeconds` — **سقف التخطي**: أقصى موضع يُسمح للطالب بالانتقال إليه. `null` تعني «استخدم `maxPosition` ».
- `maxPosition` , `watched` , `allowedPosition` — نسخ `Duration` جاهزة للمشغّل.
- `progress` — كسر بين • وا مع `clamp` (فلا يتجاوز شريط التقدّم حدّه لو أرسل الخادم ١٠٥٪).
- `isStarted` — «بدأ» إن كان هناك ثوان مشاهدة **أو** موضع محفوظ — شرط متسامح يمنع ظهور درس بدأه الطالب كأنه جديد.

`ProgressHeartbeatRequestModel`

- يُرسل دوريًا أثناء التشغيل: `positionSeconds` + `watchedDeltaSeconds` .
- **يطهّر مدخلاته قبل الإرسال**: `_position` يرفع السالب إلى صفر، و `_delta` يرفع السالب إلى صفر ويقصّ ما يتجاوز `maxWatchedDeltaSeconds = 3600` .

> سقف الساعة على الزيادة حماية مزدوجة: من خطأ حسابي في التطبيق (مثل جهاز نام ثم استيقظ فحُسب الفارق كله مشاهدة)، ومن تلاعب متعمّد لتسجيل إكمال درس بلا مشاهدته. والتطهير في **النموذج** لا في الـ Cubit يعني أنه مضمون لكل من يرسل.

lessons_filter.dart ٨٤ سطرًا

lib/core/models/lessons_filter.dart

الدور: كائن واحد يمثّل «حالة التصفية والصفحنة» لأي قائمة دروس، ويحوّل نفسه إلى معاملات استعلام.

- الحقول: `categorySubjectId` , `type` , `search` , `sort` , `descending` , `page` , `perPage` .
- `trimmedSearch` — نص البحث مقلّمًا، أو فارغًا.
- `copyWith(...)` — مع أعلام **مسح** صريحة: `clearType` , `clearSubject` , `clearSearch` . **هذا حل لمشكلة حقيقية:** في `copyWith` العادية لا يمكن التمييز بين «لا تغيّر» و«اجعلها `null` » لأن كلتيهما تُمرَّر كـ `null` . الأعلام تفصل الحالتين، فيعمل زر «إلغاء التصفية» بشكل صحيح.
- `nextPage()` — اختصار للصفحة التالية.
- `toQueryParameters()` — تبني الخريطة بإدخالات شرطية (`if` داخل حرفية الخريطة) فلا تُرسل معاملات فارغة. ويُدمج الترتيب في معامل واحد بصيغة Laravel: `-published_at` للتنازلي.
- **يعرّف** `==` **و** `hashCode` عبر `Object.hash` . ضرورة لا رفاهية: الـ Cubit يقارن المرشّح الجديد بالقديم ليتجنّب إعادة جلب نفس البيانات، والمقارنة تستخدم `trimmedSearch` لا `search` فلا تُعتبر مسافة زائدة تغييرًا.

subject_model.dart ٦٤ سطرًا

lib/core/models/subject_model.dart

الدور: المادة الدراسية كما تظهر في شبكة المواد.

- `categorySubjectId` (معرّف ربط المادة بالصف — وهو المستخدم في الطلبات) و `subjectId` (معرّف المادة المجرّدة).
- `name` , `iconUrl` , `orderNo` , `position` , `lessonsCount` , `completedLessons` , `progressPercent` .
- `progress` — كسر مقصوص بين • وا.
- `isCompleted` — `lessonsCount > 0 && completedLessons >= lessonsCount` . شرط العدد الموجب مهم: مادة بلا دروس ليست «مكتملة» (وإلا ظهرت علامة إنجاز لمادة فارغة).
- `listFrom(items)` — دالة ثابتة تحلّل القائمة **وترتّبها ترتيبًا مستقرًا**: تُفهرس العناصر بـ `(index, model)` كـ `record` ، ثم ترتّب حسب `position` ، وعند التعادل حسب الفهرس الأصلي. **لماذا؟** لأن `List.sort` في Dart غير مستقرة، فالمواد المتساوية في `position` كانت ستتبدّل أماكنها عشوائيًا في كل تحديث — وهو اهتزاز مزعج في الواجهة.

subject_content_model.dart ٥٧ سطرًا

lib/core/models/subject_content_model.dart

الدور: الصورة الكاملة لشاشة محتوى المادة: معلوماتها + أعداد تبوياتها + كتبها.

- معلومات المادة: `categorySubjectId` , `name` , `iconUrl` , `position` .
- معلومات السياق: `categoryId` , `categoryName` , `entityName` — فتستطيع الشاشة عرض «المعهد ← الصف ← المادة» بلا طلب إضافي.
- `tabs` — `SubjectTabCountsModel` متداخل (افتراضه كائن `const` فارغ).
- التقدّم: `lessonsCount` , `completedLessons` , `progressPercent` , و `progress` المحسوبة.
- `textbooks` — قائمة الكتب مضمّنة في نفس الاستجابة.

subject_tab_counts_model.dart ٣٤ سطرًا

lib/core/models/subject_tab_counts_model.dart

الدور: أعداد العناصر في كل تبويب: `videos` , `audios` , `articles` , `albums` , `liveSessions` , `textbooks` — كلها افتراضها صفر.

- `total` — المجموع.
- `isEmpty` — المجموع صفر، أي «لا محتوى في هذه المادة بعد» ← تُعرض حالة فارغة واحدة بدل ستة تبويبات فارغة.

> هذه الأعداد هي ما يسمح بإظهار شارة العدد على كل تبويب **قبل** فتحه، أي بلا جلب محتوى التبويبات كلها.

textbook_model.dart ٣٧ سطرًا

lib/core/models/textbook_model.dart

الدور: الكتاب المدرسي (PDF).

- `id` , `title` , `categorySubjectId` , `version` , `fileSize` (بايت)، `pageCount` .
- `isCurrent` — افتراضه `true` : هل هذه الطبعة الحالية (فالطبعات القديمة تُعرض كأرشيف).
- `readableSize` — يفوّض لـ `ByteFormat.readable` فتعرض الواجهة «٤٫٢ م.ب» بلا أي حساب.

live_session_model.dart ٦١ سطرًا

lib/core/models/live_session_model.dart

الدور: جلسة البث المباشر.

- `id` , `title` , `description` , `coverUrl` , `youtubeUrl` , `scheduledAt` , `reminderMinutesBefore` , `status` .
- ثلاثة أعلام من الخادم: `isUpcoming` , `isLive` , `isEnded` . وجودها بجانب `status` ليس تكرارًا: **الخادم هو من يملك الساعة**، فحساب «هل بدأت؟» من ساعة الجهاز كان سيفشل مع كل جهاز ساعته غير مضبوطة.
- `isPlayable` — `isLive` **و** وجود رابط يوتيوب غير فارغ. فزر «انضم» لا يظهر لجلسة بُثّت بلا رابط.
- `startsIn` — المدة المتبقية للبدء: تُرجع `null` إن لا وقت أو ليست قادمة، و`Duration.zero` إن مضى الوقت (بدل مدة سالبة تُعرض كـ «-٥ دقائق»).

٩٫٣ — نماذج التحميل والدول

download_progress.dart ٣٦ سطرًا

lib/core/models/download_progress.dart

الدور: تقدّم تحميل ملف: `received` و`total` بالبايت.

- `none` — ثابت `const` للحالة الصفرية.
- `isDeterminate` — `total > 0`، أي: هل نعرف الحجم الكلي؟ (بعض السيرفرات لا ترسل `Content-Length`).
- `hasStarted` — وصلت بايتات أو عرفنا الحجم.
- `fraction` — `double?`: الكسر مقصوصًا، أو `null` **إن كان الحجم مجهولًا**. وهذا التصميم بالغ الأهمية: تمريرها مباشرة لـ `LinearProgressIndicator(value: ...)` يعطي شريطًا محدّدًا عند معرفة الحجم، و**شريطًا لانهائيًا متحرّكًا** عند جهله — بلا أي شرط في الواجهة، وبلا اختلاق نسب كاذبة.
- `percent` — نسبة صحيحة بـ `floor()` (فلا تظهر ١٠٠٪ قبل الانتهاء الفعلي).
- `readableRatio` — نص مثل «٢٫١ / ٤٫٢ م.ب»، ويعرض الحجم المستقبَل فقط عند جهل الكلي. يستخدم `readableValue(..., referenceBytes: total)` لتوحيد الوحدة بين الطرفين (فلا يظهر «٩٠٠ ك.ب / ٤٫٢ م.ب»).
- `withTotalFallback(fallback)` — تعبئة الحجم الكلي من مصدر آخر (مثل `fileSize` في نموذج الكتاب) عندما لا يرسله السيرفر. فيتحوّل الشريط اللانهائي إلى محدّد.

file_download_state.dart ٤٤ سطرًا

lib/core/models/file_download_state.dart

الدور: حالة تحميل واحد كما تحتاجها الواجهة — موثّق في الملف بتعليق صريح. يجمع `phase` + `progress` + `errorKey`.

- `errorKey` — **مفتاح ترجمة** لا رسالة جاهزة، يُضبط فقط في حالة الفشل. التزام بقاعدة «الـ Cubit يبث المفتاح والواجهة تترجم».
- **مصانع مسمّاة تمنع الحالات المستحيلة:**
  - `FileDownloadState.starting({expectedBytes})` — طُلب التحميل ولم تصل بايتات بعد.
  - `FileDownloadState.running(progress)` — جارٍ.
  - `FileDownloadState.failed(errorKey)` — فشل، والمفتاح **مطلوب**.
  - `FileDownloadState.completed` — ثابت `const` بتقدّم `1/1` أي ١٠٠٪.
- `isDownloading`, `isCompleted`, `isFailed` — تفوّض كلها للتعداد.

> الفائدة من المصانع المسمّاة: يستحيل إنشاء حالة «فاشلة بلا سبب» أو «مكتملة بتقدّم ٦٠٪». **النموذج يفرض صحّته بنفسه**، والواجهة تكتفي بـ `switch` على ثلاث حالات. وكون `completed` ثابتًا `const` يعني أن كل التحميلات المكتملة تتشارك كائنًا واحدًا في الذاكرة.

country_model.dart ٦٤ سطرًا

lib/core/models/country_model.dart

الدور: الدولة — نموذج غني بالسلوك يخدم حقل الهاتف ومنتقي الدول. لا يملك `fromJson` لأن بياناته ثابتة في `AppCountries`.

- الحقول الموضعية: `isoCode`, `dialCode`, `nameEn`, `nameAr`؛ والمسمّاة: `minLength`, `maxLength`, `groups` (افتراضها `[3,3,3,3]`).
- `flag` — **حيلة أنيقة**: تبني رمز العلم التعبيري حسابيًا من حرفي رمز الدولة، عبر إزاحة كل حرف إلى نطاق «رموز المؤشر الإقليمي» (`0x1F1E6` يقابل الحرف A). النتيجة: أعلام كل دول العالم **بلا أي صورة** في التطبيق.
- `displayDialCode` — الرمز مع إشارة الزائد.
- `localizedName(languageCode)` — الاسم بحسب اللغة.
- `matches(query)` — بحث متعدّد المعايير: الاسم الإنجليزي (بلا حساسية حالة)، الاسم العربي، رمز ISO، أو **بداية** رمز الاتصال بعد حذف الزائد. والاستعلام الفارغ يطابق الكل (فتظهر القائمة كاملة ابتداءً).
- `isValidNationalNumber(digits)` — تحقّق الطول بحدود **هذه الدولة**.
- `formatNational(digits)` — تنسيق الرقم بمجموعات: يستهلك الأرقام بحسب `groups` ويضع مسافة بين المجموعات، ويُلحق الباقي كما هو إن زادت الأرقام. فلا يُفقد أي رقم أبدًا.
- `toE164(digits)` — الصيغة الدولية القياسية للإرسال للخادم.

> فصل العرض عن التخزين: الواجهة تعرض `formatNational` (مقروء)، والخادم يستقبل `toE164` (قياسي). والنموذج يملك الاثنين، فلا تتسرّب المسافات إلى قاعدة البيانات ولا تُعرض الأرقام ملتصقة للمستخدم.

١٠ — مجلد `routing/` — التوجيه والمسارات (٧ ملفات)

التطبيق يستخدم التوجيه التقليدي بالأسماء (`onGenerateRoute`) لا `go_router`. المجلد يضم جدول أسماء المسارات، مولّد الصفحات، وثلاثة ملفات «وسائط» (args) تحمل بيانات الانتقال بشكل مُؤمَّن بالأنواع، ومراقب تنقّل، وأذكى ملف في المجلد: مُحلِّل وجهة الإشعارات.

app_routes.dart ٣٢ سطرًا

lib/core/routing/app_routes.dart

الدور: أسماء المسارات كثوابت نصية — فلا يُكتب اسم مسار كنص حر في أي `pushNamed`.

- `splash = '/'` هو المسار الجذر. بقية المسارات **بلا شرطة بادئة** (`login`, `home`, `lesson-detail`...) وهو اصطلاح متّسق في الملف كله.
- ٢٥ مسارًا تغطي: المصادقة (`login`, `forgetPassword`, `setPassword`)، حالات الحساب (`review`, `rejected`)، الـ Shell (`home`)، المحتوى (`study`, `subjectContent`, `viewAll`, `lessonDetail`, `lessonTest`, `bookPreview`)، البث (`live`, `livePlayer`)، والحساب (`profile`, `editProfile`, `changePassword`, `downloadsLog`)، وأخرى (`archive`, `pictures`, `search`, `settings`, `notification`, `educationalPathways`).

app_router.dart ٣٠٦ أسطر

lib/core/routing/app_router.dart

الدور: مولّد الصفحات المركزي + منطق التوجيه بعد الإقلاع وبعد الدخول.

`onGenerateRoute(settings)`

`switch` ضخم على `settings.name` يُرجع `MaterialPageRoute` لكل مسار. الأنماط المتكرّرة المهمة:

- **شاشة البداية مغلّفة بطبقتين:** `AppUpdateGate(child: OrientationLock(child: SplashScreen(...)))` — فحص التحديث الإجباري يحيط بكل شيء، وقفل الاتجاه يُطبَّق من أول شاشة.
- **تحقّق الوسائط بـ `is` مع بديل آمن:** مثل `args is ProfileModel ? EditProfileScreen(profile: args) : const ProfileScreen()`. فلو وصل التطبيق لمسار بوسائط خاطئة (من إشعار قديم مثلًا) يُعرض شيء منطقي لا شاشة بيضاء ولا انهيار. نفس النمط في `bookPreview` (بديله شاشة المواد) و`livePlayer` (بديله قائمة الجلسات).
- **الوسائط المعقّدة تُحلَّل بمصانع `fromRoute`:** `LessonDetailArgs.fromRoute(args)`، `SubjectContentArgs.fromRoute(args)`، `LessonTestArgs.fromRoute(args)`، `SetPasswordArgs.fromRoute(args)` — فمنطق التحليل يسكن مع النموذج لا في الراوتر.
- **ردود النداء تُمرَّر من الراوتر:** الشاشات لا تعرف إلى أين تنتقل. مثلًا `ArchiveScreen(onLessonTap: ...)` و`SearchScreen(onResultTap: ...)` و`LessonDetailScreen(onGoToTest: ...)`. هذا ما يجعل الشاشات قابلة لإعادة الاستخدام ومستقلة عن خريطة التنقّل.
- **إشارات لمُشغّل الإشعارات داخل الـ `switch`:** عند بناء مسار `login` يُنادى `NotificationLauncher.instance.markSignedOut()`، وعند بناء `home` يُنادى `markReady()`. فالإشعار الذي يصل قبل جهوزية الشاشة يُحتجز ولا يُفتح على شاشة الدخول.
- `default` يُرجع `SizedBox()` فارغة — مسار مجهول لا يُسقط التطبيق.

`_onSplashCompleted(context, isLoggedIn)`

1. `if (!await AppUpdateFlow.resolve(context)) return;` — إن كان هناك تحديث إجباري أو صيانة، يتوقف كل شيء هنا.
2. `if (!context.mounted) return;` — الفحص الصحيح بعد `await` قبل استخدام السياق.
3. `pushNamedAndRemoveUntil` إلى `login` أو `home` مع مسح المكدس.

`_onLoggedIn(context, session)` — منطق التوجيه بعد الدخول

يُنادي `unawaited(getIt<ProfileCubit>().reload())` أولًا (تحميل الملف الشخصي في الخلفية بلا تعطيل الانتقال)، ثم `switch` على حالة التسجيل النشط:

الحالة	الوجهة
`pending`	`RequestReviewScreen` مع اسم الجهة والصف والهاتف — «طلبك قيد المراجعة».
`rejected`	`RejectedScreen(reason: ...)` مع سبب الرفض.
`approved`	شاشة «تمت الموافقة» **مرة واحدة فقط** ثم الرئيسية.
`archived` أو `null`	الرئيسية مباشرة.

تفصيل جيد في حالة `approved`: يُقارَن `CacheUtil.get(_approvalSeenKey)` بمعرّف التسجيل. فإن كان المستخدم قد رأى شاشة التهنئة لهذا التسجيل ← يذهب للرئيسية مباشرة. وإلا تُعرض، وفي `onContinue` يُحفظ المعرّف ثم ينتقل. حفظ **معرّف التسجيل** لا مجرد علم منطقي يعني أن الطالب الذي يُقبل في صف جديد لاحقًا يرى التهنئة مرة أخرى — وهو السلوك الصحيح.

ملاحظات: مسار `AppRoutes.review` في الـ `switch` يستخدم قيمًا مكتوبة بشكل ثابت (`'Bilhikma'`, `'1st Year'`, رقم هاتف) — تبدو بقايا تطوير؛ المسار الحقيقي المستخدم بعد الدخول يبنيه `_onLoggedIn` ببيانات صحيحة. كما أن الملف الطويل (٣٠٦ أسطر) يمكن تقسيمه إلى `app_router.dart` + `post_login_routing.dart`.

lesson_detail_args.dart ٣٤ سطرًا

`lib/core/routing/lesson_detail_args.dart`

الدور: وسائط شاشة تفصيل الدرس: `id`, `type` (افتراضها `video`)، `isCompleted`.

- `isValid` — `id > 0`، تقرأها الشاشة لتعرض خطأً بدل طلب فاشل.
- `fromLesson(lesson)` — بناء من نموذج كامل (عند النقر في قائمة): ينقل النوع وحالة الإكمال معه، **فتستطيع الشاشة أن ترسم نفسها فورًا قبل وصول أي استجابة شبكة** — لا وميض ولا هيكل عظمي بلا داعٍ.
- `fromRoute(arguments)` — تحليل متسامح بـ `switch` نمطي: كائن `LessonDetailArgs` يُرجع كما هو، `int` يُعامل كمعرّف، `String` يُحاول تحليله، وأي شيء آخر يصبح صفرًا (← `isValid == false`). هذا التسامح ضروري لأن الإشعارات والروابط العميقة تمرّر نصوصًا.

subject_content_args.dart ١٩ سطرًا

`lib/core/routing/subject_content_args.dart`

الدور: وسائط شاشة محتوى المادة: `categorySubjectId` فقط، مع `isValid` و`fromRoute` بنفس النمط المتسامح السابق (كائن / عدد / نص / صفر).

book_preview_args.dart ٥٢ سطرًا

`lib/core/routing/book_preview_args.dart`

الدور: وسائط قارئ الكتب. موثّق في أعلاه بتعليق: الكتاب إمّا ملف على الجهاز، وإمّا رابط موقّع يُبثّ من الشبكة.

- `assert(filePath != null || url != null)` في المنشئ الأساسي — **يمنع إنشاء وسائط بلا مصدر** في وضع التطوير.
- `BookPreviewArgs.network({required String this.url, ...})` — منشئ مسمّى يضمن وجود الرابط ويصفّر `filePath`. لاحظ الصيغة `required String this.url`: تُضيّق نوع الحقل من `String?` إلى `String` في هذا المنشئ — ميزة Dart تُغني عن أي فحص لاحق.
- `isNetwork`, `networkUrl` (مقلّمًا)، `localPath`.
- `contentId` — معرّف المحتوى إن كان الكتاب مرتبطًا بدرس.
- `readingKey` — **مفتاح استئناف القراءة**، بثلاث طبقات: `resumeKey` المُمرَّر إن وُجد ← وإلا `url:<آخر مقطع من الرابط>` ← وإلا `file:<اسم الملف>`.

لماذا آخر مقطع فقط؟ لأن الروابط الموقّعة تحمل رموزًا ومُهَلًا تتغير في كل جلسة. لو كان المفتاح هو الرابط الكامل لفُقد موضع القراءة في كل مرة. استخدام اسم الملف يجعل «استأنف من صفحة ٤٧» يعمل عبر الجلسات — وهذا بالضبط نوع التفصيل الذي يفرّق بين قارئ جيد وقارئ مزعج.

keyboard_dismiss_observer.dart ٢٦ سطرًا

lib/core/routing/keyboard_dismiss_observer.dart

الدور: `NavigatorObserver` يُغلق لوحة المفاتيح عند **أي** تغيّر في المكدس: `didPush` , `didPop` , `didRemove` , `didReplace` — كلها تنادي `_dropFocus()` .

- `_dropFocus()` يقرأ `FocusManager.instance.primaryFocus` ، ويخرج فورًا إن لا تركيز أو لا لوحة مفاتيح، وإلا `unfocus()` .

> المشكلة التي يحلّها: فتح شاشة جديدة بينما لوحة المفاتيح مرفوعة يُنتج انتقالًا مهتزًّا (الشاشة الجديدة تُبنى بمساحة مضغوطة ثم تتوسّع). تسجيل هذا المراقب مرة واحدة يحل المشكلة في التطبيق كله بدل كتابة `unfocus()` قبل كل `push` . والشرط الحمائي في البداية يجعل التكلفة صفرًا حين لا حاجة.

notification_destination.dart ٢٦١ سطرًا

`lib/core/routing/notification_destination.dart`

الدور: أذكى ملف في المجلد، ومسؤوليته واحدة: **أيًّا كان شكل البيانات القادمة مع الإشعار، أوجِد الشاشة الصحيحة**. يضم الكلاس العام `NotificationDestination` والكلاس الخاص `_DeepLink`.

الكائن نفسه

بسيط جدًا: `route` + `arguments`. وكل التعقيد في دوال التحليل الثابتة.

`resolve({kind, payload, deepLink})` — سلسلة الأولويات

تُجرّب أربعة مصادر بالترتيب وتأخذ أول نجاح (عبر `??` المتسلسل):

1. `fromPayload(payload)` — الحمولة المنظّمة (الأدق).
2. `_forTarget(link)` — هدف معروف في الرابط العميق.
3. `_asLesson(link)` — رابط بلا هدف لكن فيه معرّف ← يُفترض أنه درس.
4. `_fromKind(kind)` — الاحتياط الأخير: الوجهة حسب نوع الإشعار فقط.

`fromPayload(payload)`

ثلاثة فحوص بالترتيب: `hasContent` ← تفصيل الدرس (بالمعرّف والنوع)، `hasLiveSession` ← شاشة البث، `hasSubject` ← محتوى المادة. وإلا `null`.

`fromPushData(data)` — للحمولة الخام من Firebase

- يجرّب أولًا الرابط العميق بالبحث في خمسة أسماء محتملة: `deep_link`, `deeplink`, `link`, `route`, `url`.
- ثم معرّف المحتوى في `content_id` أو `lesson_id`.
- ثم `live_session_id` أو `session_id`.
- ثم `category_subject_id` أو `subject_id`.
- وأخيرًا النوع من `type` أو `kind`.

> **لماذا كل هذه الأسماء البديلة؟** لأن حمولة الإشعار يبنيها الخادم (أو لوحة تحكّم، أو خدمة طرف ثالث) وقد تتغير تسمياتها. قبول كل الصيغ المعقولة يعني أن إشعارًا بتسمية مختلفة **يفتح الشاشة الصحيحة** بدل أن يفتح الرئيسية ويُترك المستخدم يبحث. وهذه **مرونة مقصودة في طبقة الحدود** لا تهاون في الدقة.

`_forTarget(link)` — خريطة المرادفات

`switch` يستخدم أنماط «أو» (`||`) ليقبل كل المرادفات المعقولة لكل وجهة:

الوجهة	المرادفات المقبولة
تفصيل الدرس	`content`, `contents`, `lesson`, `lessons`, `media`
محتوى المادة	`subject`, `subjects`, `subject_content`, `category_subject`, `categorysubject`
البث	`live`, `lives`, `live_session`, `live_sessions`, `session`, `sessions`
المحفوظات	`archive`, `archives`, `saved`
الصور	`gallery`, `pictures`, `images`
الحساب	`profile`, `account`

المسارات التعليمية	`pathway` ، `pathways` ، `educational_pathway(s)`
المواد	`study` ، `categories`
أخرى	`search` ، `settings` ، `notification(s)`

ولاحظ التفصيل في فرع المادة: إن كان المعرّف غير صالح (`id <= 0`) يذهب لشاشة **كل المواد** بدل لا شيء — تدهور لطيف (graceful degradation).

الدوال المساعدة

- `_asLesson(link)` — تعمل فقط حين لا هدف في الرابط.
- `_isTarget(segment)` — **حيلة ذكية**: تسأل `_forTarget` نفسها بمقطع تجريبي لتعرف «هل هذه الكلمة هدف معروف؟» — فلا توجد قائمة مرادفات مكرّرة في مكان ثانٍ، ومصدر الحقيقة واحد.
- `_lesson(link)` — تبني وجهة الدرس، أو `null` إن لا معرّف.
- `_fromKind(kind)` — محتوى جديد ← المواد، جلسة/تذكير ← البث، قبول تسجيل ← الحساب، وغير ذلك `null` (فيبقى المستخدم في مكانه بدل نقلة عشوائية).
- `_firstId(values)` — أول قيمة تُحلَّل كعدد موجب.
- `_firstString(data, keys)` — أول مفتاح موجود بقيمة نصية غير فارغة، ويقبل الأرقام أيضًا فيحوّلها لنص.

`_DeepLink.parse(link)` — تحليل الرابط

1. يقلّم، ويحاول `Uri.tryParse`، ويُرجع رابطًا فارغًا عند الفشل.
2. **يدمج المضيف مع مقاطع المسار** بشرط `if (!uri.host.contains('.'))` — ذكاء حقيقي: في `myapp://lesson/42` تكون كلمة `lesson` هي «المضيف» تقنيًا، أما في `https://bilhikma.tech/lesson/42` فالمضيف نطاق حقيقي يجب تجاهله. فحص وجود النقطة يفرّق بين الحالتين.
3. يطبّع كل المقاطع (تقليم + حروف صغيرة + استبعاد الفراغ).
4. يجد **الهدف**: أول مقطع ليس عددًا وهو هدف معروف (`_isTarget`).
5. يجد **المعرّف** بترتيب أولوية ثلاثي: المقاطع **بعد** الهدف أولًا، ثم كل المقاطع، ثم معاملات الاستعلام (`id` ، `content_id` ، `lesson_id` ، `category_subject_id` ، `subject_id`).
6. يقرأ نوع المحتوى من `type` أو `content_type` في الاستعلام.

> **النتيجة العملية:** كل هذه الأشكال تعمل وتفتح الدرس ٤٢:
> `myapp://lesson/42` · `https://bilhikma.tech/content/42` · `/lessons/42?content_type=audio` · `?lesson_id=42` · `/42`
> وهذه المرونة هي ما يحوّل الإشعارات والروابط المشاركة من مصدر شكاوى إلى ميزة موثوقة.

١١ — مجلد `services/` — الخدمات (٤٣ ملفًا)

أكبر مجلد في `core` وأكثره حساسية. يُقسَّم فعليًا إلى أربع عائلات: **(أ)** خدمات الجهاز والجلسة والأمن (١٥ ملفًا)، **(ب)** خدمة الصوت `audioplayerservice/` (٥ ملفات)، **(ج)** خدمة الفيديو `videoplayerservice/` (٢٢ ملفًا + README)، **(د)** خدمات الشبكة والملفات.

الملاحظة العامة: معظم هذه الخدمات كلاسات **ثابتة** (`static`) أو **مفردة** (`instance`)، لأنها تمثّل موارد يملكها النظام لا الشاشة: مشغّل واحد، جلسة واحدة، جهاز واحد.

١١٫١ — طبقة الشبكة والتخزين

dio_service.dart ٦٧ سطرًا

`lib/core/services/dio_service.dart`

الدور: تهيئة `Dio` وتوفير دوال HTTP ساكنة.

- **المهلات:** اتصال ٣٠ ثانية، استقبال ٤٠، إرسال ٤٠. الاستقبال أطول لأن التطبيق يحمّل ملفات كبيرة.
- `init()` — ينشئ `Dio` بـ `baseUrl` والترويسات الأساسية، ويضع التوكن المحفوظ فورًا في `Authorization`، ويدمج `DeviceService.headers`، ثم يسجّل المعترضين بالترتيب: `DeviceInterceptor` ثم `AuthInterceptor`.
- `get/post/put/patch/delete` — أغلفة رقيقة.
- `updateToken(token)` — تحديث الترويسة بعد الدخول أو التجديد (وتُنادى بنص فارغ عند الخروج).

للتصحيح: يوجد `print(token)` في `_storedToken()` — وهذا **يطبع التوكن في سجل الإصدار**، وهو تسريب بيانات حساسة يجب إزالته. كما يستخدم النصوص الخام `'token'` بدل `CacheKeys.tokenKey`، ويعتمد على `dio!` (تأكيد غير فارغ) في كل دالة بدل التحقّق.

secure_store.dart ٧٠ سطرًا

`lib/core/services/secure_store.dart`

الدور: غلاف `FlutterSecureStorage` للأسرار (التوكن، معرّف الجهاز، ربط الجلسة)، مصمَّم على فرضية أن **التخزين الآمن قد يفشل** على بعض الأجهزة.

- الإعدادات: `AndroidOptions(resetOnError: true)` — إن تلف المخزن (يحدث فعلًا على أندرويد بعد ترقية أو استعادة نسخة احتياطية) يُصفَّر بدل أن يفشل للأبد. و`IOSOptions(accessibility: first_unlock)` — المفتاح يُقرأ بعد أول فتح للجهاز ولو كان مقفلًا، وهو ما تحتاجه الخدمات الخلفية.
- مهلة `5s` على كل عملية — فلا تتعلّق شاشة البداية إلى الأبد بسبب Keystore لا يستجيب.
- `_isAvailable` + `isAvailable` — **قاطع دائرة** (circuit breaker): بعد أول `TimeoutException` يُعلَّم المخزن «غير متاح لهذه الجلسة» فتعود كل العمليات التالية فورًا بلا انتظار ٥ ثوانٍ في كل مرة.
- `read(key)` — عند الفشل: يسجّل، **ويحذف المفتاح**، ويُرجع `null`. الحذف مقصود: قيمة لا يمكن قراءتها قيمة ميتة، وتركها يُعيد الفشل كل مرة.
- `write(key, value)` — تُرجع `bool` (نجاح/فشل) بدل الرمي، فيستطيع المنادي أن يتدبّر بديلًا.
- `delete(key)` — تسجّل تحذيرًا عند الفشل ولا ترمي أبدًا.

prd_services.dart ١١١ سطرًا

lib/core/services/prd_services.dart

الدور: تحميل ملفات PDF وإدارة مجلداتها. (اسم الملف فيه خطأ مطبعي: المقصود `pdf_services.dart`؛ الكلاس نفسه `PdfService` صحيح.)

- يستقبل `Dio` اختياريًا في المنشئ (`dio ?? Dio()`) — قابل للاختبار.
- **مجلدان منفصلان:** `libraryDirectory()` تحت `ApplicationDocuments` (تخزين دائم، لا يحذفه النظام) و `cacheDirectory()` تحت `Temporary` (قد يحذفه النظام عند ضيق المساحة). كلاهما داخل مجلد فرعي `textbooks`.
- `downloadPdf(...)` للمكتبة الدائمة و`downloadToCache(...)` للمعاينة المؤقتة — نفس المنطق بمجلد مختلف.
- كلتاهما تقبل `onProgress` و`cancelToken` — فالتقدّم **حقيقي** من Dio لا مُختلق، والإلغاء مدعوم.
- `clearCache()` — حذف تعاوُدي بلا رمي.
- `_download(...)` — ينشئ المجلد إن غاب، يحذف ملفًا سابقًا بنفس الاسم، ثم في `catch`: **يحذف الملف نصف المكتوب** ثم `rethrow`. هذا يمنع ترك ملف PDF تالف يبدو مكتملًا — والتعليق في الملف يذكر أن الإلغاء يُعالج بنفس الطريق.
- `_uniqueName(fileName)` — يُلحق طابعًا زمنيًا بالميلي ثانية فلا يتصادم تحميلان.
- `_sanitize(name)` — تنظيف ثلاثي: حذف لاحقة `.pdf`، استبدال المحارف الممنوعة في أنظمة الملفات والمسافات بشرطة سفلية، دمج الشرطات المتكرّرة، وحذفها من الطرفين.

١١,٢ — خدمات الجهاز

device_info_service.dart ١٠٧ أسطر

lib/core/services/device_info_service.dart

الدور: قراءة معلومات الجهاز مرة واحدة وتخزينها ساكنة، وأهم مخرجاتها `isLowEndDevice`.

- الحقول: `model`، `manufacturer`، `osVersion`، `isPhysicalDevice`، `androidSdkInt`، `totalRamMb`.
- `isLowEndDevice` — **ثلاثة معايير، أي واحد يكفي:** علم النظام `isLowRamDevice`، أو ذاكرة أقل من `2048` ميغابايت، أو إصدار أندرويد أقدم من Oreo (SDK 26). شرط `_totalRamMb > 0` و`_androidSdkInt > 0` يمنع تصنيف الجهاز كضعيف بسبب **فشل القراءة** لا بسبب ضعفه.
- `supportsPictureInPicture` — أندرويد و SDK ≥ 26 فقط (الحد الذي أُضيفت فيه الميزة).
- `fullModel` — يدمج الشركة مع الطراز، ويتجنّب التكرار بذكاء: إن بدأ الطراز باسم الشركة (`Samsung SM-A52`) لا يكرّرها.
- `ensureInitialized()` — تعمل مرة واحدة (`if (_initialized) return`)، وتُعلّم نفسها مهيّأة **حتى لو فشلت القراءة** فلا يُعاد المحاولة في كل مرة.
- `_read(plugin)` — `switch` يغطي سبع منصّات (أندرويد، iOS، macOS، ويندوز، لينكس، فوشيا، وفرع مبكر للويب) — والقيم تُمرَّر عبر `_orUnknown`.

> `isLowEndDevice` هي **المفتاح** لثلاثة قرارات في التطبيق: سعة كاش الصور (`AppBootstrap.tuneImageCache`)، سقف سرعة التشغيل (`PlaybackSpeed.exceedsDevice`)، وشدّة التأثيرات البصرية.

app_info_service.dart ٤٢ سطرًا

lib/core/services/app_info_service.dart

الدور: قراءة معلومات الحزمة: `appName` , `packageName` , `version` , `buildNumber` .

- `fullVersion` — بصيغة `1.2.3+45` ، وتُرجع `unknown` إن تعذّرت القراءة بدل `unknown+unknown` .
- نفس نمط `ensureInitialized()` الحمائي.

> تُستخدم في ترويسات الطلبات (`X-App-Version`) وفي فحص التحديث الإجباري وفي شاشة «حول التطبيق».

device_service.dart ١٠٣ أسطر

lib/core/services/device_service.dart

الدور: هوية الجهاز الدائمة (UUID) وبناء الترويسات. وهو أساس ربط الجلسة بجهاز واحد.

- خمس ترويسات معرّفة كثوابت: `X-Device-Uuid` , `X-Platform` , `X-App-Version` , `X-App-Build` , `X-Device-Model` ، وخمسة حقول مقابلة لها للجسم (`device_uuid` , `platform` , ...).
- `headers` (للترويسات) و `metadata` (لجسم الطلب، يُستخدم في الدخول) — نفس البيانات بشكلين.
- `ensureInitialized()` — تهيّئ `DeviceInfoService` و `AppInfoService` **بالتوازي** عبر `Future.wait` ، ثم تقرأ أو تنشئ المعرّف.
- `platform` — `switch` يُرجع `android` / `ios` صريحة، وللبقية اسم المنصة بحروف صغيرة.

`_readOrCreate()` — ثلاث طبقات لضمان ثبات المعرّف

1. **المخزن الآمن:** إن وُجد المعرّف يُستخدم ويُنسخ للمرآة.
2. **مرآة التفضيلات:** إن غاب من المخزن الآمن ووُجد في `SharedPreferences` ← يُستعاد، **ويُعاد كتابته للمخزن الآمن**، مع تسجيل تحذير.
3. **إنشاء جديد:** `Uuid().v4()` ، يُكتب في الاثنين، وإن فشل المخزن الآمن يُسجَّل تحذير بأن «المرآة هي التي تحمله».

> **لماذا المرآة؟** لأن فقدان معرّف الجهاز يعني أن الخادم يرى «جهازًا جديدًا» فيُبطل الجلسة ويُخرج الطالب بلا سبب. المخزن الآمن على أندرويد يفقد محتواه في حالات حقيقية (مع `resetOnError: true` خصوصًا)، فالمرآة في التفضيلات هي شبكة الأمان. و `_mirror` تتحقّق من القيمة قبل الكتابة فلا تكتب بلا داعٍ، وتستخدم `.ignore()` لتوضيح أن النتيجة غير مُنتظرة عن قصد.

١١٫٣ — الجلسة والأمن

device_session_service.dart ١٤٧ سطرًا

`lib/core/services/device_session_service.dart`

الدور: كل ما يتعلّق بجلسة الجهاز: الربط، التحقّق، الخروج، التنظيف، وكشف تعارض الأجهزة.

- `conflictCodes` — مجموعة ستّة رموز يرسلها الخادم للتعبير عن نفس المعنى (`device_conflict`, `device_mismatch`, `device_already_linked`, `device_already_registered`, `session_active_on_another_device`, `already_logged_in_on_another_device`). قبولها كلها حماية من تغيّر تسمية واحدة في الخادم.
- `loginPayload()` — بيانات الجهاز + رمز FCM إن وُجد، تُرسل مع الدخول.
- `hasSession` — موثّقة بتعليق: «أقصى ما يستطيع الجهاز أن يقوله عن تسجيل الدخول بلا سؤال الـ API» — أي مجرد وجود توكن محفوظ.
- `bind(accessToken)` — يحفظ في المخزن الآمن: معرّف الجهاز + **بصمة** التوكن.
- `validateCachedSession()` — تُرجع `DeviceSessionStatus`:
  - لا توكن ⟵ `noSession`.
  - لا ربط محلي محفوظ ⟵ **يُعاد الربط ويُرجع** `valid` مع تسجيل «التفويض للـ API». قرار مهم: غياب الربط المحلي ليس دليل تلاعب (قد يكون المخزن تصفّر)، والخادم هو الحكم الفعلي.
  - الربط موجود ⟵ تُقارن معرّف الجهاز **وبصمة** التوكن؛ التطابق ⟵ `valid`، وإلا ⟵ `deviceMismatch`.
- `_fingerprint(value)` — دالة تلبيد **FNV-1a** مكتوبة يدويًا (البذرة `0x811C9DC5` والمضاعِف `0x01000193` مع قناع ٣٢ بت) تُرجع نصًا ست عشريًا. الهدف ليس التشفير بل **عدم تخزين التوكن مرتين**: تُقارَن البصمة لا التوكن نفسه، فحتى لو قُرئ هذا المفتاح لا يُستخرج منه التوكن.
- `logout()` — يُبلغ الخادم بمهلة `_remoteLogoutTimeout = 5s`، وفي `finally` ينادي `clear()` **دائمًا**. فشل الشبكة لا يمنع الخروج المحلي.
- `clear()` — تنظيف شامل بسبع خطوات: حذف التوكنين، `CurrentUser.clear()`, `DioService.updateToken('')`, `ScreenCapturePolicy.clear()`, `GalleryDataSource.clearCache()`, وحذف مفتاحي الربط.
- `isDeviceConflict(error)` — `true` لرمز HTTP 409 أو لوجود أحد رموز التعارض.
- `_carriesConflictCode(data)` — يبحث في ثلاثة حقول (`code`, `error`, `error_code`) ثم **يغوص تعاوُديًا** في `data['data']` — فيجد الرمز سواء كان في الجذر أو متداخلًا.

> التعليق المفصّل على `_remoteLogoutTimeout` يشرح الموازنة: الخادم يستحق لحظة انتظار، لكنه لا يستحق ترك توكن صالح على الجهاز بينما شبكة ميتة تستهلك مهلة الطلب كاملة.

session_guard.dart ١٠٠ سطر

lib/core/services/session_guard.dart

الدور: المكان **الوحيد** في التطبيق الذي يُسقط جلسة غير قابلة للاستخدام ويعيد الطالب لشاشة الدخول. موثّق توثيقًا ممتازًا في الملف.

- `_isSigningOut` — علم يمنع التنفيذ المتزامن. فلو فشلت ثلاثة طلبات معًا بـ 403، يحدث خروج واحد لا ثلاثة (وإلا ظهرت ثلاث رسائل وثلاث عمليات تنقّل).
- `endIfRefused(error)` — **بوابة القرار**: لا تفعل شيئًا إن لم يكن الخطأ منهيًا للجلسة (`error.endsSession`)، ولا إن لم تكن هناك جلسة أصلًا. الشرط الثاني يمنع إزعاجًا حقيقيًا: دخول مرفوض، أو طلب تأخّر وفشل بعد خروج المستخدم بنفسه.
- `forceSignOut({noticeKey})` —
  ١. يلتقط `hadSession` قبل التنظيف.
  ٢. `DeviceSessionService.clear()`.
  ٣. يجلب الـ Navigator؛ إن غاب يُسجّل تحذيرًا ويخرج (الجلسة نُظّفت، وشاشة الدخول ستُرى عند الإقلاع التالي).
  ٤. `pushNamedAndRemoveUntil(login)`.
  ٥. يُظهر الإشعار **فقط** إن كانت هناك جلسة فعلًا (`noticeKey != null && hadSession`).
  ٦. `finally` يُرخي العلم.
- `_showNotice` — يستخدم `addPostFrameCallback` ويتحقّق من `navigator.mounted` ثم `AppToast.error`. التأجيل لإطار واحد ضروري: إظهار Toast أثناء انتقال مسار يفشل أو يظهر في المكان الخطأ.

> **الفلسفة الموثّقة في الملف:** هذا الملف **لا يقرّر** أي الأخطاء تُنهي الجلسة — القرار يُقرأ من `ErrorMapper`. والتنظيف والتنقّل يُنفَّذان دائمًا (حتى لو لم تُعرض الرسالة) لأن جهة أخرى قد تكون أسقطت الجلسة بينما شاشة محمية معروضة، ولا يجوز أن تبقى تلك الشاشة قائمة.

account_status_watcher.dart ١٥٠ سطرًا

lib/core/services/account_status_watcher.dart

الدور: يمنع التطبيق من الاستمرار على جلسة أغلقتها المنصّة من لوحة التحكّم. المشكلة الموثّقة: إيقاف الحساب إجراء في لوحة التحكّم، ولا شيء على الجهاز يعلم به إلا بسؤال الـ API.

- `pollInterval = 60s` — موثّق: قصير بما يكفي ليُسقَط الحساب الموقوف والطالب ما زال ينظر للشاشة، وطويل بما يكفي ليكلّف طلبًا صغيرًا في الدقيقة.
- `initialize()` — يشترك في مجريين: `PushEventBus` (للإشعارات) و`AppLifecycleService.onStateChanged`، ثم يبدأ المؤقّت. استخدام `??=` يمنع الاشتراك المزدوج.
- `readProfile()` — **دمج الطلبات المتزامنة** بنفس نمط `_refreshFuture` في معترض المصادقة: المنادون يتشاركون طلبًا واحدًا. موثّق بسببه: العودة للمقدّمة تجعل سياسة التصوير **وهذا المراقب** يطلبان الملف الشخصي، والـ API يجب أن يُسأل مرة لا مرتين.
- `verifyNow()` — غلاف لا ينتظر النتيجة.
- `_onPush(data)` — يتجاهل إن لا جلسة، ويتجاهل إن لم يكن الإشعار عن الحساب. والتعليق يوضح القاعدة الذهبية: **الإشعار ليس حكمًا** (قد يكون إعادة تنشيط لا إيقافًا) بل **دافع للسؤال**.
- `_isAccountStatusPush(data)` — يفحص ثلاثة أسماء حقول (`type`، `notification_type`، `kind`) ويقارن بـ `NotificationKind.accountStatus`.
- `_onLifecycleChanged(state)` — عند الاستئناف يعيد تشغيل المؤقّت (**ولا يسأل فورًا** لأن سياسة التصوير تسأل أصلًا — تعليق صريح)، وفي أي حالة أخرى **يُلغي المؤقّت** فلا يستهلك التطبيق شبكة وبطارية في الخلفية.
- `_readProfile()` — يلتقط الخطأ ويُرجع `null`، لأن الجلسة المغلقة **قد انتهت أصلًا** عبر `SessionGuard` قبل أن يصل الخطأ هنا.

> التعليق الأهم في الملف: هذه الخدمة **لا تقرّر شيئًا**. فحالة جديدة تخترعها لوحة التحكّم بعد إصدار هذا البناء تُنهي الجلسة أيضًا، بلا أن تكون قد سُمّيت في أي مكان في التطبيق — لأن القرار مبني على **رمز الرفض** لا على قائمة حالات معروفة. هذا تصميم مقاوم للمستقبل بامتياز.

app_lifecycle_service.dart ٥٣ سطرًا

lib/core/services/app_lifecycle_service.dart

الدور: مفرد (`instance`) يراقب دورة حياة التطبيق ويبثّها كمجريين، فتستطيع الـ Cubits والخدمات الاستجابة بلا `WidgetsBindingObserver` في كل واحدة.

- `onStateChanged` — كل تغيّر حالة (للخدمات التي تعمل في المقدّمة فقط).
- `onResumed` — فقط لحظات العودة للمقدّمة (لتحديث البيانات).
- كلا المجريين `broadcast` فيدعمان مشتركين متعدّدين.
- `initialize()` — يسجّل المراقب ويقرأ الحالة الابتدائية من `WidgetsBinding`.
- `didChangeAppLifecycleState` —
  - `if (previous == state) return;` — إسقاط التكرار (النظام يرسل نفس الحالة مرتين أحيانًا).
  - يبثّ على `_states` بحماية `isClosed`.
  - عند الاستئناف فقط: `unawaited(ScreenCapturePolicy.refresh())` ثم يبثّ على `_resumed`.

screen_capture_policy.dart ٦١ سطرًا

lib/core/services/screen_capture_policy.dart

الدور: يقرّر **هل هذا المستخدم مستثنى** من منع تصوير الشاشة، ويُطبّق القرار على الخدمة.

- `_isExempt` + `isExempt`.
- `initialize()` — يشترك في مجرى تقارير `ScreenCaptureService`.
- `_onReport(response)` — **آلية ذكية**: استجابة الخادم لتقرير أمني تحمل الصلاحية المحدّثة؛ فإن تغيّرت تُطبَّق فورًا. أي أن المشرف يستطيع منح استثناء من لوحة التحكّم، ويصل أثره عند أول تقرير بلا انتظار دورة كاملة.
- `resolve({canCaptureScreen})` — يحدّث العلم، يكتبه في بيانات المستخدم المخزّنة (إن وُجدت جلسة)، ثم ينادي `applyExemption`.
- `restore()` — يستعيد القيمة من بيانات المستخدم المحليّة (عند الإقلاع)، أو `clear()` إن لا جلسة.
- `refresh()` — يعيد قراءة الصلاحية من الـ API عبر `AccountStatusWatcher.readProfile()`. والتعليق يشرح أن هذا **أيضًا** ما يكشف جلسة أغلقتها المنصّة: القراءة تمرّ بالمراقب، والرفض يُنهي الجلسة في طريق العودة. وعند تعذّر القراءة **يُحتفظ بالاستثناء الحالي** لا يُصفَّر.
- `clear()` — `resolve(canCaptureScreen: false)`، أي **المنع هو الحالة الافتراضية**.

screen_capture_service.dart ٢١٠ أسطر

lib/core/services/screen_capture_service.dart

الدور: كشف التقاط/تسجيل الشاشة على المنصّتين ورفع الحدث للخادم. مفرد.

- `EventChannel('bilhikma/screen_capture')` — قناة أصلية مخصّصة لأندرويد (كود كوتلن في مجلد المنصّة).
- على iOS يستخدم حزمة `screen_protector` مع ردّي نداء: `_onScreenshot` و`_onScreenRecord`.
- `_debounce = 2s` — لمنع تكرار التقارير.
- `attach(repo)` — حقن المستودع (قابلية الاختبار).
- `applyExemption(isExempt)` — المستثنى ⟵ `stop()`، وغيره ⟵ `start()`.
- `start()` / `stop()` — محمية بعلم `_isListening`، و`stop` تُلغي الاشتراك وتُزيل مستمع iOS.
- **مكدس الشاشات:** `pushScreen(screen, {contentId})` يُرجع رمزًا (token)، و`popScreen(token)` يُزيله، و`_current` تُرجع آخر عنصر. **السبب:** حين يُلتقط صورة، نحتاج أن نعرف **أي شاشة وأي درس** كان معروضًا. المكدس (لا متغيّر واحد) هو الصحيح لأن الشاشات تتداخل، والحذف بالرمز لا بالموضع يعني أن إزالة شاشة من المنتصف لا تُفسد الترتيب.
- `report(eventType, ...)` — ثلاث بوابات قبل الإرسال: مستثنى؟ لا جلسة؟ مكرّر؟ ثم يرسل الحدث مع `contentId` و`meta` فيها اسم الشاشة، ويثبّت الاستجابة على `_reports`. الفشل يُسجَّل ولا يُرمى.
- `_onNativeEvent(event)` — يتحقّق أن الحدث خريطة، يحوّل `type` لتعداد، ويفحص `active`: الحدث غير النشط يُحدّث `_isRecording = false` ويخرج. ثم يرفع التقرير مع `source` و`display_name` إن وُجدا (للشاشة الخارجية).
- `_isDuplicate(eventType)` — **تفصيل ذكي:** لقطة الشاشة **لا تُعتبر مكرّرة أبدًا** (`return false`) لأن كل لقطة حدث مستقل يجب رفعه. أما التسجيل والشاشة الخارجية فتُكتم خلال نافذة الثانيتين، لأن النظام يرسل عنها أحداثًا متتالية لحدث واحد.

> يستخدم `debugPrint` (مع رمز تعبيري) بدل `AppLogger` في عدة مواضع — غير متّسق مع بقية المشروع.

screen_wake_service.dart ٦٧ سطرًا

`lib/core/services/screen_wake_service.dart`

الدور: إبقاء الشاشة مضاءة أثناء تشغيل الدرس. التوثيق في أعلى الملف من أفضل ما في المشروع ويشرح المشكلة بدقة تقنية: لا `video_player` ولا `youtube_player_flutter` يلمس قفل إضاءة الشاشة — فـ `video_player_android` لا يضع `FLAG_KEEP_SCREEN_ON`، و`video_player_avfoundation` لا يُعطّل مؤقّت الخمول في iOS. وصلاحية `WAKE_LOCK` المعلنة في الـ manifest هي القفل **الجزئي** الذي تحتاجه `audio_service` — يُبقي المعالج حيًّا لا الشاشة. فبلا هذه الخدمة تنطفئ الشاشة في منتصف الدرس.

- `_owners` — **عدّ مراجع** بمجموعة مالكين: القفل يُمسك ما دام **أي** مالك يريده، ويُرخى عند تخلّي الأخير. ضروري لأن مرحلة الفيديو ومرحلة الصوت والمشغّل العائم قد تعيش معًا.
- `isHeld`.
- `request(owner, {keepAwake})` — تضيف أو تحذف المالك، **وتنادي المزامنة فقط إن تغيّرت المجموعة فعلًا**. موثّق: آمنة للنداء في كل نبضة تشغيل لأن الطلبات المتطابقة تُسقَط قبل الوصول لقناة المنصّة.
- `release(owner)` — موثّقة بالاستخدام الصحيح: نادِها من `dispose()` فلا يُسرّب القفل مشغّل هُدم أثناء التشغيل.
- `_sync()` — محمية بـ `_isSyncing`، وتستخدم `while (_owners.isNotEmpty != _applied)` — **حلقة لا شرط**: التعليق يشرح أن المالكين قد يأتون ويذهبون أثناء تنفيذ تبديل غير متزامن، فالحلقة تضمن الوصول للحالة المطلوبة أخيرًا. الفشل يُسجَّل كتحذير فقط (رفض المنصّة للقفل لا يجب أن يُعطّل التشغيل).

١١,٤ — الإشعارات

push_event_bus.dart ٢٢ سطرًا

`lib/core/services/push_event_bus.dart`

الدور: ناقل أحداث بسيط (مفرد) يبثّ حمولة كل إشعار. موثّق: يسمح للـ Cubits بتحديث بياناتها **بلا اعتماد على Firebase ولا على شجرة الودجات**.

- `stream` — `broadcast`، يبثّ مرة لكل إشعار يصل والتطبيق يعمل، ومرة لكل إشعار ينقره المستخدم.
- `emit(data)` — محمية بـ `isClosed`.

> فائدة معمارية كبيرة من ٢٢ سطرًا: `NotificationsCubit` يزيد عدّاد غير المقروء، و`HomeCubit` يُحدّث قائمته، و `AccountStatusWatcher` يتحقّق من الحساب — **كلها من نفس الإشعار وبلا أن تعرف أي منها بوجود Firebase**.

push_notification_service.dart ٢٨٥ سطرًا

lib/core/services/push_notification_service.dart

الدور: كل تكامل Firebase Messaging + الإشعارات المحليّة. مبني بالكامل على قاعدة: **فشل الإشعارات لا يجوز أن يمنع التطبيق من العمل**.

- قناة أندرويد `high_importance_channel` بأهمية عالية.
- مهلتان: `_remoteTimeout = 20s` للعمليات الشبكية و`_localTimeout = 8s` للمحليّة.
- `ensureInitialized()` — محمية بـ `_hasStarted`، وتتسلسل: تهيئة Firebase (وتخرج إن فشلت) ⟵ الإشعارات المحليّة ⟵ طلب الصلاحية ⟵ ربط المستمعين ⟵ جلب الرمز ⟵ معالجة نقرة الإقلاع. ثم تضبط `_isReady` وتسجّل النتيجة.
- `_initializeFirebase()` — تُرجع `bool`؛ الفشل يُسجَّل ويُوقف المسار كله بلا رمي.
- `_initializeLocalNotifications()` — تهيئة + إنشاء القناة، والفشل يُسجَّل برسالة دقيقة: «إشعارات المقدّمة لن تُعرض» — أي أن أثر الفشل محدود ومعروف.
- `_attachMessageListeners()` — ثلاثة مستمعين: `onTokenRefresh` (يحدّث الرمز ويعيد تسجيله)، `onMessage` (التطبيق مفتوح)، `onMessageOpenedApp` (نقر إشعار والتطبيق في الخلفية).
- `registerToken()` — **بوابة مهمة**: تقرأ ترويسة `Authorization` وتخرج إن كانت `'Bearer'` فقط (أي بلا توكن). فلا يُرفع رمز FCM لمستخدم غير مسجّل — وإلا لربط الخادم الرمز بحساب خاطئ.
- `_handleLaunchTap()` — مساران: `getInitialMessage()` لإشعار Firebase الذي أقلع التطبيق، وإلا `getNotificationAppLaunchDetails()` للإشعار المحلي.
- `_onForegroundMessage` — يبثّ على الناقل **ثم** يعرض إشعارًا محليًا (لأن Firebase لا يعرض إشعارات والتطبيق مفتوح).
- `_openFromMessage` / `_openFromPayload` — يبثّان على الناقل ثم يفتحان الوجهة عبر `NotificationLauncher`. و `_openFromPayload` يفكّ JSON بحماية `FormatException`، وفي كل حالات الفشل يفتح `null` (أي شاشة الإشعارات العامة) بدل لا شيء.
- `_showForegroundNotification` — يعرض الإشعار المحلي ويضع `jsonEncode(message.data)` في الحمولة فتعمل وجهة النقر لاحقًا.

> **النمط المتّسق:** كل دالة خارجية في هذا الملف محاطة بـ `try/catch` مع `AppLogger` و`timeout`. فلا توجد نقطة واحدة يستطيع فيها فشل Firebase أن يُسقط التطبيق أو يعلّقه.

notification_launcher.dart ٦٣ سطرًا

lib/core/services/notification_launcher.dart

الدور: يحل مشكلة التوقيت: **الإشعار قد يصل قبل أن تكون شاشة التطبيق جاهزة** (أثناء الإقلاع، أو والمستخدم على شاشة الدخول).

- `_pending` — وجهة محتجزة، و `_isReady` — هل التطبيق جاهز للتنقّل.
- `open(destination)` — الوجهة المعروفة تُعالج فورًا؛ و `null` (أي «إشعار بلا وجهة واضحة») تُحتجز كوجهة لشاشة الإشعارات **فقط إن لم يكن التطبيق جاهزًا** — فلا نُقحم المستخدم الجاهز على شاشة لم يطلبها.
- `markReady()` — يُنادى عند بناء مسار `home`: يرفع العلم وينفّذ المحتجز عبر `addPostFrameCallback` (أي بعد انتهاء بناء الإطار الحالي).
- `markSignedOut()` — يُنادى عند بناء مسار `login`: يُنزل العلم **ويمسح المحتجز**. حاسم: إشعار وصل قبل الخروج لا يجوز أن يفتح شاشة محميّة بعده.
- `_handle` — غير جاهز ← احتجاز؛ جاهز ← دفع.
- `_push` — إن كان الـ Navigator غائبًا **يُعيد الاحتجاز** بدل الفقدان.
- `_navigator` — يفحص `getIt.isRegistered` أولًا فلا يرمي لو نُودي قبل تهيئة الحاقن.

> النتيجة: نقر المستخدم على إشعار درس والتطبيق مغلق ← يُقلع التطبيق، تُحتجز الوجهة، وبعد تسجيل الدخول والوصول للرئيسية **تُفتح شاشة الدرس الصحيحة**. وهذا سلوك يصعب تحقيقه بلا هذا الاحتجاز.

١١,٥ — `audioplayerservice/` — خدمة الصوت (٥ ملفات)

> مشغّل صوت كامل يعمل **في الخلفية** مع إشعار نظام وأزرار تحكّم، مبني على `audio_service` + `just_audio`. البنية: `AudioPlayerHandler` هو العقل، و `AudioTrack` هو النموذج المحمول، و `PlaybackUriResolver` هو نقطة التوسّع.

audioplayerservice.dart ٦ أسطر

lib/core/services/audioplayerservice/audioplayerservice.dart

الدور: ملف «برميل» (barrel): `library;` ثم تصدير الملفات الأربعة. فالمستهلك يكتب استيرادًا واحدًا بدل أربعة، ويمكن تغيير البنية الداخلية بلا كسر المستوردين.

audio_track.dart ٨٨ سطرًا

lib/core/services/audioplayerservice/audio_track.dart

الدور: نموذج المقطع الصوتي، **محمول تمامًا** (لا يعرف شيئًا عن `LessonModel` ولا عن ثيم التطبيق).

- الحقول: `id`, `title`, `artist`, `source` (الرابط أو المسار أو الأصل)، `imageUrl`, `localAudioPath`, `localImagePath`.
- `toMediaItem()` — يحوّل إلى `MediaItem` الذي تفهمه `audio_service`، ويضع المسارات المحليّة والرابط في `extras` فلا تُفقد.
- `_artUri()` — **أولوية الصورة المحليّة على الشبكية**: فإشعار النظام يظهر بغلافه فورًا وبلا إنترنت.
- `fromMediaItem(item)` — التحويل العكسي (يحتاجه `addQueueItem` عندما يأتي الطلب من نظام التشغيل). يستخرج `imageUrl` من `extras`، أو من `artUri` إن بدأ بـ `http`.
- `copyWith(...)`.

playback_uri_resolver.dart ٣٥ سطرًا

lib/core/services/audioplayerservice/playback_uri_resolver.dart

الدور: نقطة التوسّع الأساسية في الخدمة: واجهة مجرّدة `PlaybackUriResolver` بدالة واحدة `resolve(track) → Uri`، مع تنفيذ افتراضي.

`DefaultPlaybackUriResolver` يجرّب أربع حالات بالترتيب:

1. المصدر يبدأ بـ `assets/` ⟵ `asset:///...`.
2. يوجد `localAudioPath` **والملف موجود فعلًا** (`await f.exists()`) ⟵ ملف محلي. التحقّق من الوجود مهم: مسار محفوظ لملف حُذف يجب أن يتراجع للشبكة لا أن يفشل.
3. المصدر يبدأ بـ `http(s)` ⟵ شبكة.
4. المصدر مسار ملف موجود ⟵ ملف. وإلا يُرجَع كما هو.

> **لماذا واجهة مجرّدة؟** لأن هذا التطبيق يحتاج **رمز وصول مؤقتًا** لكل وسيط محمي (`MediaAccessModel`). فيستطيع حاقن مخصّص أن يطلب الرمز من الـ API ويبني الرابط الموقّع عند التشغيل — بلا أي تعديل على `AudioPlayerHandler`. وهذا ما يحفظ قابلية نقل الخدمة كاملة.

audio_player_service.dart ٥٧ سطرًا

lib/core/services/audioplayerservice/audio_player_service.dart

الدور: تهيئة الخدمة + حل مشكلة غلاف الإشعار. كلاسان:

`AudioPlayerService.init(...)`

- يهيّئ `JustAudioBackground` بمعرّف قناة الإشعار واسمها، و`androidNotificationOngoing` (إشعار لا يُمسح بالسحب أثناء التشغيل).
- يقبل `resolver` مخصّصًا و`defaultArtAsset`.
- يُنشئ `AssetNotificationArt` و**يبدأ تجهيزه فورًا بلا انتظار** (`unawaited(art.uri())`) — تسخين مسبق فيكون الغلاف جاهزًا قبل أول تشغيل.
- يُرجع `AudioPlayerHandler` مهيّأً.

`AssetNotificationArt`

المشكلة: إشعار النظام لا يستطيع قراءة أصول Flutter (`assets/...`) — يحتاج ملفًا حقيقيًا على القرص. **الحل:** تقرأ الأصل من `rootBundle`، تكتبه في المجلد المؤقّت كملف، وتُرجع `Uri.file`، **وتُخزّن النتيجة** (`_cached`) فلا تُعاد الكتابة. وتُرجع `null` على الويب حيث لا نظام ملفات.

audio_player_handler.dart ٢٩٧ سطرًا

lib/core/services/audioplayerservice/audio_player_handler.dart

الدور: أكبر ملف في خدمة الصوت وقلبها. يمدّد `BaseAudioHandler` ويخلط `QueueHandler` و`SeekHandler`، فيستجيب لأزرار إشعار النظام وشاشة القفل والسمّاعات تلقائيًا.

الحالة الداخلية

- `_player` — نسخة `AudioPlayer` واحدة.
- `_playQueue` + `_queueIndex` — قائمة التشغيل والموضع فيها.
- `_repeatOneEnabled`, `_shuffleEnabled`, `_shuffleHistory`, `_random`.
- `_playbackSpeed`.
- `_playChain` — **الحقل الأهم:** `Future` متسلسل.

> **مشكلة `_playChain` وحلّها:** لو نقر المستخدم «التالي» ثلاث مرات بسرعة، تنطلق ثلاث عمليات تحميل متوازية، وتنتهي بترتيب غير مضمون، فيُشغَّل المقطع الخطأ. الحل هنا: كل عملية تشغيل تُربط بنهاية السابقة (`_playChain.then((_) => _loadAndPlay(track))`)، فتُنفَّذ العمليات **بالتتابع** والنتيجة النهائية هي آخر نقرة دائمًا. ولاحظ `_playChain = run.catchError((_) {})` — فشل عملية لا يكسر السلسلة لمن بعدها.

المنشئ

- يشترك في `playbackEventStream` ⟵ `_broadcastState`.
- يشترك في `durationStream` ⟵ يحدّث مدة `mediaItem`. ضروري لأن المدة الحقيقية تُعرف بعد بداية التحميل، وبلا هذا التحديث يظهر شريط الإشعار بمدة صفر.

الدوال الرئيسية

- `hasNext` / `hasPrevious` — تحسبان بحسب الوضع: في الخلط يُعتمد على `_shuffleHistory`، وفي الترتيب العادي على الموضع.
- `playTrack(track, {contextQueue, contextIndex})` — تشغيل مقطع **مع سياقه**: إن مُرّرت قائمة يُعتمد عليها (مع `clamp` للفهرس)، وإلا يُنشأ طابور من مقطع واحد.
- `_loadAndPlay(track)` — التسلسل الكامل: إيقاف ما سبق (بـ `try/catch` فارغ) ⟵ بناء `MediaItem` ⟵ إضافة الغلاف الاحتياطي إن غاب ⟵ نشر الطابور والعنصر الحالي (**قبل** التحميل، فتُحدَّث الواجهة فورًا) ⟵ حلّ الـ URI ⟵ ضبط المصدر ⟵ تطبيق وضع التكرار والسرعة ⟵ التشغيل.
- `_sourceForResolvedUri(uri, tag, {useLockCaching})` — يُرجع `LockCachingAudioSource` لروابط HTTP (**تُحمَّل أثناء التشغيل وتُخزَّن** فإعادة السماع لا تستهلك بيانات)، و`AudioSource.uri` العادي للملفات والأصول وللويب.
- **تراجع ذكي في `_loadAndPlay`:** يُحاول أولًا بـ `useLockCaching: true`، وعند الفشل (قرص ممتلئ، صلاحيات، رابط لا يدعم الطلبات الجزئية) **يُعيد المحاولة بلا تخزين**. فالتشغيل ينجح على حساب الكاش لا يفشل.
- `play`, `pause`, `seek`, `stop` — تجاوزات مباشرة.
- `setPlaybackSpeed(speed)` — مع `clamp(0.25, 2.0)`.
- `skipToNext()` — **حالات متعدّدة:** طابور فارغ ⟵ لا شيء؛ مقطع واحد ⟵ إعادة من البداية؛ خلط ⟵ يُسجّل الحالي في التاريخ ويختار عشوائيًا **من غير الحالي** (لا تكرار فوري)؛ ترتيب عادي وآخر الطابور ⟵ إعادة من البداية؛ وإلا التالي.
- `skipToPrevious()` — في الخلط يرجع عبر `_shuffleHistory.removeLast()` فيعود للمقاطع **بترتيبها الفعلي** لا عشوائيًا؛ وإلا السابق؛ وإلا إعادة من البداية (سلوك المشغّلات المعتاد).

- `_publishPreviewMediaItem(track)` — **تفصيل تجربة مستخدم ممتاز:** ينشر بيانات المقطع الجديد **قبل** تحميله، فيتغيّر العنوان في الإشعار فورًا عند النقر بدل أن يبقى القديم ثانية أو ثانيتين.
- `setRepeatMode` / `setShuffleMode` — تُحدّثان الحالة الداخلية **و** تنشران `playbackState` فتتحدّث أيقونات نظام التشغيل.
- `addQueueItem` / `removeQueueItem` — تكامل مع طلبات النظام.
- `_broadcastState(event)` — يبني حالة التشغيل للنظام: أزرار (سابق / تشغيل أو إيقاف / تالي)، فهارس الأزرار المضغوطة في أندرويد، إجراءات النظام المسموحة (تقديم/تأخير)، وخريطة تحويل `ProcessingState` إلى `AudioProcessingState`. ولاحظ `playing` تستثني حالة `completed` — فلا يبقى الإشعار يقول «يُشغَّل» بعد انتهاء المقطع.

ملاحظات: الملف يستخدم `var` و`final` بلا أنواع صريحة بخلاف بقية المشروع، ويحتوي `catchError` فارغًا في مواضع (كتم أخطاء بلا تسجيل) — إضافة `AppLogger` هناك ستُسهّل التصحيح.

١١,٦ — `videoplayerservice/` — خدمة الفيديو (٢٢ ملفًا + README)

وحدة مستقلّة تمامًا — الـ README ينصّ صراحة: «لا اعتماد على بقية التطبيق (لا `Lesson`، ولا ثيم التطبيق) — انسخ المجلد كاملًا إلى مشروع آخر». توفّر مشغّلًا عائمًا: عرض كامل بإيماءات مخصّصة، يتقلّص إلى مربّع عائم قابل للسحب وتغيير الحجم بالقرص، **مع** دعم «صورة داخل صورة» الأصلي في أندرويد عند مغادرة التطبيق. وتشغّل كل أنواع الفيديو **بواجهة واحدة**: يوتيوب (عادي/مباشر/shorts/embed) عبر محرّك IFrame، و `mp4` / `webm` / HLS/DASH عبر `video_player`.

README.md

`lib/core/services/videoplayerservice/README.md`

الدور: توثيق كامل للوحدة: جدول المصادر والمحرّكات، جدول الملفات ووظائفها، التبعيات المطلوبة، خطوات إعداد PiP على أندرويد، أمثلة الاستخدام، وقائمة قرارات تصميمية مشروحة. ومن أبرز ما فيه:

- الضوابط تُسلَّم **داخل** سطح المشغّل (`buildSurface`) لا تُكدَّس حوله، لأن محرّك يوتيوب يرسم عبر `platform view` ويجب أن تسكن طبقته داخله ليتركّب العرض صحيحًا.
- `GlobalKey` مشترك يُعيد توطين نفس المشغّل بين «موسّع ⇄ عائم» و«طولي ⇄ عرضي»، **فلا يُعاد التشغيل من البداية** عند تغيّر التخطيط.
- نبضات التشغيل **لا تُمرَّر** عبر `FloatingVideoController`؛ الودجات الطرفية تستمع لـ `VideoPlayback` مباشرة، فتحديث الموضع لا يُعيد بناء سطح الفيديو.
- اختيار جودة يوتيوب مُزال لأن `setPlaybackQuality` لم يعد يفعل شيئًا في يوتيوب الحديث؛ فالمنتقي صار للسرعة، وهي تعمل على المحرّكين.
- `isLive` مجرّد تخمين: يصحّح نفسه بمجرّد أن يُبلّغ المجرى عن مدة (أو يعجز).

videoplayerservice.dart ٩ أسطر

`lib/core/services/videoplayerservice/videoplayerservice.dart`

برميل تصدير لسبعة ملفات عامة (ولاحظ أنه **لا يصدّر** مجلد `widgets/` — فهي تفاصيل داخلية).

video_item.dart ١٦٩ سطرًا

lib/core/services/videoplayerservice/video_item.dart

الدور: النموذج المحمول + كل منطق التعرّف على روابط يوتيوب والبث المباشر.

- الحقول: `id` , `kind` , `source` , `isLive` , `title` , `subtitle` , `description` , `date` , `shareText` , `httpHeaders` .
- `VideoItem.youtube(...)` — بمعرّف فيديو.
- `VideoItem.network(...)` — برابط، و`isLive` اختياري: إن لم يُمرَّر يُخمَّن بـ `looksLive(url)` .
- `VideoItem.fromUrl(url, ...)` — **الدالة التي يُنصح باستخدامها**: تُرجع `null` للرابط الفارغ أو ليوتيوب بلا معرّف مستخرج، وإلا تختار النوع تلقائيًا. و`isLive` ليوتيوب يُخمَّن من وجود `/live/` في الرابط.
- `youtubeVideoId` و`mediaUri` — وصول آمن بحسب النوع (`null` للنوع الآخر).
- `isYoutubeUrl(url)` — يفحص المضيف لثلاثة نطاقات: `youtube.com` , `youtube-nocookie.com` , `youtu.be` .
- `extractYoutubeId(url)` — يغطي كل أشكال روابط يوتيوب: `youtu.be/ID` (أول مقطع)، ثم يبحث عن العلامات `live` / `shorts` / `embed` / `v` ويأخذ ما بعدها، وأخيرًا معامل الاستعلام `v` .
- `looksLive(url)` — يبحث عن `.m3u8` أو `/live/` أو `livestream` .

video_playback.dart ٨٧ سطرًا

lib/core/services/videoplayerservice/video_playback.dart

الدور: الواجهة المجرّدة التي تتحدّث معها الواجهة — **محايدة تجاه المحرّك**. تمدّد `ChangeNotifier` .

- `VideoPlayback.forItem(item)` — مصنع يختار المحرّك بـ `switch` على النوع. **هذا السطر الواحد هو كل ما تعرفه الواجهة عن وجود محرّكين.**
- الخصائص المجرّدة: `isReady` , `isPlaying` , `isBuffering` , `isLive` , `position` , `duration` , `playbackSpeed` , `errorMessage` .
- `aspectRatio` — غلاف محمي حول `reportedAspectRatio` : إن كانت القيمة غير منتهية أو سالبة أو صفرًا يُستخدم `16/9` . يحمي من تخطيط منهار بسبب محرّك لم يُبلّغ بعد.
- `canSeek` — ليس بثًّا مباشرًا **و** له مدة موجبة.
- `togglePlayPause()` و`seekBy(delta)` — منطق مشترك مكتوب مرة واحدة؛ و`seekBy` تقصّ الهدف بين الصفر والمدة وتخرج بلا عمل إن تعذّر التنقّل.
- `buildSurface({key, overlay})` — كل محرّك يبني سطحه ويضع الضوابط داخله.

> **أذكى ما في الملف:** تجاوز `notifyListeners()` نفسه لينادي `ScreenWakeService.request(this, keepAwake: keepsScreenAwake && isPlaying)` . التعليق يشرح: كل تغيّر حالة من **كلا** المحرّكين يمرّ من هنا، فهذه هي النقطة الوحيدة التي تُبقي قفل الإضاءة متوافقًا مع `isPlaying` — مُمسَكًا أثناء التشغيل، ومُرخى عند الإيقاف والانتهاء والخطأ والهدم. و`dispose` يُرخي القفل أيضًا.
> الفائدة: **يستحيل** أن يُنسى تحرير القفل، لأن التحرير مربوط بآلية الإشعار نفسها لا بنداء يدوي.

youtube_video_playback.dart ١١٨ سطرًا

lib/core/services/videoplayerservice/youtube_video_playback.dart

الدور: تنفيذ `VideoPlayback` فوق مشغّل يوتيوب IFrame.

- في المنشئ: `YoutubePlayerFlags` بـ `autoPlay: true` و`hideControls: true` و`hideThumbnail: true` و`controlsVisibleAtStart: false` — **إخفاء كامل لضوابط يوتيوب** لأن التطبيق يرسم ضوابطه الخاصة.
- `_onValue()` — مستمع واحد ينقل كل الحالة من `YoutubePlayerValue` إلى الحقول المحليّة ثم ينادي `notifyListeners()`.
- `isLive` — `item.isLive` **أو** (جاهز ومدته صفر) — التصحيح الذاتي المذكور في الـ README.
- `reportedAspectRatio` — دائمًا `16/9` لأن محرّك IFrame لا يُبلّغ عن النسبة.
- `errorMessage` — يُبنى من `value.errorCode`.
- `buildSurface` — يبني `YoutubePlayer` بلا مؤشّر تقدّم ولا أزرار، ويضع الطبقة (إن وُجدت) في `Stack` فوقه.
- `dispose` — يُزيل المستمع ثم يهدم المتحكّم ثم `super.dispose()` (الذي يُرخي قفل الشاشة).

network_video_playback.dart ١٢٢ سطرًا

lib/core/services/videoplayerservice/network_video_playback.dart

الدور: تنفيذ `VideoPlayback` فوق `video_player` (ExoPlayer على أندرويد / AVPlayer على iOS).

- `_value` — **نمط وقائي ممتاز**: `_controller?.value ?? const VideoPlayerValue.uninitialized()`، فكل الخصائص تُقرأ بلا أي فحص `null`، وتُرجع قيمًا منطقية قبل التهيئة.
- `_isDisposed` — علم يُفحص بعد كل `await` فلا يُلمس متحكّم مهدوم (سبب شائع للانهيار).
- `initialize()` — يتحقّق من صلاحية الـ URI ويضع رسالة خطأ واضحة إن فسد، ثم ينشئ المتحكّم بـ `networkUrl` **مع** `httpHeaders` (وهو ما يتيح تشغيل الوسائط المحميّة بترويسة مصادقة)، ويشترك، ثم `initialize()` و`play()` داخل `try/catch`.
- `isLive` — نفس منطق التصحيح الذاتي.
- `reportedAspectRatio` — النسبة **الحقيقية** من المحرّك عند الجهوزية.
- `errorMessage` — `_errorMessage ?? _value.errorDescription` — أي خطأ التهيئة أولًا ثم خطأ المحرّك.
- `buildSurface` — خلفية سوداء (`ColoredBox`) تمنع الوميض الأبيض، وداخلها `AspectRatio` + `VideoPlayer` عند الجهوزية فقط.
- `dispose` — يرفع العلم أولًا، يُزيل المستمع، يهدم بـ `unawaited`، ويصفّر المرجع.

floating_video_controller.dart ١٢٦ سطرًا

lib/core/services/videoplayerservice/floating_video_controller.dart

الدور: المتحكّم الذي تملكه الشاشة: يفتح/يصغّر/يوسّع/يغلق، ويحمل هندسة المربّع العائم. يمدّد `ChangeNotifier` فقط — **لا** `flutter_bloc` (حفاظًا على قابلية النقل).

- الحالة: `_playback` , `_isExpanded` , `_pipOffset` , `_pipSize` (افتراضها `150×86`)، `_playerKey` .
- `open(item)` — يخرج فورًا للمصدر الفارغ، يهدم التشغيل السابق، يبني تشغيلًا جديدًا، **ينشئ** `GlobalKey` **جديدًا** (هو مفتاح إعادة التوطين بين الأوضاع)، يُشعر المستمعين فورًا (فتظهر حالة التحميل)، يفعّل PiP بلا انتظار، ثم ينتظر التهيئة.
- `_enablePictureInPicture()` — ثلاث بوابات: الخيار مفعّل؟ الجهاز يدعم (`DeviceInfoService.supportsPictureInPicture`)؟ النظام متاح الآن (`isPipAvailable`)؟ والرفض يُسجَّل كتحذير فقط.
- `retry()` — إعادة فتح نفس العنصر.
- `expand()` / `minimize()` — محميتان بشرط الحالة فلا إشعار بلا تغيير.
- `closeVideo()` — يهدم ويُصفّر **كل** الهندسة للحالة الافتراضية (فلا يُفتح الفيديو التالي في موضع غريب)، ويُلغي PiP.
- `updatePipOffset` / `updatePipSize` .
- `dispose()` — يهدم التشغيل ويُلغي PiP.

floating_video_overlay.dart ٣٤ سطرًا

lib/core/services/videoplayerservice/floating_video_overlay.dart

الدور: الودجة الوحيدة التي تضعها في `Stack` فوق محتوى التطبيق.

- `AnimatedBuilder(animation: controller)` — تستمع للمتحكّم.
- لا فيديو ⟵ `SizedBox.shrink()` (صفر تكلفة).
- وإلا `Positioned.fill` يحوي `ExpandedVideoPlayer` أو `VideoPipPlayer` بحسب `isExpanded` .

video_player_options.dart ٥٦ سطرًا

lib/core/services/videoplayerservice/video_player_options.dart

الدور: كل ما يُخصَّص في المشغّل، مجموعًا في كلاس `const` واحد — وهو ما يحفظ استقلال الوحدة عن ثيم التطبيق.

- الألوان: `backgroundColor` , `accentColor` , `foregroundColor` , `gradientColors` .
- النصوص: `speedSheetTitle` , `descriptionLabel` , `liveLabel` , `retryLabel` — **تُمرَّر مترجمة من الخارج** فلا تحتاج الوحدة حزمة ترجمة.
- `speedOptions` — قائمة `MapEntry<String, double>` (عنوان ⟵ سرعة).
- `enablePictureInPicture` .
- `onShare` — خطّاف مشاركة مخصّص.
- `detailsBuilder` — **نقطة توسّع قوية**: يستبدل كتلة التفاصيل أسفل الفيديو بأي ودجة من التطبيق المضيف.
- `resolveGradient()` — تُرجع التدرّج المُمرَّر، أو تبني واحدًا تلقائيًا من لون الخلفية ممزوجًا بالأسود ٣٥٪ عبر `Color.lerp` .

video_share.dart ٢٥ سطرًا

lib/core/services/videoplayerservice/video_share.dart

- `isAvailable(item, options)` — يوجد `onShare` أو `shareText` غير فارغ. **تستخدمها الواجهة لإخفاء زر المشاركة** بدل إظهار زر لا يفعل شيئًا.
- `send(item, options)` — يفضّل `onShare` المخصّص، وإلا يستخدم `SharePlus` بنص المشاركة.

١١,٧ — `videoplayerservice/widgets/` — ودجات المشغّل (١٣ ملفًا)

expanded_video_player.dart ١٠٤ أسطر

…/widgets/expanded_video_player.dart

الدور: العرض الموسّع (ملء الشاشة).

- يبني `playback.buildSurface` مع **مفتاح المتحكّم المشترك** وطبقة فيها: `VideoControlsOverlay`، وشريط أعلى فيه زر التصغير (سهم لأسفل) وزر المشاركة (إن كان متاحًا).
- **تخطيطان بحسب الاتجاه:**
  - **عرضي:** الفيديو يملأ الشاشة بلا أي شيء آخر.
  - **طولي:** خلفية متدرّجة، ثم `AspectRatio(16/9)` للفيديو، ثم `Expanded` فيه `SingleChildScrollView` للتفاصيل — إما `options.detailsBuilder` أو `VideoDetails` الافتراضية.
- `SafeArea(bottom: false)` — حماية علوية فقط.

video_controls_overlay.dart ٢٨٥ سطرًا

…/widgets/video_controls_overlay.dart

الدور: أكبر ودجة في الخدمة: كل الإيماءات والضوابط. `StatefulWidget` لأنها تملك حالة عرض مؤقتة.

الحالة والمؤقّتات

- `_controlsVisible` — تبدأ `true` ثم تختفي الضوابط بعد **٣ ثوانٍ** عبر `_hideTimer`.
- `_isFastForward` — وضع «٢× أثناء الضغط المطوّل».
- `_isLandscape` — حالة التدوير اليدوي.
- `_seekDirection` — -١ / ٠ / ١ لإظهار أيقونة التنقّل، وتُصفَّر بعد `600ms` عبر `_seekFeedbackTimer`.
- `dispose` — يُلغي المؤقّتين **ويُعيد الاتجاه للوضع الطولي**، فلا يبقى التطبيق مقلوبًا بعد إغلاق الفيديو.

الإيماءات (منطقة الفيديو مقسومة نصفين)

- **النصف الأيسر:** نقرة = إظهار/إخفاء الضوابط · نقرة مزدوجة = -١٠ ثوانٍ · سحب لأسفل = تصغير.
- **النصف الأيمن:** نقرة = إظهار/إخفاء · نقرة مزدوجة = +١٠ ثوانٍ · **ضغط مطوّل = ٢× ما دام الإصبع** · سحب لأسفل = تصغير.
- `textDirection: TextDirection.ltr` مفروض صراحة على الصف — **قرار صحيح**: «اليسار يرجع واليمين يقدّم» اصطلاح عالمي في مشغّلات الفيديو ولا يجوز أن ينقلب مع العربية.
- `behavior: HitTestBehavior.opaque` — فتُستقبل النقرات على المناطق الشفافة أيضًا.

الطبقات

- **الخطأ أولًا:** إن وُجدت رسالة خطأ تُعرض `VideoPlaybackError` وحدها (بزرّي إعادة وإغلاق) ولا شيء غيرها.
- **التحميل:** `!isReady || isBuffering` ⟵ دوّارة بلون التمييز، وتُستبدل بزر التشغيل عند الجهوزية.
- **أعلى اليمين (عند الظهور):** زر التدوير (يتغيّر رمزه بين ملء الشاشة والخروج منه) وزر منتقي السرعة.
- **الأسفل:** شارة **LIVE** للبث المباشر، أو `VideoProgressBar` لغيره — **تبديل كامل لا تعطيل**، وهو السلوك الصحيح لأن شريط تقدّم البث بلا معنى.
- كل تفاعل ينادي `_scheduleHideControls()` فيُعاد ضبط عدّاد الإخفاء — فلا تختفي الضوابط وسط استخدامها.
- `_onSwipeDownToMinimize` — يشترط سرعة سحب `> 200` فلا يُصغَّر الفيديو بلمسة عابرة.
- `_setFastForward` — محمية بـ `_isFastForward == enabled` وبـ `canSeek` (فلا تتغيّر سرعة بث مباشر).

video_pip_player.dart ١٥٤ سطرًا

…/widgets/video_pip_player.dart

الدور: المربّع العائم القابل للسحب وتغيير الحجم.

- ثوابت: `_margin = 12` , `_minWidth = 110` , `_aspect = 150/86` .
- `LayoutBuilder` يحسب `maxX` و`maxY` ثم **يقصّ** الموضع داخلهما (`clamp`) — فيبقى المربّع داخل الشاشة دائمًا، **ويُصحّح موضعه تلقائيًا عند تدوير الجهاز**.
- الموضع الافتراضي `Offset(_margin, maxY)` — أسفل البداية.
- `onScaleUpdate` — **إيماءة واحدة لعملين**: إن كان عدد الأصابع ≥ ٢ تُغيَّر الحجم (مع الحفاظ على النسبة وقصّ العرض بين الحد الأدنى وعرض الشاشة)؛ وأي إزاحة لنقطة التمركز تُحرّك المربّع. و`_gestureStartSize` يُحفظ في `onScaleStart` فيكون التحجيم نسبيًا لا تراكميًا.
- `_onTapUp(details, size)` — **منطق ذكي حسب موضع النقر**: الضوابط مخفية ← النقرة تُظهرها فقط؛ النقر داخل دائرة نصف قطرها `22` من المركز ← تشغيل/إيقاف؛ أي مكان آخر ← توسيع.
- التصميم: زوايا `14` ، ظل ناعم، `clipBehavior: Clip.antiAlias` .
- يستخدم **نفس** `controller.playerKey` — فالانتقال من العائم للموسّع لا يُعيد التشغيل.

video_progress_bar.dart ١١٤ سطرًا

…/widgets/video_progress_bar.dart

الدور: شريط تقدّم مرسوم يدويًا (لا `Slider`) للتحكّم الكامل في الشكل والإيماءة.

- `_dragFraction` — **حل مشكلة حقيقية**: أثناء السحب يُعرض موضع **إصبع المستخدم** لا موضع المشغّل. فلا يرتدّ المقبض للخلف بسبب تأخّر المحرّك في الاستجابة. وعند `onHorizontalDragEnd` يُصفَّر فيعود الشريط لمتابعة المشغّل.
- يستجيب لـ `onTapDown` أيضًا — فالنقر على موضع ينتقل إليه مباشرة.
- `Directionality(TextDirection.ltr)` مفروض — **صحيح**: شريط الزمن يتقدّم من اليسار لليمين في كل اللغات.
- التركيب: مسار رمادي بعرض كامل، شريط ملوّن بعرض `width * fraction` ، ومقبض دائري `14` بكسل موضعه مقصوص بين `0` و`width - 14` فلا يخرج عن الشريط في الطرفين.
- ارتفاع منطقة اللمس `24` بينما الشريط `3` — **منطقة لمس أكبر بكثير من الشكل المرئي**، وهي قاعدة أساسية في إمكانية الوصول.

video_details.dart ٩١ سطرًا

…/widgets/video_details.dart

الدور: كتلة التفاصيل الافتراضية أسفل الفيديو: العنوان (٢٦px عريض)، العنوان الفرعي (شفافية ٧٠٪)، التاريخ مع أيقونة تقويم (٥٠٪)، ثم عنوان «الوصف» والنص (٨٠٪ بتباعد أسطر `1.7`).

- كل قسم مشروط بـ `isNotEmpty` بعد `trim()` — فلا فراغات ولا عناوين بلا محتوى.
- التمييز بالشفافية التدرّجية (١٠٠٪ / ٧٠٪ / ٥٠٪ / ٨٠٪) بدل ألوان متعدّدة — يبقي الوحدة محايدة تجاه أي ثيم.

video_speed_sheet.dart ٨٨ سطرًا

…/widgets/video_speed_sheet.dart

الدور: ورقة سفلية لاختيار سرعة التشغيل.

- `static show(...)` — دالة ثابتة تُظهر `showModalBottomSheet`، **وتُغلق الورقة بنفسها** بعد الاختيار (`pop` ثم `onSpeedSelected`) — فالمنادي لا يتعامل مع الإغلاق.
- تبني الخيارات بحلقة على `options.speedOptions`، والسرعة الحالية تُبرَز بلون التمييز وخط عريض.
- `SafeArea` + `SingleChildScrollView` + `Flexible` — فتعمل مع قوائم سرعات طويلة وعلى الشاشات القصيرة.

video_playback_error.dart ٧٢ سطرًا

…/widgets/video_playback_error.dart

الدور: حالة الخطأ: خلفية سوداء ٧٢٪، أيقونة تحذير، الرسالة (`maxLines: 3` + `ellipsis`)، ثم زر «إعادة المحاولة» وزر إغلاق دائري.

> وجود **إجراءين** (إعادة + إغلاق) هو الصواب: المستخدم لا يُترك في طريق مسدود، ولا يُفرض عليه الانتظار.

video_pip_controls.dart ٨٢ سطرًا

…/widgets/video_pip_controls.dart

الدور: ضوابط المربّع العائم: زر تشغيل/إيقاف في المركز وزر ✕ أعلى اليمين، كلاهما بـ `AnimatedOpacity` (٢٠٠ms).

- زر الإغلاق ملفوف بـ `IgnorePointer(ignoring: !isVisible)` — **تفصيل مهم**: الزر الشفاف لا يستقبل نقرات، فلا يُغلق المستخدم الفيديو بالخطأ وهو يحاول إظهار الضوابط.
- زر التشغيل داخل `AnimatedBuilder` منفصل يستمع لـ `playback` — فتغيّر حالة التشغيل يُعيد بناء الأيقونة وحدها.

video_circle_icon_button.dart ٢٧ سطرًا

…/widgets/video_circle_icon_button.dart

زر أيقونة دائري بخلفية سوداء ٤٥٪ — مستخدم في التصغير والمشاركة والتدوير والسرعة والإغلاق. مكوّن واحد يوحّد مظهر كل أزرار المشغّل.

video_seek_icon.dart ١٩ سطرًا

…/widgets/video_seek_icon.dart

أيقونة تغذية راجعة للتنقّل (±١٠ ثوانٍ): دائرة سوداء ٤٥٪ بأيقونة ٢٨px. تظهر وتختفي بـ `AnimatedOpacity` من الودجة الأم.

video_live_badge.dart ٤٣ سطرًا

…/widgets/video_live_badge.dart

شارة البث المباشر: خلفية حمراء ٩٠٪، نقطة بيضاء دائرية ٧px، ثم النص (المُمرَّر مترجمًا من `options.liveLabel`). تحلّ مكان شريط التقدّم في البث المباشر.

video_speed_badge.dart ٣٦ سطرًا

…/widgets/video_speed_badge.dart

شارة «٢×» تظهر في منتصف اليمين أثناء الضغط المطوّل: كبسولة سوداء ٥٥٪ فيها أيقونة تقديم بلون التمييز ونص `2x`. تغذية راجعة فورية تُفهم بلا تعليم.

١٢ — مجلد `themes/` — الثيمات والتنسيق (٧ ملفات)

نظام تصميم كامل: لوحة ألوان مسمّاة، ثيمان (فاتح/داكن)، ٤٠+ نمط نص مسمّى بالدور لا بالحجم، رموز للمسافات وأنصاف الأقطار والحركة، وانتقال موحّد بين الصفحات. القاعدة: **لا قيمة لونية أو نمط نص مكتوب مباشرة في أي ودجة.**

app_colors.dart ١٨ سطرًا

lib/core/themes/app_colors.dart

الدور: اللوحة الخام — ١٢ لونًا مسمّى بالاسم الوصفي لا بالدور، تُستهلك من `AppTheme` فقط.

اللون	القيمة	الدلالة
`cream`	#EEE2CB	الخلفية الأساسية في الوضع الفاتح
`creamCard`	#EFE8DC	خلفية البطاقات
`darkGreen`	#324532	الأخضر الداكن (اللون الثانوي)
`green`	#405940	الأخضر الأساسي
`actionGreen`	#6B946B	أخضر الإجراءات
`ringTrack`	#A6BFA6	مسار حلقات التقدّم
`gold`	#C0A572	الذهبي (لون التمييز)
`sandSoft` / `sand`	#D0BB95 / #DFD2B9	درجتا الرملي
`tileGreen`	#567656	أخضر المربّعات
`ink`	#000000	لون النص
`ornamentOpacity` + `ornament`	#B38339 / 0.3	لون الزخرفة وشفافيتها

اللوحة كلها من عائلة «الكريمي + الأخضر + الذهبي» — هوية بصرية إسلامية/تعليمية متماسكة. وتعريف `ornamentOpacity` كرقم مع الألوان ذكاء صغير: يضمن أن كل الزخارف في التطبيق بنفس الشفافية.

app_theme.dart ٢٦٦ سطرًا

lib/core/themes/app_theme.dart

الدور: أهم ملف تنسيق في المشروع. يضم كلاسين: `AppTheme` (الثيمان ومخططا الألوان) و `AppTextStyles` (كل أنماط النص).

`lightColorScheme`

مخطّط `ColorScheme.light` كامل يربط اللوحة الخام بالأدوار الدلالية: `primary` ← الأخضر، `secondary` ← الأخضر الداكن، `tertiary` ← الذهبي، `surface` ← الكريمي، `surfaceContainerHighest` ← كريمي البطاقات، `surfaceContainerHigh` ← مسار الحلقات، `outline` ← الأخضر. **تعبئة المخطّط كاملًا** (لا الاكتفاء بلونين و `fromSeed`) هو ما يمنع ألوانًا مولّدة لا تناسب الهوية.

`darkColorScheme`

ليس انعكاسًا آليًا للفاتح، بل لوحة مستقلّة: الأساسي يتحوّل إلى **طيني محمر** (#A54735)، والسطح #171717، والثانوي الحاوي أخضر فاتح (#9DBF9D) للتباين على الداكن. القيم مكتوبة هنا مباشرة لا في `AppColors`.

`light()` و `dark()`

- كلتاهما تنادي `AppScreenUtil.ensureConfigured()` أولًا — ضروري لأن أنماط النص تستخدم `.sp`، وبلا تهيئة ترمي `ScreenUtil`.
- `useMaterial3: false` — **قرار مقصود**: التصميم مبني على مفردات Material 2، والانتقال إلى 3 كان سيغيّر أشكال الحقول والأزرار والحوارات كلها.
- تضبطان: `scaffoldBackgroundColor`، `iconTheme`، `appBarTheme` (بـ `elevation: 0` — تصميم مسطّح)، `colorScheme`، `pageTransitionsTheme`، `textTheme`.
- الداكن يضيف: `dividerColor`، `cardColor`، `progressIndicatorTheme`، `textButtonTheme` — تصحيحات لازمة لأن الافتراضات الداكنة في Flutter لا تناسب هذه اللوحة.

`AppTheme.styles(context)` و `AppTextStyles`

الوصول للأنماط عبر `AppTheme.styles(context).cardTitle`. الكلاس `@immutable` يحمل `ColorScheme`، وكل الأنماط تُبنى بدالة خاصة `_style(...)` تستخدم **خط Cairo** من Google Fonts.

الأنماط مسمّاة بدورها لا بحجمها — وهذه هي القيمة الأساسية للملف. أمثلة من أكثر من ٤٠ نمطًا:

- **العناوين:** `statusTitle` (26sp)، `authTitle` (24)، `screenTitle` (20)، `sectionTitle` (18)، `panelTitle` (17)، `bannerTitle`، `groupTitle`.
- `splashTitle` — **الاستثناء الوحيد**: يستخدم خط **Aref Ruqaa** (خط رقعة عربي) بدل Cairo، فيُعطي شاشة البداية طابعًا خطيًّا عربيًا أصيلًا.
- **البطاقات:** `cardTitle`، `cardSubtitle`، `cardBlurb` (بتباعد 1.6)، `cardCaption`.
- **الصفوف:** `rowTitle`، `rowValue`.
- **الخيارات:** `optionLabel` / `optionLabelSelected` — زوج يختلف بالوزن فقط (500 ↔ 800) فيُبرز المختار بلا تغيير حجم ولا ارتجاف في التخطيط.
- **التسميات:** `labelStrong`، `labelMedium`، `labelSmall`، `pillLabel`، `badgeLabel`، `progressLabel`، `caption`.
- **الإحصاءات:** `statValue`، `statLabel`.
- **الحقول:** `fieldInput`، `fieldHint`، `searchHint`، `fieldError` (بلون `error`).

- **الجسم:** `bodyLarge` , `bodyMedium` , `bodySmall` , `note` (16sp بتباعد 1.5).
- `brandTagline` — بتباعد أسطر 1.7، مناسب للنص العربي الطويل.
- `textTheme` — يربط الأنماط المخصّصة بأسماء Flutter القياسية (`headlineLarge` ← `authTitle` ...) فتستفيد ودجات Material الجاهزة من نفس الهوية.

> **لماذا التسمية بالدور أفضل؟** لأن «غيّر حجم عناوين البطاقات» تصبح تعديلًا في سطر واحد هنا، بدل بحث عن كل `fontSize: 15` في المشروع وتمييز أيها عنوان بطاقة. وملاحظة: التباعد المحدّد (1.5–1.7) في الأنماط الطويلة مهم للعربية تحديدًا لأن حروفها ذات امتدادات رأسية أكثر.

app_page_transitions.dart

`lib/core/themes/app_page_transitions.dart`

الدور: انتقال المسارات الموحّد للتطبيق: تلاشٍ عابر (fade-through) مع رفعة تحجيم صغيرة جدًا (من `0.98` إلى `1`).

- `AppFadeThroughTransitions extends PageTransitionsBuilder` بمدة `AppMotion.normal` .
- **محايد تجاه الاتجاه عن قصد** (موثّق): لا شيء ينزلق، فتُقرأ نفس الحركة صحيحة في العربية (RTL) والإنجليزية (LTR) بلا أي انعكاس. حلٌّ أنيق لمشكلة انتقالات الانزلاق في التطبيقات ثنائية الاتجاه.
- `if (MediaQuery.disableAnimationsOf(context)) return child;` — **احترام تفضيل تقليل الحركة** في إمكانية الوصول: الصفحة تظهر بلا أي حركة.
- `CurvedAnimation` بمنحنى `AppMotion.enter` ومنحنى عكسي `AppMotion.exit.flipped` .
- `appPageTransitionsTheme` — ثابت `const` يطبّق نفس الباني على **الست منصّات**، فلا يختلف الإحساس بين أندرويد و iOS.

app_motion.dart ٢١ سطرًا

`lib/core/themes/app_motion.dart`

الدور: رموز الحركة. التعليق في أعلى الملف يوضّح فلسفته: القيم **مطابقة للأسلوب المستخدم فعلًا** في `app_toast.dart` (`easeOutCubic` / 220ms) و `splash_timeline.dart` — **فتبنّي الرموز ليس تغييرًا في الإحساس أبدًا**.

- `fast = 150ms` , `normal = 220ms` , `slow = 320ms` .
- `stagger = 40ms` — الفاصل بين عنصرين متتاليين في دخول متدرّج.
- `shimmer = 1100ms` — دورة كنس واحدة للهيكل العظمي.
- `enter = Curves.easeOutCubic` , `exit = Curves.easeIn` .

> نهج ممتاز في إدخال نظام تصميم على مشروع قائم: **سمّ ما هو مستخدم فعلًا** أولًا، فيصبح التبنّي تغيير اسم بلا مخاطرة بصرية، ثم وحّد بعد ذلك.

app_spacing.dart ٢٧ سطرًا

lib/core/themes/app_spacing.dart

الدور: كلاسان برمزي المسافات وأنصاف الأقطار، بنفس فلسفة `AppMotion` (موثّقة: «تسمية المقياس المستخدم بحكم الواقع، فالتبنّي إعادة تسمية خالصة لا إزاحة بصرية»).

- `AppSpacing`: `xs=4`, `sm=8`, `md=12`, `lg=16`, `xl=20`, `xxl=24`.
- `AppRadii`: `sm=6`, `md=8`, `lg=10`, `xl=12`, `xxl=14`, `pill=20`.

> التعليق ينصّ على أن `.w` / `.h` / `.r` تُطبَّق **في موضع الاستدعاء** كما في كل المشروع — أي `AppSpacing.lg.w` — فتبقى القيم أرقامًا مجرّدة قابلة للتوسّع مع حجم الشاشة.

notification_tone_palette.dart ٩٧ سطرًا

lib/core/themes/notification_tone_palette.dart

الدور: لوحة كاملة لكل «نبرة» إشعار. كلاس `@immutable` بسبعة ألوان: `background`, `border`, `accent`, `onAccent`, `iconInk`, `title`, `body`.

- `NotificationTonePalette.of(context, tone)` — تختار الخريطة المناسبة بحسب `Theme.of(context).brightness`.
- خريطتان كاملتان (`_light` و`_dark`) لثلاث نبرات = **٤٢ لونًا معرّفًا يدويًا**:
  - **إداري** ← عائلة ذهبية/ترابية.
  - **تنبيه** ← عائلة برتقالية/حمراء.
  - **درس** ← عائلة خضراء.
- `readBackground(colors)` — **الدالة الأذكى**: تمزج لون الخلفية مع سطح الثيم بنسبة ٤٥٪ عبر `Color.lerp`، فتُنتج نسخة **باهتة** للإشعار المقروء. فالفرق بين «مقروء» و«غير مقروء» شدّة لون لا شكل مختلف — ويُولّد تلقائيًا بلا ٤٢ لونًا إضافيًا.

> تعريف الألوان بشكل كامل (لا مشتقّات محسوبة) يضمن تباينًا مقروءًا في كل تركيبة — وهو ما لا تضمنه الاشتقاقات الآلية.

notification_kind_accents.dart ٣٤ سطرًا

lib/core/themes/notification_kind_accents.dart

الدور: لون تمييز واحد **لكل نوع** إشعار (أدقّ من النبرة): خريطتان لثمانية أنواع × وضعين = ١٦ لونًا.

- `of(context, kind)` — بنفس منطق اختيار الخريطة.
- `chipTint = 0.16` — نسبة تلوين خلفية الشارة من لون التمييز: فتُحسب خلفية الشارة من اللون نفسه بلا تعريف لون ثانٍ.
- ألوان الوضع الداكن ليست نفسها مُفتَّحة آليًا بل مختارة يدويًا (مثلًا «محتوى جديد»: #1F6B35 في الفاتح و #7FC68F في الداكن) — لأن اللون الداكن على خلفية داكنة يحتاج سطوعًا وتشبّعًا مختلفين تمامًا ليبقى مقروءًا.

> **طبقتان من الألوان للإشعارات:** `NotificationTonePalette` تُلوّن **البطاقة** (ثلاث نبرات، تجميع عام)، و `NotificationKindAccents` تُلوّن **الأيقونة والشارة** (ثمانية أنواع، تمييز دقيق). هكذا تبدو بطاقات الدروس عائلة واحدة، ومع ذلك يفرّق المستخدم بين «محتوى جديد» و«تذكير بث» بنظرة.

إضافة إلى مجلد `drawer/` : drawer_profile_skeleton.dart

lib/core/drawer/presentation/widgets/drawer_profile_skeleton.dart

الدور: هيكل عظمي لصف هوية المستخدم في القائمة الجانبية قبل وصول بياناته. التعليق في الملف يذكر المشكلة التي يحلّها صراحة: «حتى الآن كان هذا خانة نص مسطّرة فارغة في كل فتح للقائمة».

- `AppShimmer` يحوي صفًّا: `SkeletonBox(52×52, isCircular: true)` لموضع الصورة، ثم `SkeletonTextLine(fontSize: 15.sp, widthFactor: 0.7)` لموضع الاسم.
- سطح الهيكل مأخوذ من `colorScheme.tertiaryContainer` — **لا لون رمادي ثابت** — فيبدو الهيكل جزءًا من البطاقة الملوّنة لا رقعة غريبة.
- الأبعاد مطابقة تمامًا لأبعاد المحتوى الحقيقي (`52.w` في `DrawerProfileMonogram` و`15.sp` في نمط `cardTitle`) — فلا إزاحة في التخطيط لحظة وصول البيانات.

كما تغيّر `app_drawer.dart` ليحسب سقف العرض من حجم الشاشة: `ScreenSize.isTablet(context) ? 400.w : 320.w` بدل الرقم الخام `320`. والتعليق يشرح السبب: الرقم الخام بجوار قياسات `.w` داخل القائمة كان يجعل اللوحة ومحتواها يتوسّعان بمعدّلين مختلفين على الشاشات الصغيرة والكبيرة.

١٣ — مجلد `utils/` — المساعدات (١٨ ملفًا)

أدوات مساعدة صغيرة ومستقلّة، معظمها كلاسات ثابتة. أهمّها `error_mapper.dart` (قلب نظام الأخطاء) و `cache_util.dart` (كل تخزين التفضيلات) و `lesson_view_data.dart` (جسر بين التعدادات والواجهة).

app_language.dart ١٣ سطرًا

lib/core/utils/app_language.dart

الدور: حامل لغة الواجهة الحالية بلا الحاجة إلى `BuildContext`.

- `fallbackCode = 'ar'` — العربية هي الاحتياط واللغة الابتدائية.
- `code` (قراءة) و`update(code)` التي **تتجاهل النص الفارغ** فلا تُفسد اللغة بقيمة فاسدة.

لماذا هذا مهم؟ لأن `Json.asOptionalLocalizedString` في طبقة الشبكة و`DeviceInterceptor` (ترويسة `Accept-Language`) يحتاجان اللغة، وكلاهما بعيد تمامًا عن شجرة الودجات. هذا الملف هو الجسر.

app_logger.dart ٣٤ سطرًا

lib/core/utils/app_logger.dart

الدور: غلاف موحّد حول `dart:developer.log` بثلاث مستويات: `info`, `warning` (تضيف بادئة `WARNING`), `error` (بمستوى `1000` + تتبّع المكدس).

- معامل `name` افتراضه `bilhikma`، ويُمرَّر في كل الخدمات بأسماء نطاقات (`auth`, `push`, `device_session`, `wakelock`, ...) فتتصفّى السجلات بالنطاق في DevTools.

استخدام `developer.log` لا `print` متعمّد: الأول منظَّم (اسم، مستوى، خطأ، مكدس) ويُحجَب في الإصدار، والثاني يطبع كل شيء في كل بناء.

app_regex.dart ٣٣ سطرًا

lib/core/utils/app_regex.dart

الدور: كل التعابير النمطية والتحقّقات في مكان واحد.

- `passwordMinLength = 6` .
- تعابير: `email` , `passwordLetter` , `passwordNumber` , `passwordUpper` , `passwordLower` , `passwordSymbol` (أي محرف ليس حرفًا ولا رقمًا)، `phone` (بادئة `+` أو `00` اختيارية ثم ٧–١٥ رقمًا)، و `phoneAllowedChars` (للاستخدام في `inputFormatters`).
- دوال مساعدة مقروءة: `isValidPhone` (تُقلّم أولًا)، `isValidEmail` ، و `passwordHas*` الستّ، و `passwordHasMixedCase` التي تجمع شرطين.

> تناقض يستحق التوحيد: `passwordMinLength = 6` هنا، لكن `PasswordStrength.of` تمنح نقطة للطول `>= 8` . فالمستخدم قد يمرّ بالتحقّق بكلمة طولها ٦ وتُصنَّف «ضعيفة» — سلوك مقبول لكنّ الأفضل توحيد العتبتين أو توثيق الفرق.

app_screen_util.dart ٢٢ سطرًا

lib/core/utils/app_screen_util.dart

الدور: تهيئة `flutter_screenutil` بأمان.

- `designSize = Size(412, 917)` — مقاس التصميم المرجعي (هاتف أندرويد متوسّط).
- `minTextAdapt: true` و `splitScreenMode: true` .
- `ensureConfigured()` — **حيلة عملية**: تحاول قراءة `ScreenUtil().screenWidth` ؛ فإن رمت `Error` (أي لم تُهيّأ بعد) تهيّئها بمقاس التصميم. تُنادى من `AppTheme.light()/dark()` لأن أنماط النص تستخدم `.sp` وتُبنى أحيانًا قبل أن يغلّف `ScreenUtilInit` الشجرة.

screen_size.dart ١٢ سطرًا

lib/core/utils/screen_size.dart

- `size(context)` — يستخدم `MediaQuery.sizeOf` (النسخة الأحدث التي تُقلّل إعادة البناء مقارنة بـ `MediaQuery.of`).
- `isTablet(context)` — `shortestSide >= 600` ، وهو **المعيار الصحيح**: استخدام `shortestSide` لا العرض يعني أن التصنيف لا يتغيّر عند تدوير الجهاز.
- `isMobile(context)` — نقيضها.

byte_format.dart ٥٠ سطرًا

lib/core/utils/byte_format.dart

الدور: تنسيق أحجام الملفات. الوحدات: `B` , `KB` , `MB` , `GB` .

- `readable(bytes)` — **تُرجع نصًا فارغًا للصفر أو السالب** (لا «0 B») فتختفي من الواجهة حين لا معنى لها. تقسّم على ١٠٢٤ حتى الوحدة المناسبة، وتُظهر منزلة عشرية واحدة إلا للبايت (لا معنى لـ «512.0 B»).
- `readableValue(bytes, {referenceBytes})` — **الدالة الذكية**: تُرجع **الرقم فقط بلا وحدة**، محسوبًا بوحدة `referenceBytes` . تُستخدم في `DownloadProgress.readableRatio` لإنتاج «٤٫٢ / ٣٫١ م.ب» بدل «٤٫٢ م.ب / ٢١٥٠ ك.ب» — توحيد الوحدة بين طرفي النسبة.
- `unitOf(bytes)` و `_unitOf` و `_factor(unit)` — مساعدات لحساب الوحدة ومعامل التحويل.

cache_util.dart ٦٤ سطرًا

lib/core/utils/cache_util.dart

الدور: غلاف `SharedPreferences` — كل تخزين التفضيلات في التطبيق يمرّ من هنا.

- `_prefs` قابل للفراغ و `isReady`.
- `init()` — حمائية (`if (_prefs != null) return`)، وعند الفشل **تُسجّل خطأً وتكمل** برسالة صريحة: «التفضيلات غير متاحة — سيعمل التطبيق بلا حالة مخزّنة».
- الكاتبات: `setBool`, `setString`, `setDouble`, `setInt`, `setStringList` — كلها تستخدم `_prefs?.` (الوصول الآمن).
- `get({key})` تُرجع `dynamic`، و `remove({key})`.

> **النتيجة المعمارية:** فشل `SharedPreferences` (وهو نادر لكنه يحدث) **لا يُسقط التطبيق** — كل الكتابات تصبح عمليات فارغة وكل القراءات `null`، فيعمل التطبيق بسلوكه الافتراضي. و`get` ترجع `dynamic` ولذلك يفحص كل منادٍ النوع (`token is String`) كما لوحظ في الخدمات.

current_user.dart ٧٥ سطرًا

lib/core/utils/current_user.dart

الدور: قراءة وتعديل المستخدم المخزّن في الكاش (كـ JSON) بلا الحاجة لشبكة ولا لـ Cubit.

- `cachedUser()` — تفكّ وتُرجع `AuthUserModel?`.
- `canCaptureScreen` — اختصار مباشر بافتراض `false` (الأكثر أمانًا). تقرأها `ScreenCapturePolicy`.
- `saveFields({fullName, email})` — **تحديث جزئي ذكي**: تفكّ الـ JSON، تقلّم القيم، **وتكتب فقط إن تغيّر شيء فعلًا** (`hasChanged`). موثّقة بسببها: إبقاء الكاش متوافقًا مع التعديلات في مكان آخر، فلا يُظهر أي قارئ للكاش بيانات لحظة الدخول بعد تعديلها.
- `setCaptureExemption(bool)` — نفس النمط لحقل الصلاحية، وتخرج فورًا إن كانت القيمة مطابقة.
- `clear()` — حذف المفتاح.
- `_decode()` — قراءة محمية: تفحص النوع والفراغ، وتلتقط أخطاء `jsonDecode`، وتُرجع `null` للـ JSON الفاسد.

> تعديل الـ JSON كخريطة (بدل فكّه لنموذج وتعديله وإعادة ترميزه) **يحافظ على الحقول التي لا يعرفها النموذج** — فلو أضاف الخادم حقلًا جديدًا لا يُحذف من الكاش عند تعديل الاسم.

error_mapper.dart ١٥٤ سطرًا

lib/core/utils/error_mapper.dart

الدور: قلب نظام الأخطاء في التطبيق. يحوّل أي خطأ إلى `AppException` بمفتاح ترجمة مستقرّ، **ويحتكر قرار «هل انتهت الجلسة؟»**. يضم كلاسين.

`AppException`

- `key` — مفتاح الترجمة (هو ما تعرضه الواجهة).
- `message` — رسالة الخادم الخام (للسجل أو للعرض عند الحاجة).
- `errors` — أخطاء الحقول، مع `errorsFor(field)`.
- `statusCode` , `data`.
- `endsSession` — موثّق: هل يعني الرمز أن الجلسة نفسها انتهت فالتوكن على الجهاز لا قيمة له. **يُقرَّر مرة واحدة هنا ويُحمل من هنا.**
- `toString()` مفيد للسجلات.

`ErrorMapper`

- `sessionEndedCodes = {401, 403}` — ومعه **أهم تعليق في المشروع**:

> حالة الحساب نفسها (`active` , `suspended`، وأي اسم تخترعه لوحة التحكّم لاحقًا) **لا تُستشار أبدًا عن قصد**. فالحالات قائمة إدارية تنمو بلا إعادة بناء التطبيق، وحالة لم يسمع بها هذا البناء كانت ستُقرأ كـ «كل شيء بخير، تابع». أما **رمز** الرفض فلا ينمو: إلى أي حالة نُقل الحساب، يُرفض الطلب بأحد هذين الرمزين، وهذه هي كل الإشارة التي يحتاجها التطبيق.

- `sessionEndedKey = 'session_expired'` — **إشعار واحد لكل الرموز**، فحالة جديدة في لوحة التحكّم لا تحتاج نصًا جديدًا هنا.
- `endsSession(statusCode)` و `sessionIsOver(error)`.
- `map(error)` — تُرجع المفتاح النصي فقط.
- `toException(error)` — الدالة المركزية: `AppException` تُرجع كما هي؛ `DioException` تُفكَّك (جسم ← مغلّف ← رمز) وتُبنى منها استثناء كامل؛ وأي شيء آخر يصبح `something_went_wrong`.
- `fromEnvelope(envelope, {statusCode})` — لبناء الاستثناء من مغلّف فاشل.
- `_failingCode(transportCode, envelopeCode)` — **دالة دقيقة جدًا**، وموثّقة: المغلّف الذي يصل هنا **قد فشل أصلًا**، فرمز نقل في نطاق 2xx لا يقول شيئًا عن السبب — الرمز الذي وضعه الـ API في الجسم هو القائل، وهو الذي يُقرَّر عليه مصير الجلسة. المنطق: رمز نقل ≤ 300 يفوز؛ وإلا إن كان رمز المغلّف ليس رمز النجاح يفوز رمز المغلّف؛ وإلا رمز النقل.
- `_mapDio(error)` — تعارض الجهاز أولًا ← `device_already_linked`؛ ثم كل أنواع المهلات وأخطاء الاتصال ← `no_internet_connection`؛ ثم `badResponse` ← تعيين بحسب الرمز.
- `_mapStatusCode(statusCode)` — الرموز المنهية للجلسة ← `session_expired`؛ ثم، 400 ← `invalid_credentials`، 429 ← `validation_error`، 422 ← `not_found`، 404 ← `too_many_requests`، و≥500 ← `server_error` (بنمط علائقي محمي `_ when ...`)، وإلا `something_went_wrong`.

> **الصورة الكاملة لنظام الأخطاء:** `AppException(key, endsSession)` ← `ErrorMapper.toException` ← `DioException` ← `SessionGuard.endIfRefused` (إن كانت منهية) ← الـ Cubit يبثّ `key` ← الواجهة تُظهر `key.tr()` في `AppToast`. **لا تعيين أخطاء مكرّر في أي مكان آخر في التطبيق.**

lesson_view_data.dart ٧٧ سطرًا

lib/core/utils/lesson_view_data.dart

الدور: أربعة **امتدادات** (extensions) تضيف بيانات العرض للتعدادات. وهذا فصل معماري أنيق: التعداد يبقى نقيًا من مفردات الواجهة (`Color` ، `IconData`)، والواجهة تحصل على ما تحتاج.

- `LessonCategoryView` — `label` و`icon` لتبويبات الفئات الأربع.
- `ContentTypeView` — `label` (للعرض: «فيديو»)، `filterLabel` (للتصفية: «ملفات الفيديو») — **تمييز دقيق في الصياغة بحسب السياق**، و`icon` من `Icons`.
- `LessonStatusView` — `badgeLabel` (`String?`)، و`badgeColors(colors)` التي تُرجع `record` من Dart 3: `({Color background, Color foreground})`. المقفل يأخذ ألوان الخطأ، والبقية ألوان `primaryContainer`. إرجاع **الزوج معًا** يضمن عدم تركيب خلفية مع نص لا يتباين معه.
- `MediaSourceView` — `badgeLabel` و`badgeColors` (ثابتة للمصدرين).

> ملاحظة: `LessonStatus.available` و`inProgress` يتقاسمان العنوان `lesson_in_progress` — فالدرس المتاح وغير المبدوء يُعرض كـ «قيد التقدّم». قد يكون مقصودًا، لكنه يستحق مفتاحًا منفصلًا لو أُريد التمييز.

lesson_detail_formats.dart ٢٢ سطرًا

lib/core/utils/lesson_detail_formats.dart

- `clock(duration)` — تنسيق زمني تكيّفي: `mm:ss` لما دون الساعة و`hh:mm:ss` لما فوقها. فلا يظهر «00:05:30» لدرس من خمس دقائق.
- `position({totalSeconds, elapsedSeconds})` — «المنقضي / **المتبقّي**» (لا «المنقضي / الإجمالي»)، مع حماية `left < 0 ? 0` فلا يظهر وقت سالب لو تجاوز الموضع المدة المسجّلة.
- `_twoDigits` — `padLeft(2, '0')`.

live_formats.dart ١٧ سطرًا

lib/core/utils/live_formats.dart

- `time(moment)` — نظام ١٢ ساعة **مع ترجمة «ص/م»** عبر `'am'.tr()` / `'pm'.tr()`. والحساب `hour % 12 == 0 ? 12 : hour % 12` يعالج منتصف الليل والظهر صحيحًا.
- `date(date)` — بصيغة `dd-MM-yyyy`.

> تنسيق يدوي بدل `DateFormat` من `intl` — يضمن صيغة واحدة في كل اللغات (فلا تتغيّر صيغة التاريخ بين العربية والإنجليزية)، لكنه يفقد مزايا التعريب الآلي للأرقام وأسماء الأشهر.

download_progress_throttle.dart ٣١ سطرًا

lib/core/utils/download_progress_throttle.dart

الدور: يحل مشكلة أداء حقيقية: `Dio` ينادي `onReceiveProgress` **مئات المرات في الثانية**؛ ويبثّ حالة لكل نداء يعني مئات إعادة بناء في الثانية.

- `byteStep` — افتراضه `256KB`.
- `shouldEmit(progress)` — **استراتيجيتان بحسب معرفة الحجم:**
  • **حجم معروف** ← يبثّ فقط إذا تغيّرت **النسبة المئوية الصحيحة**، أي ١٠٠ إعادة بناء كحد أقصى للتحميل كله.
  • **حجم مجهول** ← يبثّ كل `256KB` (لأن النسبة لا تُحسب).
- `if (progress.received < _received) reset();` — **كشف إعادة البدء**: تراجع عدد البايتات يعني أن التحميل أُعيد من الصفر، فتُصفَّر الحالة.
- `reset()` — تضبط القيم على `-1` لا على `0`، فيُبثّ أول تقدّم فعلي (ولو كان ٠٪).

media_controls_visibility.dart ٥٥ سطرًا

lib/core/utils/media_controls_visibility.dart

الدور: يمدّد `ValueNotifier<bool>` لإدارة ظهور ضوابط الوسائط.

- `delay` — افتراضه ٣ ثوانٍ، و`_canHide` علم «يُسمح بالإخفاء».
- `setPlaying(isPlaying)` — **القاعدة الذهبية**: الضوابط تختفي فقط أثناء التشغيل. فعند الإيقاف تُنادى `show()` وتبقى ظاهرة بلا مؤقّت — لأن المستخدم الذي أوقف الفيديو يريد الضوابط.
- `poke()` — إظهار + إعادة تشغيل المؤقّت (تُنادى عند أي تفاعل).
- `show()` — إظهار دائم (إلغاء المؤقّت).
- `toggle()` — المخفية تُظهر بـ `poke()`، والظاهرة تُخفى فورًا مع إلغاء المؤقّت.
- `dispose()` — يُلغي المؤقّت (منع تسريب).

> كونها `ValueNotifier` فائدة أدائية مهمة: الودجة التي تستمع لها عبر `ValueListenableBuilder` تُعاد بناؤها وحدها، فلا يُعاد بناء سطح الفيديو كله كل ثلاث ثوانٍ.

phone_number_input_formatter.dart ٤٣ سطرًا

lib/core/utils/phone_number_input_formatter.dart

الدور: `TextInputFormatter` ينسّق رقم الهاتف أثناء الكتابة بحسب الدولة المختارة.

- يحمل `Country` في المنشئ (فالتنسيق يتبع الدولة).
- `formatEditUpdate` يعمل بأربع خطوات:

١. تنظيف كل ما ليس رقمًا.

٢. **قصّ الطول**: `country.maxLength` مع **سماح برقم إضافي إن بدأ الرقم بصفر** — تفصيل صحيح لأن المستخدم العربي يكتب `09xxxxxxxx` والصفر بادئة محلية لا جزء من الرقم الدولي.

٣. تنسيق بالمجموعات عبر `country.formatNational`.

٤. **حساب موضع المؤشّر** بعد التنسيق.

> **الخطوة الرابعة هي الأصعب وأكثر ما يُخطئ فيه المبرمجون:** إضافة المسافات تُزحزح المؤشّر، فلو لم يُحسب موضعه يقفز إلى آخر النص عند كل حرف — ويستحيل التعديل في وسط الرقم. الحل هنا: يُحسب **عدد الأرقام** قبل المؤشّر في النص القديم، ثم يُمشى في النص المنسّق حتى تُعدّ نفس الأرقام، فيكون ذلك هو الموضع الصحيح. منطق قصير ودقيق.

custom_bottom_sheet.dart ٥٥ سطرًا

lib/core/utils/custom_bottom_sheet.dart

الدور: دالة موحّدة لإظهار كل الأوراق السفلية في التطبيق بنفس الشكل.

- `showModalBottomSheetContainer({context, widget, backgroundColor, whenComplete})`.
- `_maxHeightFactor = 0.85` — سقف ارتفاع ٨٥٪ من الشاشة فلا تغطي الورقة كل شيء.
- `isScrollControlled: true` مع `backgroundColor: Colors.transparent` للورقة نفسها، والخلفية الحقيقية تُرسم في `Container` داخلي — **نمط ضروري** للزوايا الدائرية الصحيحة والظل العلوي.
- `padding bottom: MediaQuery.viewInsets.bottom` — **الأهم**: ترفع المحتوى فوق لوحة المفاتيح، فلا يُحجب حقل الإدخال داخل الورقة.
- زوايا `25`، وظل أعلى الورقة (`offset: (0, -5)`)، واللون الافتراضي `colorScheme.tertiary`.

orientation/orientation_lock.dart ٢٥ سطرًا

lib/core/utils/orientation/orientation_lock.dart

الدور: ودجة تغلّف أي شجرة وتضبط الاتجاه المسموح مرة واحدة.

- `_done` — علم يضمن التنفيذ مرة واحدة فقط ولو أُعيد البناء مرات.
- تنادي `setPreferredOrientationByDevice(context)` داخل `build` ثم تُرجع `child`.

> النداء داخل `build` يعمل لكنه ليس الموضع الأنظف لعمل جانبي؛ `didChangeDependencies` (حيث يتوفّر `MediaQuery` أيضًا) أو `addPostFrameCallback` أدقّ. تُستخدم هذه الودجة حول `SplashScreen` في الراوتر.

orientation/orientation_util.dart ٢٣ سطرًا

lib/core/utils/orientation/orientation_util.dart

الدور: ثلاث دوال حرّة لضبط الاتجاه: `setPreferredOrientationByDevice(context)` (تفوّض بحسب `ScreenSize.isMobile`)، `setPreferredOrientationByDeviceType({isMobile})`، و `getPreferredOrientationsByDeviceType(isMobile)`.

> **ملاحظة واضحة:** الفرعان في كل من الدالتين الأخيرتين **يُرجعان نفس النتيجة** (`portraitUp` فقط) — أي أن التفريق بين الهاتف واللوح **معطّل فعليًا**: التطبيق مقفل على الوضع الطولي على كل الأجهزة. يبدو أنه إمّا قرار متعمّد لم تُحذف بنيته، أو ميزة لوحية غير مكتملة. الحالة الحالية: الشرط والدالة `getPreferred...` (غير المستخدمة) يمكن تبسيطهما إلى سطر واحد — أو تفعيل الدعم العرضي للألواح إن كان ذلك المقصود. وتجدر الإشارة إلى أن `VideoControlsOverlay` يتجاوز هذا القفل مؤقّتًا عبر `SystemChrome` مباشرة عند تدوير الفيديو، ويُعيده للطولي في `dispose`.

١٤ — مجلد `widgets/` — الودجات المشتركة (٧٥ ملفًا)

أكبر مجلد في `core` عددًا. كل ودجة هنا **غبية**: تستقبل بيانات جاهزة ورُدود نداء، وتعرض، وتمرّر الأحداث — بلا أي اتصال بشبكة، ولا تخزين، ولا حسابات منطقية. ينقسم المجلد إلى: ودجات عامة (`app_*`)، أزرار وحقول، بطاقات الدروس والمواد والجلسات، وعائلة التحميل `loading/`.

١٤٫١ — الهيكل العام والتنقّل

app_chrome.dart ٢٤ سطرًا

lib/core/widgets/app_chrome.dart

الدور: الغلاف الذي يحيط كل شاشات التطبيق: يطبّق مقياس الخط، والحماية الأمنية، و`SafeArea`.

- `_minFactor = 0.9` و`_maxFactor = 1.3` — ومعهما **تعليق يشرح إصلاح خطأ حقيقي**: تثبيت الطرفين على `fontScale` (كما كان) **يُلغي تفضيل المستخدم في إعدادات النظام تمامًا**. الصيغة الحالية تسمح لإعداد إمكانية الوصول بتحريك النص حول مقياس التطبيق بنسبة -١٠٪ إلى +٣٠٪.
- `SecurityGuard` يحيط المحتوى — فحماية التصوير مطبّقة على كل الشاشات بلا استثناء.
- `SafeArea(top: false, left: false, right: false)` — حماية سفلية فقط (شريط الإيماءات)، لأن الحماية العليا تُدار في كل شاشة بحسب تصميمها.

app_content_width.dart

lib/core/widgets/app_content_width.dart

الدور: يقصّ أقصى عرض لمحتوى الشاشة ويوسّطه. موثّق بدقة: الهواتف أضيق من السقف فلا يتغيّر شيء فيها؛ أما على اللوح أو في نافذة مقسّمة فيمنع تمدّد البطاقات إلى «أشرطة نص وصورة مصغّرة تائهة في طرف».

- `defaultMaxWidth = 680`.
- **تعليق مهم على `maxWidth`:** القيمة بالبكسلات المنطقية **وليست بـ `.w` عن قصد** — لأن هذا قرار عن مساحة الجهاز الحقيقية لا قياس تصميم يُقاس بمقياس.

app_section_scaffold.dart ٩٠ سطرًا

lib/core/widgets/app_section_scaffold.dart

الدور: الهيكل الموحّد لكل شاشة فرعية: شريط علوي + محتوى، مع إدارة ذكية للقائمة الجانبية.

- يكتشف **هل يوجد drawer في الـ Scaffold المضيف** (`Scaffold.maybeOf(context)?.hasDrawer`) أم أن هذه الشاشة تملك واحدًا.
- `_menuButton(...)` — يُرجع `null` إن لم يُطلب زر القائمة أو لم يوجد drawer أصلًا، **فلا يظهر زر لا يفتح شيئًا**. وعند وجوده يفتح الـ drawer الصحيح (الخاص أو المضيف).
- **منطق المواضع:** إن وُجد زر رجوع، يصبح زر القائمة **الإجراء الثاني في جهة البداية** (`secondaryLeading`) بدل أن يحلّ محل الأول — والتعليق يوضّح ذلك.
- يغلّف المحتوى بـ `AppContentWidth` فتستفيد كل الشاشات الفرعية من سقف العرض تلقائيًا.
- `Builder` حول الشريط العلوي — لازم للحصول على سياق يستطيع `Scaffold.of` أن يرى الـ Scaffold الجديد.

app_top_bar.dart ٥٥ سطرًا

lib/core/widgets/app_top_bar.dart

الدور: الشريط العلوي: عنوان موسّط مع إجراءات على الجهتين.

- يبني زر الرجوع تلقائيًا من `onBack` إن لم يُمرَّر `leading` صريح.
- `secondaryLeading` — موثّق: يسمح للشاشة بإظهار زر الرجوع **وزر القائمة معًا** بلا أن ينزاح زر القائمة إلى جهة النهاية، «وهو ما يُقرأ كالجهة الخطأ في العربية».
- **حساب الهامش التكيّفي:** `startInset` يصبح `104.w` عند وجود أيقونتين و`56.w` لأيقونة واحدة — فالعنوان يبقى موسّطًا بصريًا ولا يتصادم مع الأزرار.
- `EdgeInsetsDirectional` و`AlignmentDirectional` في كل المواضع — دعم RTL صحيح.
- العنوان `maxLines: 2` + `ellipsis`، و`ConstrainedBox(minHeight: 48.h)`.

app_top_bar_back_button.dart ٣١ سطرًا

lib/core/widgets/app_top_bar_back_button.dart

زر رجوع دائري `44×44` بخلفية `surfaceContainerLowest`، بأيقونة `arrow_back_ios_new`. تركيب `Material + CircleBorder + clipBehavior + InkWell` هو الترتيب الصحيح لتموّج دائري نظيف.

> الأيقونة `arrow_back_ios_new` (سهم لليسار) **تنقلب تلقائيًا** في RTL لأنها من الأيقونات ذات الدعم الاتجاهي في Flutter.

app_bottom_nav_bar.dart ٦٣ سطرًا

lib/core/widgets/app_bottom_nav_bar.dart

الدور: شريط التنقّل السفلي المنحني، مبني على `curved_labeled_navigation_bar`.

- `barHeight` ثابت ساكن (`70.h`) — **يستخدمه** `AppToast` لوضع الإشعار فوق الشريط لا تحته.
- `MediaQuery.withClampedTextScaling(maxScaleFactor: 1)` — **قصّ مقياس النص للعناوين فقط**: تكبير الخط من إعدادات النظام كان سيُفسد تخطيط الشريط. والقصّ محدود بهذه الودجة فلا يؤثّر على بقية الشاشة.
- `DefaultTextStyle.merge(maxLines: 1, ellipsis, center)` — حماية إضافية للعناوين الطويلة.
- يبني العناصر بحلقة على `AppTab.values`، والعنوان من `tab.label.tr()`، والمختار يأخذ لونًا مميّزًا.
- `animationDuration: 400ms` للحركة المنحنية.

bottom_nav_tab_icon.dart ٤٩ سطرًا

lib/core/widgets/bottom_nav_tab_icon.dart

الدور: أيقونة تبويب واحدة بحالتين.

- غير مختار ← أيقونة بسيطة بلون `onSecondary` (خروج مبكر، صفر تكلفة إضافية).
- مختار ← **دائرتان متداخلتان**: هالة خارجية `60` بلون السطح (تقطع الشريط المنحني فتبدو الأيقونة «ناتئة» منه)، وداخلية `48` بلون `secondary`، والأيقونة بلون `tertiaryContainer`.
- الأبعاد ثوابت مسمّاة (`iconSize`, `selectedDiameter`, `haloDiameter`).

app_drawer_menu_button.dart ٢٤ سطرًا

lib/core/widgets/app_drawer_menu_button.dart

زر فتح القائمة الجانبية: `CustomBarIcon` يحوي أيقونة `assetsMenuIcon` بلون `tertiary` وحشوة `13.w`.

custom_bar_icon.dart ٤١ سطرًا

lib/core/widgets/custom_bar_icon.dart

الأساس المشترك لكل أزرار الأشرطة الدائرية: قطر افتراضي `44.w`، خلفية `surfaceContainerLowest`، بظل `elevation: 2` خفيف، و`InkWell`. يقبل أي `child` فيُستخدم للأيقونات والصور والشارات.

١٤٫٣ — الحالات: تحميل، خطأ، فراغ، انتقال

app_state_switcher.dart

`lib/core/widgets/app_state_switcher.dart`

الدور: تلاشٍ متقاطع بين حالات الشاشة الأربع (تحميل / خطأ / فراغ / محتوى). موثّق توثيقًا دقيقًا:

- الأبناء يُطابَقون كما يطابقهم `AnimatedSwitcher`، **فتبديل نوعين مختلفين من الودجات يتحرّك، أما إعادة بناء عادية لنفس النوع فلا** — وهذا بالضبط المطلوب: لا تلاشٍ عند كل تحديث بيانات.
- `StackFit.passthrough` في `layoutBuilder` — يمنح الابن الداخل **والخارج** قيود الأب نفسها، فيحفظ المحتوى القابل للتمرير ارتفاعه الكامل طول مدة التلاشي (بلا ارتجاف).
- تحت «تقليل الحركة» يُعرض الابن مباشرة **بلا مبدّل في الشجرة أصلًا**.
- يستخدم صيغة Dart الحديثة `?currentChild` (عنصر قائمة شرطي للقيمة الفارغة).

app_fade_slide_in.dart ٦٤ سطرًا

`lib/core/widgets/app_fade_slide_in.dart`

الدور: حركة الدخول الوحيدة التي يُعاد استخدامها في التطبيق: تلاشٍ مع انزلاق صغير لأعلى.

- `index` — لتدريج دخول مجموعة عناصر، و`maxStaggerIndex = 8` يوقف نمو التأخير **فلا تستمر قائمة طويلة في الحركة إلى الأبد**.
- **القرار التقني الأهم** (موثّق): التأخير مدمج داخل `Interval` على `tween` واحد **لا في مؤقّت** — «فلا يوجد شيء يُلغى حين يُهدم العنصر في منتصف الحركة». هذا يمنع فئة كاملة من أخطاء `setState after dispose`.
- يحسب `start = delay / total` بالميكروثانية مع حماية القسمة على صفر.
- `rise` — المسافة لأعلى، افتراضها `12.h`.
- تحت «تقليل الحركة» يُرجع الابن فورًا.

app_empty_view.dart ٩٩ سطرًا

`lib/core/widgets/app_empty_view.dart`

الدور: حالة «لا بيانات» الموحّدة: أيقونة + عنوان اختياري + رسالة + زر إجراء اختياري.

- `LayoutBuilder` يحسب `minHeight`: من قيود الأب إن كانت منتهية، وإلا **نصف ارتفاع الشاشة** — فتُتوسَّط الحالة الفارغة بشكل صحيح داخل قائمة قابلة للتمرير بقيود غير محدودة.
- `titleKey` اختياري — موثّق: «لحالة فارغة تحتاج أن تقول ما هو المفقود قبل أن تشرحه».
- `actionLabelKey` + `onAction` — الزر يظهر فقط مع رد النداء، وعنوانه الافتراضي `retry`.
- كل النصوص مفاتيح ترجمة (`messageKey.tr()`).

app_error_view.dart ٧٨ سطرًا

lib/core/widgets/app_error_view.dart

الدور: حالة الخطأ الموحّدة: أيقونة + رسالة + زر إعادة محاولة.

- `errorKey` (مفتاح ترجمة) **أو** `message` (نص جاهز)، والأولوية للثاني، والاحتياط `something_went_wrong`.
- `icon` افتراضها `cloud_off_rounded`، وموثّقة: «تُعرض فوق الرسالة فتُقرأ حالة الخطأ كشقيقة لـ `AppEmptyView` لا كسطر نص عارٍ».
- `onRetry` **مطلوب** — فلا يمكن إنشاء حالة خطأ بلا مخرج للمستخدم. قرار تصميمي جيد يفرضه نظام الأنواع.
- نفس منطق `LayoutBuilder` للتوسيط.

app_toast.dart ١٩٤ سطرًا

lib/core/widgets/app_toast.dart

الدور: نظام الرسائل العابرة في التطبيق — بديل `SnackBar` بتحكّم كامل. يضم `AppToastKind` و`AppToast` و`_ToastView`.

- `_current` — `OverlayEntry` واحد فقط: كل رسالة جديدة **تُزيل السابقة** فلا تتراكم الرسائل.
- `show(context, message, {kind, duration})` — يحصل على الـ Overlay من **مفتاح الـ Navigator العالمي أولًا** ثم من السياق كاحتياط — فتعمل الرسائل من الخدمات التي لا تملك سياقًا (مثل `SessionGuard`).
- `success(...)` و`error(...)` — اختصارات، و`hide()` للإزالة اليدوية.
- `onDismissed` يفحص `identical(_current, entry)` — **حماية دقيقة**: رسالة انتهت مدتها لا تُزيل رسالة أحدث حلّت مكانها.
- `_ToastView` — `StatefulWidget` بـ `AnimationController` (٢٢٠ms): تلاشٍ + انزلاق من الأسفل (`Offset(0, 0.6)`) بمنحنى `easeOutCubic`، ويُعرَض `reverse()` قبل الإزالة فيختفي بسلاسة لا يُبتر.
- الموضع: `bottom: AppBottomNavBar.barHeight + 12.h` — **فوق شريط التنقّل دائمًا**، فلا تُحجب الرسالة.
- قابل للنقر للإخفاء الفوري (`GestureDetector(onTap: _dismiss)`).
- `_icon` و`_accent(colors)` — أيقونة ولون تمييز لكل نوع (معلومة/نجاح/خطأ) مع خلفية موحّدة `secondary`.
- `dispose` يُلغي المؤقّت والمتحكّم، وكل خطوة بعد `await` تفحص `mounted`.

app_progress_bar.dart ٧٦ سطرًا

lib/core/widgets/app_progress_bar.dart

الدور: شريط تقدّم بميزة **طبقتين**.

- `progress` — التقدّم المؤكّد، و`pendingProgress` — تقدّم **معلّق/متوقّع** (لم يُحفظ على الخادم بعد).
- عند وجود تقدّم معلّق أكبر، يُرسم شريطان متراكبان: المعلّق بشفافية ٤٠٪ والمؤكّد بلون كامل — فيرى الطالب تقدّمه الفعلي والتقدّم غير المحفوظ معًا.
- `showPercent` — النسبة بـ `floor()` في جهة النهاية (`AlignmentDirectional.centerEnd`).
- `ClipRRect` بنصف قطر = الارتفاع، فتكون الأطراف نصف دائرية تلقائيًا.

١٤٫٣ — الصور والوسائط

app_network_image.dart ٥٢ سطرًا

lib/core/widgets/app_network_image.dart

الدور: غلاف `CachedNetworkImage` الموحّد — لا تُستخدم `Image.network` في أي مكان.

- `errorWidget` **مطلوب** — فيستحيل نسيان حالة الفشل.
- `memCacheWidth` — تحسين الذاكرة بتفكيك الصورة بعرض العرض الفعلي.
- `cacheKey` — مفتاح كاش مستقلّ عن الرابط، **ضروري هنا** لأن روابط الوسائط الموقّعة تتغيّر كل جلسة، فبلا مفتاح ثابت تُحمَّل نفس الصورة من جديد في كل مرة.
- `fadeInDuration: AppMotion.normal` و`fadeOutDuration: Duration.zero` — ومعهما **تعليق يشرح السبب**: الصور كانت «تظهر فجأة» فصارت تتلاشى داخلة؛ وإبقاء التلاشي الخارج فوريًا يمنع ظهور صورة مستبدلة من خلف التي تحتها.

app_image_viewer.dart ١٥٠ سطرًا

lib/core/widgets/app_image_viewer.dart

الدور: عارض صورة بملء الشاشة: تقريب بالقرص والنقر المزدوج، وسحب للإغلاق، بلا قصّ. موثّق في أعلاه.

- `static open(context, {url, cacheKey, heroTag, caption})` — **التوثيق ينصّ على النقطة المهمة**: مرّر **نفس الرابط** الذي استخدمته الصورة المصغّرة فتأتي البايتات من الكاش مباشرة (بلا تحميل ثانٍ).
- يُرجع فورًا للرابط الفارغ، ويستخدم `rootNavigator: true` فيُغطّي كل شيء.
- `PageRouteBuilder` بـ `opaque: false` و`barrierColor: transparent` — لازم ليعمل السحب للإغلاق مع خلفية متغيّرة الشفافية.
- `PhotoView` بمقاييس محسوبة: `contained` للأدنى والابتدائي و`covered * 3` للأقصى.
- `_zoomed` — يُحدَّث من `scaleStateChangedCallback` و**يُعطّل السحب للإغلاق أثناء التقريب** فلا تتعارك الإيماءتان.
- `heroAttributes` اختياري — انتقال سلس من الصورة المصغّرة.
- زر إغلاق مع `tooltip` مترجم، وتعليق اختياري أسفل الصورة (`maxLines: 3`).
- `_error()` — أيقونة صورة مكسورة + رسالة مترجمة.

drag_dismiss_backdrop.dart ٧٨ سطرًا

lib/core/widgets/drag_dismiss_backdrop.dart

الدور: خلفية داكنة تتبع السحب الرأسي وتُغلق الشاشة عند تجاوز العتبة. موثّقة كقابلة لإعادة الاستخدام في أي عارض وسائط.

- الثوابت: `_dismissThreshold = 130` (مسافة الإغلاق)، `_dragLimit = 400` (أقصى سحب)، `_flingVelocity = 800` (سرعة القذف)، `_minBackdrop = 0.4` (أقل عتمة).
- **طريقتان للإغلاق:** سحب بعيد **أو** قذف سريع — فالمستخدم الذي يسحب قليلًا بسرعة يُغلق أيضًا، وهو توقّع طبيعي.
- **العتمة تتناسب عكسيًا مع السحب** فتُنتج إحساسًا بـ «رفع الصورة من الخلفية»، وبحد أدنى ٤٠٪ فلا تشفّ الخلفية تمامًا.
- `_snapping` — يُستخدم كمفتاح للمدة: `180ms` للرجوع التلقائي و`0` أثناء السحب. **حيلة أنيقة**: نفس الودجة `AnimatedSlide` تتبع الإصبع فورًا وترتدّ بسلاسة.
- `enabled` — يوقف كل شيء أثناء التقريب.

app_icon.dart ٥٧ سطرًا

`lib/core/widgets/app_icon.dart`

الدور: ودجة واحدة لكل أصول الصور في التطبيق، تخفي الفرق بين SVG و PNG.

- **الكشف بامتداد الملف:** `.svg` ← `SvgPicture.asset` مع `ColorFilter(srcIn)`؛ وإلا `Image.asset` مع `colorBlendMode: srcIn`. فالمنادي يمرّر `color` ولا يعرف نوع الملف.
- `size` اختصار لـ `width` و`height` معًا.
- تحسين صغير: إن كانت الحشوة صفرًا **لا يُضاف** `Padding` إلى الشجرة أصلًا.

app_circle_avatar.dart ٢٩ سطرًا

`lib/core/widgets/app_circle_avatar.dart`

صورة مستخدم بديلة: دائرة بخلفية `onSurface` بشفافية ٩٪ وأيقونة `person` بحجم ٥٥٪ من القطر. القطر الافتراضي `50.w`، والأيقونة تتناسب معه تلقائيًا.

١٤,٤ — الأزرار والحقول

custom_button.dart ٨٤ سطرًا

`lib/core/widgets/custom_button.dart`

الدور: الزر الأساسي في التطبيق، وله **سمة بصرية مميّزة**.

- `threeRadius` و`lastRadius` — **التوقيع البصري للتطبيق**: ثلاث زوايا بنصف قطر واحد والزاوية السفلية اليسرى بنصف قطر آخر (افتراضه صفر). فتبدو الأزرار بشكل «قطرة» غير متناظر يميّز هوية التطبيق.
- `isLoading` — **تحميل داخل الزر**: يُستبدل النص بدوّارة **ويُعطَّل** `onPressed` (`isLoading ? null : onPressed`) — التزام بقاعدة «لا تحجب الشاشة لعملية صغيرة» ومنع النقر المزدوج في الوقت نفسه.
- النص `maxLines: 1` + `ellipsis` بنمط `buttonLabel`.
- الدوّارة تأخذ لون النص فتبقى مقروءة على أي خلفية.

> `disabledBackgroundColor: backgroundColor ?? Colors.black` — الاحتياط الأسود قاسٍ بصريًا؛ الأنسب لون من الثيم بشفافية.

custom_outline_button.dart ٨٤ سطرًا

`lib/core/widgets/custom_outline_button.dart`

النسخة المحدّدة بإطار من الزر السابق: نفس نظام الزوايا ونفس `isLoading`، مع `borderColor` و`borderWidth`. ولطف في التصميم: لون النص يرث لون الإطار افتراضيًا (`textColor ?? stroke`) فيكون الزر متناسقًا بمعامل واحد.

custom_text_field.dart ١١٧ سطرًا

lib/core/widgets/custom_text_field.dart

الدور: حقل الإدخال الموحّد، مبني على `TextFormField` (التزامًا بقاعدة المشروع في النماذج).

- يغطي كل ما يحتاجه حقل: `validator`، `onChanged`، `obscureText`، `readOnly`، `keyboardType`، `inputFormatters`، `maxLength`، `maxLines`، الأيقونات، الألوان، الحشوة، نصف القطر، `textInputAction`، `onFieldSubmitted`.
- **أهم معامل:** `textDirection` — وله تعليق توضيحي: يفرض قراءة القيمة المكتوبة من اليسار لليمين **مع بقاء التطبيق بالعربية**. يُستخدم للبيانات اللاتينية فقط (البريد، الهاتف، كلمة المرور) حيث يُعيد خوارزم الاتجاه ترتيب `@` و`.` و`+` والأرقام بشكل غير متوقّع. **والنص التوضيحي ورسالة الخطأ وموضع الأيقونات تستمر في اتباع اتجاه التطبيق.**
- تنفيذ ذلك: `textAlign: isLtr ? TextAlign.left : TextAlign.start` و `hintTextDirection: isLtr ? Directionality.of(context) : null`.
- كل أنواع الإطارات معرّفة (`enabled`، `focused`، `error`، `focusedError`) بنفس نصف القطر، والخطأ بلون `error`.
- `counterText: ''` — يُخفي عدّاد الأحرف المزعج الذي يظهر مع `maxLength`.

> هذا الحقل مثال جيد على مشكلة لغوية حقيقية حُلّت بدقة: كتابة بريد إلكتروني في تطبيق عربي بلا هذا الإصلاح تُنتج ترتيبًا مرئيًا مشوّهًا يُقلق المستخدم رغم صحة القيمة.

phone_number_field.dart ٢٥٢ سطرًا

lib/core/widgets/phone_number_field.dart

الدور: أكبر ودجة حقل في المشروع: حقل هاتف كامل بمنتقي دولة. (أكثر من ١٥٠ سطرًا — مبرَّر لأنه يجمع منتقي الدولة والحقل والتحقّق والتنسيق في وحدة واحدة متماسكة؛ ومع ذلك يمكن تقسيمه إلى `refactor/`.)

يعمل بالتكامل مع: `AppCountries` (البيانات)، `Country` (التنسيق والتحقّق)، `PhoneNumberInputFormatter` (التنسيق أثناء الكتابة)، `CountryPickerSheet` (الاختيار)، و`AppRegex` (التحقّق النهائي). والنتيجة للمنادي: حقل واحد يُرجع رقمًا بصيغة `E.164` جاهزة للخادم.

country_picker_sheet.dart ١٧٥ سطرًا

lib/core/widgets/country_picker_sheet.dart

الدور: ورقة سفلية لاختيار الدولة مع بحث فوري.

- `static show(context, {selected}) → Future<Country?>` — تُرجع الدولة المختارة أو `null` عند الإلغاء.
- `_filter(query)` — يستخدم `country.matches(query)` (المنطق في النموذج لا في الودجة).
- ارتفاع `0.78.sh` ثابت، مع `viewInsets.bottom` فيرتفع مع لوحة المفاتيح.
- حالة فارغة مترجمة (`no_countries_found`) عند عدم وجود نتائج.
- `_CountryTile` — ودجة خاصة: **العلم** (المحسوب حسابيًا من رمز الدولة)، الاسم باللغة الحالية، ورمز الاتصال **مع** `textDirection: TextDirection.ltr` **مفروضًا** — فيُعرض `+963` لا `963+`. والمختار يُبرَز بـ `selectedTileColor`.
- ملاحظة في الاستيراد: `import ... hide TextDirection;` — لتفادي تعارض الاسم بين `easy_localization` و Flutter.

١٤٫٥ — الحوارات والأوراق

app_confirm_dialog.dart ١٠٩ أسطر

lib/core/widgets/app_confirm_dialog.dart

الدور: حوار التأكيد العام (النسخة المعمّمة من `LogoutConfirmDialog`).

- `static show(...) → Future<bool>` — تُرجع `false` دائمًا عند الإلغاء أو النقر خارجًا.
- كل النصوص **مفاتيح**: `titleKey` , `messageKey` , `confirmKey` , `cancelKey` (افتراضها `cancel`).
- `messageArgs` — يُمرَّر إلى `tr(namedArgs:)` فتُدرج قيم في الرسالة («هل تريد حذف {name}؟») بلا دمج نصي يدوي يكسر الترجمة.
- `iconPath` افتراضه أيقونة الحذف، ويُعرض في `AppDialogGlyph` .
- يستخدم `AppDialogActions` للأزرار — فالحوار نفسه ٣ أجزاء مركّبة.

app_dialog_glyph.dart ٤١ سطرًا

lib/core/widgets/app_dialog_glyph.dart

الأيقونة المربّعة في ترويسة الحوارات: `38×38` بزوايا `10.r` ، خلفيتها افتراضيًا لون الخطأ بشفافية ٧٥٪ (فتُقرأ الحوارات كتنبيهات)، وداخلها `AppIcon` بحجم `20.w` . الألوان قابلة للتجاوز لحوار غير تحذيري.

app_dialog_actions.dart ٦٣ سطرًا

lib/core/widgets/app_dialog_actions.dart

صف أزرار الحوار الموحّد: زر تأكيد مملوء (بلون الخطأ افتراضيًا، قابل للتجاوز بـ `confirmColor`) وزر إلغاء محدّد، كلاهما في `Expanded` بعرض متساوٍ وارتفاع `44.h` .

> ترتيب «تأكيد ثم إلغاء» موحّد في كل حوارات التطبيق — والاتساق هنا أهم من أي ترتيب بعينه، لأنه يبني ذاكرة عضلية صحيحة لدى المستخدم.

coming_soon_sheet.dart ٩٩ سطرًا

lib/core/widgets/coming_soon_sheet.dart

الدور: ورقة «قريبًا» للميزات غير المكتملة.

- `static show(context, {messageKey})` — والرسالة قابلة للتخصيص لكل ميزة.
- التركيب: `SheetHandle` ، دائرة `84.w` بخلفية `tertiary` ٢٥٪ فيها أيقونة صاروخ، العنوان، الرسالة، ثم زر إغلاق بعرض كامل.

> أفضل بكثير من زر لا يفعل شيئًا أو رسالة Toast عابرة: المستخدم يفهم أن الميزة **موجودة في الخطة** لا معطوبة.

sheet_handle.dart ١٩ سطرًا

lib/core/widgets/sheet_handle.dart

المقبض الصغير أعلى الأوراق السفلية: `42×4` بزوايا دائرية ولون `onSurfaceVariant` بشفافية ٥٠٪. إشارة بصرية عالمية إلى «هذه الورقة قابلة للسحب».

drag_dismiss_backdrop.dart / custom_bottom_sheet.dart

(انظر `drag_dismiss_backdrop.dart` أعلاه في قسم الوسائط، و`utils/custom_bottom_sheet.dart` في فصل المساعدات.)

١٤,٦ — الشارات والزخارف

app_count_badge.dart ٣٨ سطرًا

`lib/core/widgets/app_count_badge.dart`

- `if (count <= 0) return child;` — **خروج مبكر**: بلا عدد، بلا `Stack` ولا أي تكلفة إضافية في الشجرة.
- `max = 99` ⟵ تُعرض `99+` فلا يتمدّد الشكل مع الأعداد الكبيرة.
- `clipBehavior: Clip.none` مع `PositionedDirectional(top: -2.h, end: -2.w)` — الشارة تخرج قليلًا عن حدود الابن **في جهة النهاية الصحيحة** بحسب الاتجاه.

count_badge_bubble.dart ٣٧ سطرًا

`lib/core/widgets/count_badge_bubble.dart`

الفقاعة نفسها: حد أدنى `18×18` (فتبقى دائرية لرقم واحد وتتمدّد لرقمين)، خلفية `error`، **وإطار بلون السطح بعرض** `1.5` — وهذا الإطار هو ما يجعل الشارة مقروءة فوق أي خلفية أو أيقونة. النص `10.sp` بوزن ٧٠٠ وتباعد `1.1` للتوسيط الرأسي الدقيق.

brand_badge.dart ٣١ سطرًا

`lib/core/widgets/brand_badge.dart`

شعار التطبيق في دائرة: قطر افتراضي `56.w`، خلفية `secondary`، حشوة **نسبية** (`diameter * 0.24`) فيبقى الشعار متناسبًا عند أي حجم، واللون `tertiary`.

card_ornament.dart ٣٢ سطرًا

`lib/core/widgets/card_ornament.dart`

الدور: زخرفة واحدة داخل بطاقة.

- **انعكاس أفقي بحسب الاتجاه**: في LTR يُطبَّق `Matrix4.diagonal3Values(-1, 1, 1)` (قلب أفقي) وفي RTL تُترك كما هي. فالزخرفة المصمَّمة للعربية تبدو صحيحة في الإنجليزية أيضًا بلا ملف ثانٍ.
- اللون `onSurface` بشفافية ٨٪ — خفيفة جدًا بحيث لا تنافس المحتوى.

ornamented_card.dart ٧٦ سطرًا

lib/core/widgets/ornamented_card.dart

الدور: البطاقة المزخرفة — العنصر البصري الأكثر تميّزًا في التطبيق. كلاسان:

- `OrnamentedCard` — حاوية بخلفية `tertiaryContainer` وزوايا `12.r` و`clipBehavior: Clip.antiAlias`، فيها طبقة زخرفة ثم المحتوى. الزخرفتان الافتراضيتان: `assetsOrnamentRight` و`assetsOrnamentBottomLeft`، وكلتاهما قابلة للتعطيل بتمرير `null`.
- `OrnamentBackdrop` — طبقة الزخرفة وحدها (قابلة للاستخدام المستقلّ)، مغلّفة بـ `IgnorePointer` **فلا تعترض الزخرفة نقرات المستخدم** — تفصيل بسيط لكن نسيانه يُنتج بطاقة لا تستجيب للنقر.

index_strip.dart ٢٦ سطرًا

lib/core/widgets/index_strip.dart

شريط رقم الترتيب في جانب بطاقة الدرس: عرض أدنى `26.w`، خلفية `tertiary`، والرقم في الأعلى. يعطي قوائم الدروس إحساس «المنهج المرقّم».

status_header.dart ٣٢ سطرًا

lib/core/widgets/status_header.dart

ترويسة شاشات الحالة (قيد المراجعة / مرفوض / تمت الموافقة): عنوان بنمط `statusTitle` ثم ملاحظة بنمط `note`، كلاهما موسّط.

educational_path_text.dart ٥٥ سطرًا

lib/core/widgets/educational_path_text.dart

الدور: عرض المسار التعليمي (الجهة ← المرحلة ← الصف) كمسار تنقّل.

- يستقبل `segments` (من `MyEnrollmentsModel.activePathNames`) و**يعكسها** (`segments.reversed`) — فيُعرض الأخصّ (الصف الحالي) أولًا ثم الآباء.
- **الفاصل يتغيّر بالاتجاه:** `‹` في RTL و`›` في LTR — فيشير السهم دائمًا نحو جهة التدرّج الصحيحة.
- `i == 0` يأخذ نمط «الحالي» (`labelStrong` بلون `secondary`) والبقية نمط «السلف» (`bodyMedium` باهت).
- `Wrap` لا `Row` — فالمسار الطويل ينزل لسطر ثانٍ بدل أن يفيض.

capped_seek_bar.dart ٨٣ سطرًا

lib/core/widgets/capped_seek_bar.dart

الدور: شريط تنقّل **بسقف** — الميزة التربوية الأساسية في التطبيق: **الطالب لا يستطيع تخطّي ما لم يشاهده**.

- `maxPosition` — السقف المسموح، و`onSeek` للانتقال، و`onBlocked` يُنادى حين يحاول المستخدم تجاوز السقف (فتُظهر الشاشة رسالة توضيحية)، و`onInteraction` لإبقاء الضوابط ظاهرة.
- `_dragSeconds` — **قصّ أثناء السحب نفسه**: المقبض يتوقّف عند السقف فورًا بدل أن يسمح بالسحب ثم يرتدّ — تغذية راجعة فورية وواضحة.
- `secondaryTrackValue: cap` — يرسم Flutter مسارًا ثانويًا حتى السقف، **فيرى المستخدم المنطقة المتاحة بصريًا** قبل أن يحاول.
- `onChangeEnd` — يصفّر حالة السحب، ينادي `onBlocked` إن حاول التجاوز، ثم يُجري الانتقال **بالقيمة المقصوصة**.
- حماية الحالات الحدّية: `total <= 0` يُعطّل الشريط (`onChanged: null`) ويضع `max: 1` فلا يرمي `Slider` استثناء.
- `SliderTheme` مخصّص: ارتفاع `4.h`، مقبض `6.r`، و`noOverlay` (بلا هالة ضغط).

١٤,٧ — بطاقات الدروس (١٤ ودجة)

> بطاقة الدرس مثال نموذجي على **التركيب** (composition): بدل ملف واحد بمئتي سطر، هناك ١٤ ودجة صغيرة يُركّب بعضها على بعض — وكل واحدة قابلة لإعادة الاستخدام وحدها، ولكل منها هيكل عظمي مقابل.

lesson_card.dart ٧٦ سطرًا

lib/core/widgets/lesson_card.dart

الدور: البطاقة الكاملة للدرس، تُركّب خمس ودجات في `Stack`.

- `OrnamentBackdrop` خلفية، ثم صف: `LessonThumbnail` + `LessonCardDetails`.
- `LessonMediaTag` في جهة البداية أسفل البطاقة، و`LessonSaveIcon` أعلى البداية.
- **سلوك الدرس المقفل:** `onTap: isLocked ? () => _reportLocked(context) : onTap` — النقر على درس مقفل **يُظهر رسالة** «الدرس مقفل» بدل أن لا يفعل شيئًا. فرق كبير في التجربة: المستخدم يعرف أن النقرة وصلت وأن هناك سببًا.
- أيقونة الحفظ تُخفى للدرس المقفل (لا معنى لحفظ ما لا يمكن مشاهدته).
- `IntrinsicHeight` + `CrossAxisAlignment.stretch` — فتتمدّد الصورة المصغّرة لارتفاع النص بأي طول.

lesson_card_details.dart ٨٨ سطرًا

lib/core/widgets/lesson_card_details.dart

الدور: الجزء النصي من البطاقة، وكل أقسامه شرطية — فلا فراغات لبيانات غائبة.

- صف العنوان + `LessonStatusBadge` (العنوان في `Expanded` بسطرين كحد أقصى).
- الوصف يظهر فقط إن وُجد، بسطر واحد.
- صف البيانات الوصفية: التاريخ (إن وُجد) والمدة (إن كانت موجبة)، والفاصل بينهما يظهر **فقط إن وُجد الاثنان** — تفصيل دقيق يمنع مسافة شاردة.
- شريط التقدّم يظهر فقط حين `lesson.status.showsProgress` (أي «قيد التقدّم») — فلا شريط صفري لدرس لم يُبدأ ولا شريط ممتلئ لدرس مكتمل.
- كل عنصر وصفي في `Flexible` — فالتاريخ الطويل لا يدفع المدة خارج البطاقة.

lesson_thumbnail.dart ٥٠ سطرًا

lib/core/widgets/lesson_thumbnail.dart

الصورة المصغّرة: عرض افتراضي `104.w`، زوايا `10.r`، خلفية `surfaceContainerLowest`. بلا رابط ← `LessonFallbackArt` مباشرة؛ ومع رابط ← `AppNetworkImage` بنفس البديل في حالة الخطأ. وتُضاف طبقة `LessonLockScrim` فوقها للدرس المقفل.

lesson_fallback_art.dart ١٤ سطرًا

lib/core/widgets/lesson_fallback_art.dart

بديل الصورة: أيقونة الكتاب بحشوة `10.w` و`BoxFit.contain`. ودجة من أربعة أسطر لكن وجودها يضمن أن **كل** موضع تغيب فيه صورة يبدو متّسقًا.

lesson_lock_scrim.dart ٣٢ سطرًا

lib/core/widgets/lesson_lock_scrim.dart

طبقة القفل فوق الصورة: لون شبه شفاف (٥٥٪) **يتغيّر بحسب نوع المحتوى** — `primary` للفيديو و`secondary` لغيره — وأيقونة قفل `40.w` بلون النص المناسب لكل خلفية.

lesson_media_tag.dart ٣٤ سطرًا

lib/core/widgets/lesson_media_tag.dart

شريط نوع الوسيط في زاوية البطاقة. **الشكل مميّز**: `BorderRadiusDirectional.only(topEnd: 64.r)` مع حشوة غير متناظرة (`fromSTEB(16, 30, 28, 8)`) فيُنتج شكلًا منحنيًا «يخرج» من زاوية البطاقة. واستخدام `Directional` في الزاوية والحشوة يجعله ينقلب صحيحًا في LTR.

lesson_status_badge.dart ٣٧ سطرًا

lib/core/widgets/lesson_status_badge.dart

شارة حالة الدرس. تُرجع `SizedBox.shrink()` إن لم يكن للحالة عنوان، وتأخذ لونيها من `status.badgeColors(colors)` (الزوج المضمون التباين من `lesson_view_data.dart`). **صفر منطق في الودجة**: كل القرارات في التعداد وامتداده.

lesson_source_badge.dart ٣٤ سطرًا

lib/core/widgets/lesson_source_badge.dart

شارة مصدر الوسيط (ملف / يوتيوب) بنفس بنية الشارة السابقة تمامًا وبنفس آلية الألوان.

lesson_meta_item.dart ٤٤ سطرًا

lib/core/widgets/lesson_meta_item.dart

عنصر بيانات وصفية: أيقونة `15.w` + نص. و`isIconTrailing` يبدّل الترتيب فتُستخدم نفس الودجة في تصميمات تريد الأيقونة بعد النص. النص في `Flexible` مع `ellipsis`.

lesson_save_icon.dart ٧٠ سطرًا

lib/core/widgets/lesson_save_icon.dart

الدور: أيقونة الحفظ/الإزالة — **الاستثناء الوحيد المقصود** من قاعدة «الودجة غبية»: تصل إلى Cubit مشترك مباشرة.

- `getIt<LessonSaveCubit>()` — الـ Cubit `lazySingleton` مشترك، فتتزامن حالة الحفظ في **كل** القوائم في التطبيق فورًا.
- `buildWhen` يقارن `isSaved(lessonId)` **لهذا الدرس فقط** — فحفظ درس في قائمة من ٥٠ درسًا يُعيد بناء أيقونة واحدة لا خمسين.
- `_toggle` — ينادي الـ Cubit، **يفحص** `context.mounted` **بعد** `await`، ثم يُظهر Toast «تم الحفظ / أُزيل».
- `Tooltip` بنص مترجم متغيّر بالحالة — إضافة جيدة لإمكانية الوصول.
- الأيقونة تتغيّر شكلًا (`bookmark` ↔ `bookmark_border`) ولونًا معًا — تمييز لا يعتمد على اللون وحده.

> الوصول المباشر للـ Cubit هنا مبرَّر معماريًا: البديل هو تمرير الحالة ورد النداء عبر كل بطاقة وكل قائمة وكل شاشة — وهو تسريب للحالة في كل الطبقات. أيقونة الحفظ «مكوّن ذكي» (smart component) بوعي.

lesson_category_chip.dart ٥٩ سطرًا

lib/core/widgets/lesson_category_chip.dart

رقاقة فئة واحدة بحالتين: المختارة مملوءة بـ `secondary` بلا إطار، وغير المختارة شفافة بإطار `outline` بعرض `1.2.w`. `StadiumBorder` للشكل الكبسولي، والأيقونة والعنوان من التعداد وامتداده.

lesson_category_chips.dart ٣٨ سطرًا

lib/core/widgets/lesson_category_chips.dart

الشريط الأفقي للرقاقات: `SizedBox(height: 44.h)` يحوي `ListView.separated` أفقيًا — فيعمل التمرير إن زادت الفئات، والفواصل `10.w`. يبني من `LessonCategory.values`.

lesson_subject_header.dart ٧٤ سطرًا

lib/core/widgets/lesson_subject_header.dart

ترويسة المادة أعلى قائمة دروسها: `IndexStrip` (إن كان الترتيب موجبًا) + `OrnamentBackdrop` + `BrandBadge(64.w)` + اسم المادة + `AppProgressBar`. و`order` اختياري يتراجع إلى `subject.position`.

١٤,٨ — بطاقات المواد والجلسات

subject_card.dart ٩٤ سطرًا

lib/core/widgets/subject_card.dart

الدور: بطاقة المادة في الشبكة. **لاحظ التصميم المعماري:** تستقبل `title` و`subtitle` و`progress` و`order` كقيم **خام** لا كائن `SubjectModel`.

> هذا جعل البطاقة قابلة لإعادة الاستخدام مع أي مصدر بيانات (مادة، نتيجة بحث، عنصر أرشيف) — فصل نظيف عن النموذج. والثمن: على المنادي استخراج القيم. موازنة صحيحة لودجة بهذا الانتشار.

التركيب: `IndexStrip` + `OrnamentBackdrop` + `BrandBadge(58.w)` + عنوان + عنوان فرعي + شريط تقدّم، كلها داخل `InkWell`.

live_session_tile.dart ٥٠ سطرًا

lib/core/widgets/live_session_tile.dart

بطاقة جلسة البث: `OrnamentedCard` يحوي صورة الغلاف (عبر `LessonThumbnail` بنوع `video` — **إعادة استخدام ذكية**) و`LiveSessionDetails`.

live_session_details.dart ٨٦ سطرًا

lib/core/widgets/live_session_details.dart

تفاصيل الجلسة: العنوان، الوصف (شرطي)، صف التاريخ والوقت (شرطي، عبر `LiveFormats` و`LessonMetaItem` المُعاد استخدامها)، ثم زر الإجراء في جهة النهاية.

live_session_action_button.dart ٥٠ سطرًا

lib/core/widgets/live_session_action_button.dart

الدور: زر الجلسة بحالتين: **مباشرة الآن** ← زر مملوء «شاهد الآن»؛ **غير مباشرة** ← زر محدّد «ذكّرني لاحقًا».

> **تفصيل ممتاز في إمكانية الوصول:** `width = 132.w * MediaQuery.textScalerOf(context).scale(1)` — عرض الزر **ينمو مع مقياس خط المستخدم**. فلا يُقصّ النص بـ `ellipsis` لمن يستخدم خطًا كبيرًا. وهذا حل نادر الاستخدام رغم بساطته.

١٤,٩ — كلمة المرور وسرعة التشغيل

password_requirements_checklist.dart ٤٨ سطرًا

lib/core/widgets/password_requirements_checklist.dart

قائمة متطلبات كلمة المرور: حاوية بخلفية خفيفة وعنوان، ثم حلقة على `PasswordRequirement.values` تسأل كل متطلب `isSatisfiedBy(password)`. **بلا أي منطق تحقّق في الودجة** — كل شيء في التعداد.

password_requirement_row.dart ٦١ سطرًا

lib/core/widgets/password_requirement_row.dart

- `Semantics(checked: isSatisfied)` — **قارئ الشاشة يعلن «محقّق/غير محقّق»**، فلا تعتمد المعلومة على رؤية الأيقونة.
- `AnimatedSwitcher(180ms)` مع `ValueKey<bool>` — تبديل سلس بين دائرة فارغة وعلامة صح. المفتاح ضروري وإلا لم يرَ المبدّل تغييرًا.
- اللون واحد للأيقونة والنص (`tone`) فيتبدّل السطر كله معًا.
- العنوان يُترجم مع `namedArgs: requirement.labelArgs` — فيُدرج الطول الأدنى في النص.

playback_speed_button.dart ٦٣ سطرًا

lib/core/widgets/playback_speed_button.dart

زر السرعة الحالية: كبسولة بأيقونة `speed_rounded` والعنوان. `onOverlay` يبدّل اللوحة بين «فوق فيديو» (خلفية سوداء ٤٥٪، نص أبيض) و«في الواجهة العادية» (ألوان الثيم). والعنوان بـ `textDirection: TextDirection.ltr` مفروضًا فيُعرض `1.5x` لا `x1.5`.

playback_speed_sheet.dart ١٨٠ سطرًا

lib/core/widgets/playback_speed_sheet.dart

الدور: ورقة اختيار السرعة، وهي **الواجهة التي تنفّذ القاعدتين التربوية والتقنية** المعرّفتين في `PlaybackSpeed`.

- `static show(...)` — تُغلق الورقة **قبل** تنفيذ الاختيار (`pop()` ثم `onSelected`) فلا تتعارض الحركة مع تغيير السرعة.
- `_noteKey` — **بأولوية واضحة**: جهاز ضعيف ← `playback_speed_device_note`؛ وإلا درس غير مكتمل ← `playback_speed_locked_note`؛ وإلا لا ملاحظة. فتُعرض **ملاحظة واحدة** تشرح السبب الأهم لا رسالتان متزاحمتان.
- `_Option` — عنصر واحد بثلاث حالات مرئية، و`isLocked` يُحسب من قاعدتين معًا: `needsCompletedLesson && !isLessonCompleted` أو `exceedsDevice(...)`.
- `trailing` بنمط `switch` على **زوج** (record pattern): `(true, _)` ← قفل؛ `(false, true)` ← صح؛ `(false, false)` ← لا شيء. استخدام أنيق لمطابقة أنماط Dart 3.
- الخيار المقفل: `onTap: null` (معطّل فعلًا) **ونصه بشفافية ٤٠٪** — فهو مرئي ومفهوم لا مخفي. وهذا هو الصواب: المستخدم يعلم أن السرعة موجودة ويعلم شرط إتاحتها.
- «١×» يُعرض بنص مترجم «عادي» (`playback_speed_normal`) لا `1x`.

١٤,١٠ — عائلة التحميل `loading/` (١٦ ملفًا)

نظام هياكل عظمية (skeletons) مكتمل، يتبع قواعد المشروع في التحميل حرفيًا: **لا دوّارة عامة لتحميل أول شاشة**، بل هيكل **يحاكي التخطيط الحقيقي**؛ والدوّارة محصورة في الأزرار؛ وكل شيء يحترم تفضيل «تقليل الحركة». البنية: ثلاث **بدائيات** (`AppShimmer` ، `SkeletonBox` ، `SkeletonTextLine`)، ومساعدان (`AppSkeletonList` ، `AppInlineLoader`)، و١١ هيكلًا مركّبًا في `skeletons/`.

app_shimmer.dart ١٦١ سطرًا

lib/core/widgets/loading/app_shimmer.dart

الدور: محرّك اللمعان. ثلاثة كلاسات بتصميم أدائي ذكي:

- `AppShimmerDriver` — `InheritedWidget` يحمل حركة كنس واحدة لكل ما تحته. موثّق: «فتعمل قائمة من صفوف الهياكل على **مؤقّت واحد** بدل مؤقّت لكل صف».
- `AppShimmerGroup` — يملك الحركة (`AnimationController` بمدة `AppMotion.shimmer`) ويبثّها عبر الـ Driver. وتحت «تقليل الحركة» **تُوقَف الحركة ولا يُجدول أي إطار**.
- `AppShimmer` — الودجة العملية:
  - تستخدم حركة السلف إن وُجدت، **وتهدم متحكّمها الخاص** عند وجودها (`_disposeController()`) فلا يعمل مؤقّتان.
  - وإلا تنشئ متحكّمًا خاصًا — فتعمل الودجة وحدها بلا مجموعة.
  - الاحتياط `AlwaysStoppedAnimation(0.5)` — لمعان ثابت في الوسط حين لا حركة.
- `ShaderMask` بـ `BlendMode.srcATop` و`LinearGradient` من ثلاث محطات (شفاف ⟵ ٥٥٪ ⟵ شفاف).
- **الكنس يحترم الاتجاه:** `begin`/`end` ينقلبان في RTL، فيمرّ اللمعان من اليمين لليسار في العربية.
- حساب `head = -band + sweep * (1 + 2*band)` مع تعليق: «يسير من خارج حافة إلى خارج الحافة الأخرى تمامًا» — فلا تظهر بداية أو نهاية مفاجئة للمعان.

> **قيمة هذا التصميم:** شاشة فيها ١٥ صف هيكل عظمي تعمل بمؤقّت واحد و`ShaderMask` لكل صف، بدل ١٥ `AnimationController`. الفرق ملموس على الأجهزة الضعيفة — وهو بالضبط الجهاز الذي يرى الهيكل العظمي أطول.

skeleton_box.dart ٤٢ سطرًا

lib/core/widgets/loading/skeleton_box.dart

الدور: الكتلة البديلة الواحدة. موثّقة: تطلي لونًا **معتمًا مشتقًّا من الثيم** فيكون للمعان ما يمرّ فوقه.

- `Color.alphaBlend(colors.onSurface.withValues(alpha: 0.10), surface ?? colors.surface)` — **الفكرة الدقيقة**: يُمزج اللون **مسبقًا** مع سطح الخلفية فتكون النتيجة لونًا معتمًا لا شفافًا. الكتلة الشفافة كان `ShaderMask` سيُظهر ما تحتها.
- `surface` — موثّق: «اللون الذي تجلس عليه هذه الكتلة، حين لا يكون سطح الـ Scaffold». وهذا ما يجعل الهياكل داخل البطاقات الملوّنة تبدو صحيحة.
- `isCircular` للصور الدائرية، و`radius` افتراضه `8.r`.

skeleton_text_line.dart ٥٦ سطرًا

lib/core/widgets/loading/skeleton_text_line.dart

الدور: سطر نص بديل — **وأدقّ ملف في عائلة التحميل**. التوثيق يشرح الفكرة بوضوح: يحتجز **صندوق السطر الكامل** الذي سيشغله النص الحقيقي (`fontSize` × معامل السطر) ويطلي شريطًا **أقصر** داخله. «الشريط هو الجزء المرئي، والصندوق هو الجزء الذي يجب أن يتطابق».

- `_lineFactor = 1.5` — معلّق عليه: «خط Cairo يُرسم بنحو هذا المضاعف من حجمه».
- ارتفاع الشريط `fontSize * 0.78` وارتفاع الصندوق `fontSize * 1.5`.
- `widthFactor` (كسر من العرض) أو `width` ثابت، والثاني له الأولوية.
- `AlignmentDirectional.centerStart` — الشريط يبدأ من جهة بداية القراءة.

> **هذه هي نقطة الفرق** بين هيكل عظمي جيد وآخر مزعج: لو احتجز الهيكل ارتفاع **الشريط** فقط، لاهتزّ التخطيط كله لحظة وصول النص. هذا الملف يقيس صندوق السطر الحقيقي، فلا إزاحة ولا قفزة في موضع التمرير.

app_skeleton_list.dart ٤٧ سطرًا

lib/core/widgets/loading/app_skeleton_list.dart

الدور: سلسلة صفوف هياكل متطابقة. موثّق أنها «معمّمة من هيكل سجل التحميلات الذي كان يفعل هذا بشكل صحيح أصلًا» — أي أن النمط استُخرج من ممارسة قائمة لا اختُرع.

- يغلّف القائمة بـ `AppShimmerGroup` فـ «عدد الصفوف لا يضاعف المؤقّتات أبدًا».
- `itemBuilder` **بلا معامل فهرس** — موثّق: «الصفوف متبادلة بحكم التعريف، فلا فهرس يُمرَّر». تصميم واجهة يمنع سوء الاستخدام.
- `physics` افتراضها `NeverScrollableScrollPhysics` — **الافتراض الصحيح**: الهيكل لا يُمرَّر (لا شيء لرؤيته أسفله) ولا يتعارك مع التمرير الأب.

app_inline_loader.dart ٢٨ سطرًا

lib/core/widgets/loading/app_inline_loader.dart

دوّارة محصورة في صندوقها، لإجراء بدأه المستخدم لحظتها. والتوثيق **ينصّ على القاعدة صراحة**: «تنتمي داخل زرّ أو صف. وحالة التحميل الأولى لأي شاشة تكون هيكلًا عظميًا، **لا هذه أبدًا**». قطر افتراضي `20.w` وسماكة `2.2`.

skeletons/lesson_card_skeleton.dart ٩٨ سطرًا

lib/core/widgets/loading/skeletons/lesson_card_skeleton.dart

يحاكي `LessonCard`: الصورة `104.w`، صف العنوان والشارة، سطر الوصف، وصف البيانات الوصفية. ويمرّر `surface: tertiaryContainer` لكل كتلة فتتناسب مع خلفية البطاقة الحقيقية.

skeletons/lesson_cards_skeleton.dart ٣٠ سطرًا

…/skeletons/lesson_cards_skeleton.dart

سلسلة غير قابلة للتمرير من بطاقات الدروس. والتوثيق يحدّد حالة الاستخدام بدقة: «لاستخدامها داخل قائمة تُمرَّر أصلًا — عند تبديل فئة أو تبويب، **حيث انهيار القائمة إلى دوّارة موسّطة كان سيُقفز موضع التمرير**».

skeletons/subject_card_skeleton.dart ٩٣ سطرًا

…/skeletons/subject_card_skeleton.dart

يحاكي `SubjectCard`: شريط الترتيب، شارة العلامة، العنوان، العنوان الفرعي، التقدّم. وموثّق: «كل قياس منسوخ من البطاقة الحقيقية فلا يتزحزح شيء عند الوصول». ويلاحظ في تعليقاته أن `IndexStrip` الحقيقي شريط معتم، فيُحاكى كذلك.

skeletons/live_session_tile_skeleton.dart ٩٤ سطرًا

…/skeletons/live_session_tile_skeleton.dart

يحاكي `LiveSessionTile` كاملًا: الصورة، العنوان، الوصف، صف التاريخ والوقت، وزر الإجراء في الطرف.

skeletons/notification_card_skeleton.dart ١٠٢ أسطر

…/skeletons/notification_card_skeleton.dart

يحاكي بطاقة الإشعار: شريط التمييز (`_barWidth = 4`)، مربّع النوع (`_tileSize = 40`)، صف العنوان، أسطر المتن. وفيه تعليق يوضّح أن البطاقة الحقيقية تستخدم `IntrinsicHeight` ليتمدّد شريط التمييز داخل قائمة غير محدودة — **«ومُحاكى هنا للسبب نفسه»**. هذا مستوى عالٍ من الدقة في المطابقة.

skeletons/lesson_subject_header_skeleton.dart ٨٤ سطرًا

…/skeletons/lesson_subject_header_skeleton.dart

يحاكي `LessonSubjectHeader`: شريط الترتيب، شارة العلامة `64.w`، اسم المادة، وشريط التقدّم (عبر `ProgressBarSkeleton` المُعاد استخدامه).

skeletons/category_chips_skeleton.dart ٣٥ سطرًا

…/skeletons/category_chips_skeleton.dart

يحاكي `LessonCategoryChips`. و**تفصيل بصري لطيف**: العروض `[104, 86, 96, 110]` متفاوتة — وموثّق: «العروض تتفاوت كما تتفاوت العناوين الحقيقية، **فلا يُقرأ الشريط كشبكة**». الهيكل المتطابق تمامًا يبدو آليًا؛ والتفاوت الطفيف يبدو كمحتوى قادم.

skeletons/media_tabs_skeleton.dart ٣٨ سطرًا

…/skeletons/media_tabs_skeleton.dart

يحاكي صف تبويبات الوسائط المقسّم: حاوية بحشوة `8.w` وزوايا `12.r`، فيها `tabCount` كتلة بارتفاع `38.h` موزّعة بـ `Expanded`.

skeletons/section_header_skeleton.dart ٥٦ سطرًا

…/skeletons/section_header_skeleton.dart

يحاكي ترويسة القسم: العنوان، وعنوان فرعي اختياري (`hasSubtitle`)، ورقاقة «عرض الكل» اختيارية (`hasAction`) — فالمنادي يطابق الهيكل مع شكل ترويسته بالضبط.

skeletons/progress_bar_skeleton.dart ٣٨ سطرًا

…/skeletons/progress_bar_skeleton.dart

يحاكي `AppProgressBar`: نص النسبة في جهة النهاية (عرض `30.w`) ثم المسار بارتفاع `6.h`. و`showPercent` و`surface` يطابقان معاملات الأصل. **وهو الهيكل الوحيد بلا** `AppShimmer` **خاص** — لأنه يُستخدم دائمًا **داخل** هيكل آخر يملك اللمعان أصلًا.

> **ملاحظة على حالة المجلد:** شجرة `lib/core` كانت تُعدَّل أثناء إعداد هذا المستند (جلسة تحرير أخرى تعمل على تحسينات واجهة وتحميل). الأعداد والمحتوى في هذا الملف تعكس حالة الشجرة لحظة القراءة: **٢٣٠ ملفًا** موزّعة كالتالي — `app`: ٤، `constants`: ٤، `di`: ١، `drawer`: ١٢، `enums`: ٣١، `interceptors`: ٢، `models`: ٢٠، `network`: ٦، `routing`: ٧، `services`: ٤٣، `themes`: ٧، `utils`: ١٨، `widgets`: ٧٥.

١٥ — خلاصة: نقاط القوة وفرص التحسين

أبرز نقاط القوة المعمارية

- **مركزية القرارات:** قرار «هل انتهت الجلسة؟» في `ErrorMapper` فقط، وتنفيذه في `SessionGuard` فقط. وتعيين الأخطاء في مكان واحد. والمفاتيح والمسارات والأصول كل منها في ملف واحد.
- **مقاومة المستقبل:** كل `fromJson` في التعدادات لها قيمة افتراضية آمنة؛ و`Json.*` لا ترمي أبدًا؛ وقرار الجلسة مبني على **رمز** الرفض لا على قائمة حالات. فنوع إشعار جديد أو حالة حساب جديدة لا تكسر هذا البناء.
- **التعدادات الغنية:** لا `switch` على الأنواع في الواجهة؛ السلوك يسكن مع التعداد، والعرض في امتداداته (`lesson_view_data.dart`).
- **دعم RTL جدّي:** `Directional` في كل المواضع، وانعكاس الزخارف، وانتقال محايد للاتجاه، و`textDirection` مفروض للبيانات اللاتينية، وفواصل مسار تتبدّل بالاتجاه. وفي الوقت نفسه **فرض LTR حيث يجب** (شريط الزمن، إيماءات ±١٠ ثوانٍ، رموز الاتصال).
- **إمكانية الوصول:** احترام «تقليل الحركة» في أربعة مواضع، و`Semantics` في قائمة المتطلبات، وقصّ مقياس النص **محليًا** حيث يلزم بدل إلغاء تفضيل المستخدم، وعرض زر ينمو مع الخط.
- **نظام تحميل مكتمل:** هياكل عظمية تطابق التخطيط الحقيقي قياسًا بقياس، بمؤقّت واحد مشترك، ودوّارات محصورة في الأزرار.
- **تعليقات تشرح «لماذا» لا «ماذا»:** في `ScreenWakeService`، `ErrorMapper`، `SessionGuard`، `AccountStatusWatcher`، `SkeletonTextLine`، `AppChrome` وغيرها — وهي أعلى قيمة توثيقية في المشروع.
- **أداء مدروس:** `buildWhen` دقيق، `memCacheWidth`، `DownloadProgressThrottle`، كاش صور مضبوط حسب قوة الجهاز، `ValueNotifier` للضوابط، ودمج الطلبات المتزامنة في ثلاثة مواضع.

فرص التحسين المرصودة

- **حرج:** `print(token)` في `DioService._storedToken()` — يطبع توكن المصادقة في سجل الإصدار. يجب حذفه.
- **متوسّط:** غياب `lib/core/localization/lang_keys.dart` — كل مفاتيح الترجمة نصوص حرّة (`'home'.tr()`) بخلاف قواعد المشروع، فالمفتاح الخاطئ يظهر للمستخدم بلا أي تحذير من المحلّل.
- **متوسّط:** `settings_action_enum.dart` يستورد بـ `package:bilhikma/...` داخل `lib/` — خرق لقاعدة الاستيراد النسبي.

- **متوسّط:** `AuthInterceptor` و`DioService` و`DeviceSessionService` تستخدم النصوص الخام `'token'` و`'refresh_token'` بدل `CacheKeys`.
- **صغير:** عدم اتساق في السجل: `debugPrint` في `ScreenCaptureService` و`DeviceInfoService` و`AppInfoService` و`CurrentUser` بدل `AppLogger`.
- **صغير:** `orientation_util.dart` — فرعا الشرط يُرجعان نفس النتيجة، والتفريق بين الهاتف واللوح معطّل فعليًا.
- **صغير:** تناقض عتبة طول كلمة المرور (٦ في `AppRegex` مقابل ٨ في حساب القوة).
- **صغير:** ملفات تتجاوز سقف ١٥٠ سطرًا تستحقّ التقسيم: `app_router.dart` (٣٠٦)، `phone_number_field.dart` (٢٥٢)، `app_toast.dart` (١٩٤)، `playback_speed_sheet.dart` (١٨٠)، `country_picker_sheet.dart` (١٧٥)، `app_shimmer.dart` (١٦١)، `video_controls_overlay.dart` (٢٨٥).
- **صغير:** إعادة تسمية `prd_services.dart` إلى `pdf_service.dart`، وتصحيح أخطاء مطبعية في أسماء الأصول.
- **صغير:** قيم مكتوبة ثابتة في مسار `AppRoutes.review` تبدو بقايا تطوير.

> **الحكم العام:** هذه طبقة أساسية مبنية بعناية فوق المتوسّط بكثير. الالتزام بالفصل بين الطبقات متّسق، والقرارات الصعبة (الجلسة، حماية المحتوى، التحميل، الاتجاه، إمكانية الوصول) مُعالجة بوعي ومُوثّقة بأسبابها. الملاحظات أعلاه كلها قابلة للمعالجة في جلسة عمل واحدة، وأهمّها على الإطلاق: **حذف** `print(token)` وإضافة `LangKeys`.